AWS CREATIVE LEARNING

Prepared by DevsArun | https://devsarun.io/

Cloud ko picture, kahani aur real-life examples ke saath samjho. Har concept ko apni online shop ke safar se jodo.

CLOUD FOUNDATIONS

001 What is Cloud Computing

Seedhi baat: Cloud internet ke through on-demand compute, storage, databases aur software resources use karne ka model hai. Provider physical facilities operate karta hai; customer apne workload ke liye resources provision karta hai.

TASVEER SE SAMJHO

Dukaan + cloud

Dukaan ko website chalane ke liye computer, photos rakhne ke liye storage aur orders ke liye database chahiye. Cloud in cheezon ko zarurat ke hisaab se use karne ka rasta deta hai. Sale badhne par capacity badhane ki planning ki ja sakti hai.

REAL-LIFE EXAMPLE

Riya ki online shop festival sale ke liye extra servers launch kar sakti hai aur demand girne par hata sakti hai.

YAAD RAKHO

Cloud resources cost-free ya automatically secure nahi hote: permissions, monitoring aur cleanup zaroori hain.

002 Traditional IT vs Cloud Computing

Seedhi baat: Traditional setup mein hardware kharidna, install karna aur capacity ka advance estimate lagana padta hai. Cloud mein supported capacity API se provision ho sakti hai aur charges selected pricing model se aate hain.

REAL-LIFE EXAMPLE

Apni shop ke liye warehouse kharidne aur demand ke hisaab se space rent karne ka difference socho.

YAAD RAKHO
Cloud bhi fixed commitments, minimum charges aur operations cost la sakta hai; total cost compare karo.

003 Types of Cloud: Public, Private, Hybrid

Seedhi baat: Public cloud mein provider resources multiple customers ke liye offer karta hai, with logical isolation. Private cloud organisation ke dedicated environment mein hota hai. Hybrid mein cloud aur existing private/on-prem systems connected rehte hain.

REAL-LIFE EXAMPLE
Storefront cloud par, lekin legacy inventory system company ke data center mein: hybrid example.

YAAD RAKHO
Dedicated environment ko automatically compliant ya hybrid connection ko automatically reliable mat samjho.

004 Service Models: IaaS, PaaS, SaaS

Seedhi baat: IaaS mein virtual machines aur networks jaise infrastructure configure karte ho. PaaS mein application chalane ke platform ke kai operations provider handle karta hai. SaaS mein ready-made application use hoti hai.

TASVEER SE SAMJHO
Service models ko workshop ki tarah samjho

Baayen building blocks: infrastructure tum zyada configure karte ho. Beech ki workshop: platform kuch kaam sambhalta hai. Daayen ready shop: ready software use hota hai. Ye sirf analogy hai; asli responsibilities service par depend karti hain.

REAL-LIFE EXAMPLE
EC2 par khud app aur OS manage karna IaaS ke closer hai; managed application platform PaaS-like aur hosted email SaaS example hai.

YAAD RAKHO
Service labels approximate hain: actual operational responsibility product aur configuration se decide hoti hai.

005 Advantages of Cloud Computing

Seedhi baat: Fast provisioning, demand-based capacity adjustment, global reach aur facilities management ka burden kam hona cloud ke potential advantages hain. Large provider scale kuch efficiencies la sakta hai.

REAL-LIFE EXAMPLE
Sale se pehle extra capacity add karo, sale ke baad scale down karke unused resources hatao.

YAAD RAKHO
Benefit architecture, quotas, app readiness aur cost management par depend karta hai; automatic savings guarantee nahi.

006 Disadvantages of Cloud Computing

Seedhi baat: Cloud mein ongoing charges, misconfiguration risk, internet/network dependencies, migration effort aur vendor-specific features se lock-in ho sakta hai. Data residency aur service quotas bhi constraints ban sakte hain.

REAL-LIFE EXAMPLE
Galat setup se idle resources ka bill badh sakta hai aur network outage se remote application unavailable ho sakti hai.

YAAD RAKHO
Design mein exit strategy, cost controls, resilience aur security responsibility shuru se rakho.

007 Cloud Use Cases: Real-world examples

Seedhi baat: Web hosting, backup, data analytics, machine learning aur disaster recovery cloud workloads ke examples hain. Pehle business outcome aur constraints samjho, phir services choose karo.

REAL-LIFE EXAMPLE
Online store static images S3 se serve kare, app compute par chale aur orders database mein store hon; analytics pipeline sales trends bataye.

YAAD RAKHO
Ek hi reference architecture sab workloads ke liye sahi nahi: compliance, cost aur recovery objectives vary karte hain.

008 Introduction to AWS

Seedhi baat: Amazon Web Services cloud platform hai jisme compute, storage, databases, networking aur security jaise building blocks milte hain. AWS ne internet-based infrastructure services 2006 mein dena shuru kiya.

REAL-LIFE EXAMPLE
App owner compute aur database choose karta hai, IAM access define karta hai aur operational metrics monitor karta hai.

YAAD RAKHO
AWS products ki availability aur feature lifecycle time ke saath badalte hain; deployment se pehle current documentation dekho.

009 AWS Global Infrastructure Overview

Seedhi baat: AWS infrastructure Regions, Availability Zones aur edge locations mein organised hai. Ye concepts workload placement, resilience aur user proximity samajhne mein help karte hain.

REAL-LIFE EXAMPLE

Application ko ek Region ki multiple AZs mein deploy karo; edge delivery se geographically distributed users ke close content cache ho sakta hai.

YAAD RAKHO

Diagram conceptual hai: backup, multi-AZ replication aur disaster recovery automatically enable nahi hote.

010 Regions & Availability Zones

Seedhi baat: Region geographical area hai; har Region mein multiple Availability Zones hoti hain. AZ isolated infrastructure location/failure domain hoti hai, jo ek se adhik data centers involve kar sakti hai.

REAL-LIFE EXAMPLE

App ke instances alag AZs mein rakhna single-AZ failure risk kam karta hai; database aur network dependencies bhi assess karo.

YAAD RAKHO

Subnet ek hi AZ mein hota hai. Multi-AZ placement se zero downtime guaranteed nahi hota.

011 Edge Locations & CDN

Seedhi baat: Content delivery network cached content users ke kareeb edge locations se serve karke origin se network distance aur repeated load kam kar sakta hai. CloudFront AWS ka CDN hai.

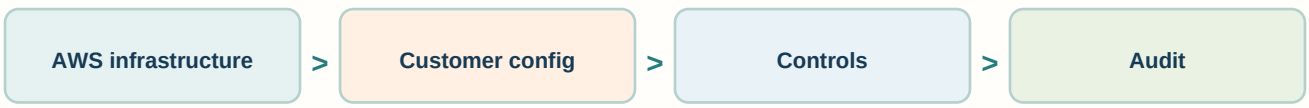
REAL-LIFE EXAMPLE

Product images CloudFront edge se serve karo; cache miss par request configured origin jaise S3 tak ja sakti hai.

YAAD RAKHO

Edge location Region ya AZ nahi hai. Cache invalidation, access permissions, headers aur cost design karo.

012 Shared Responsibility Model



Seedhi baat: AWS cloud ki underlying infrastructure security ka responsible hai; customer apne workload ki identity, data aur settings ke liye responsible rehta hai. Exact split service ke hisaab se change hota hai.

TASVEER SE SAMJHO
Suraksha ki do zimmedariyan

Baayen provider data center aur physical hardware protect karta hai. Daayen customer apni identities, data aur settings protect karta hai. Managed service mein exact division alag ho sakta hai; bucket permissions customer ko hi check karni hoti hain.

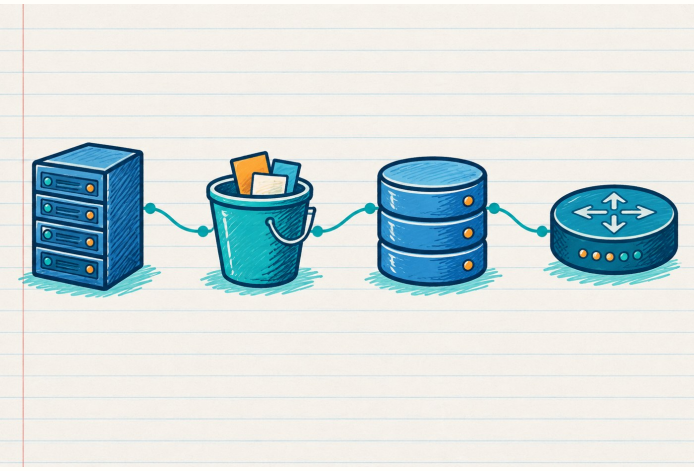
REAL-LIFE EXAMPLE
EC2 guest OS patching customer sambhalta hai; managed database mein provider kuch engine operations handle kar sakta hai.

YAAD RAKHO
Physical data-center security sahi ho to bhi public bucket policy ya stolen credentials se exposure ho sakta hai.

AWS CORE SERVICES

013 What is AWS

Seedhi baat: AWS ko cloud building-blocks ki library samjho: compute code chalata hai, storage files rakhta hai, database structured queries serve karta hai aur networking traffic connect karti hai.



TASVEER SE SAMJHO
Cloud ke building blocks

Server compute ka idea hai, bucket object storage ka, cylinder database ka aur router network ka. Yeh parts milkar app bana sakte hain; picture koi fixed order ya required architecture nahi dikhati.

REAL-LIFE EXAMPLE
Ek shop EC2, S3, RDS, VPC aur IAM ko combine karke application bana sakti hai.

YAAD RAKHO
Service use karne ke liye account, identity permissions, Region aur billing controls samajhna zaroori hai.

014 AWS Service Categories

Seedhi baat: Services ko problem ke hisaab se group karo: compute, storage, databases, networking, security, management, analytics aur AI/ML. Category naam solution choose karne ka starting point hai.

REAL-LIFE EXAMPLE
Image archive ke liye storage; transactional order processing ke liye database; monitoring ke liye observability service compare karo.

YAAD RAKHO
Ek category mein kai products alag trade-offs rakhte hain; service label se hi final decision mat lo.

015 Compute Services Overview

Seedhi baat: EC2 virtual servers deta hai; Lambda event-driven functions ke liye hai; ECS/EKS containers orchestrate karte hain; Fargate supported containers ke liye server management abstract karta hai.

REAL-LIFE EXAMPLE
Long-running custom process ke liye EC2 aur short event-driven task ke liye Lambda compare kar sakte ho.

YAAD RAKHO
Execution limits, runtime, networking, operational ownership aur cost compare karo; serverless ka matlab server nahi hai aisa nahi.

016 Storage Services Overview

Seedhi baat: S3 object storage hai; EBS EC2 block volumes provide karta hai; EFS shared file interface deta hai. Backup aur archive storage alag recovery aur retention needs address karte hain.

REAL-LIFE EXAMPLE
Product photos S3 mein, EC2 boot disk EBS par, aur compatible shared Linux files EFS par ho sakti hain.

YAAD RAKHO
Object, block aur file APIs interchangeable nahi; durability, performance aur charges separately assess karo.

017 Database Services Overview

Seedhi baat: RDS managed relational database engines, Aurora compatible relational platform aur DynamoDB key-value/document access patterns ke options hain. Graph, time-series aur cache solutions bhi milte hain.

REAL-LIFE EXAMPLE
Order aur payment relationships ke liye relational model; high-scale key lookup ke liye DynamoDB evaluate karo.

YAAD RAKHO
Data model, query patterns, consistency, availability aur operational requirements choose karne se pehle define karo.

018 Networking Basics in AWS

Seedhi baat: VPC isolated virtual network hai jisme CIDR, subnets, route tables aur security controls configure hote hain. Internet gateway, NAT, endpoints, VPN aur Direct Connect alag connectivity needs address karte hain.

REAL-LIFE EXAMPLE
Public load balancer se private application tier tak traffic allow karo; private subnet ko public se alag route do.

YAAD RAKHO
Private subnet ka matlab automatic internet access ya automatic security nahi; routes aur policies check karo.

019 Security Basics in AWS

Seedhi baat: Identity, least privilege, network boundaries, encryption, logging aur response ko layers mein design karo. AWS infrastructure protect karta hai aur customer apne resources ki configuration sambhalta hai.

REAL-LIFE EXAMPLE
Workforce role se limited access do, MFA use karo, sensitive data encrypt karo aur CloudTrail events review karo.

YAAD RAKHO
Single security group ya encryption setting complete security strategy nahi; regular validation aur incident plan chahiye.

020 Billing Basics

Seedhi baat: AWS charges service-specific usage, allocation, requests, data transfer aur applicable commitments par depend karte hain. Billing console aur Cost Explorer se costs samajho.

REAL-LIFE EXAMPLE
Tag ke through test environment cost allocate karo; month end se pehle idle resources aur transfer charges review karo.

YAAD RAKHO
Estimates actual bill nahi hote. Tax, Region, storage retention aur running resources ki charges check karo.

021 AWS Free Tier

Seedhi baat: Free Tier offers eligibility, account plan, limits aur expiration terms ke according vary karti hain. Sign-up ke time applicable current AWS terms aur account billing console check karo.

REAL-LIFE EXAMPLE
Sandbox banane se pehle estimate aur budget alert set karo; har lab ke baad resource list dekhkar cleanup karo.

YAAD RAKHO
Budget alert hard spending cap nahi; Free Tier ke bahar usage aur unsupported resources par charges aa sakte hain.

022 AWS Management Console

Seedhi baat: Console browser-based interface hai jahan account aur selected Region mein supported resources inspect aur configure kiye ja sakte hain.

REAL-LIFE EXAMPLE
EC2 dashboard mein Region choose karke instance state aur security group inspect karo.

YAAD RAKHO
Console ke current account, Region aur user identity confirm karo; production changes approval aur audit ke saath karo.

023 AWS CLI Overview

Seedhi baat: AWS CLI commands se supported AWS APIs call karke resources inspect aur automate kar sakte ho. Credentials, configuration profiles aur Region execution context define karte hain.

REAL-LIFE EXAMPLE
Pehle caller identity aur Region check karo; phir read-only listing command se resources dekho.

YAAD RAKHO
Access keys shell history ya code mein expose mat karo; temporary credentials aur least privilege prefer karo.

024 SDKs Overview

Seedhi baat: AWS SDKs programming language ki libraries provide karte hain jinke through applications AWS APIs call karti hain. Authentication ke liye supported credential provider chains aur roles use hote hain.

REAL-LIFE EXAMPLE
Application SDK se S3 object retrieve kare, retry/error handling implement kare aur response validate kare.

YAAD RAKHO
Secrets source code mein hard-code mat karo; SDK retries, timeouts aur IAM permissions explicitly plan karo.

025 Resource Tagging & Documentation Usage

Seedhi baat: Tags key-value metadata hain jo supported resources ko owner, environment ya project se associate kar sakte hain. Cost allocation ke liye relevant tag activation aur coverage check karni hoti hai.

REAL-LIFE EXAMPLE
Environment=dev aur Owner=team tag se cleanup aur cost review organise karo; official docs se supported features validate karo.

YAAD RAKHO
Tags IAM ya billing classification ko har situation mein automatically enforce nahi karte; sensitive data tags mein mat rakho.

IAM

026 What is IAM

Seedhi baat: IAM AWS resources ke access ko manage karne ka mechanism hai. Principal request karta hai; policy evaluation action, resource aur request context ko consider karti hai.

TASVEER SE SAMJHO
IAM: pehchaan aur permission

Gatekeeper ki tarah pehle dekho kaun request kar raha hai, phir uski permission check karo. Neeche wali visitor badge temporary role credentials ka idea dikhati hai. Real request mein resource aur conditions bhi evaluate hote hain.

REAL-LIFE EXAMPLE
App role ko sirf ek bucket ke specific objects read karne ki permission do.

YAAD RAKHO
Authentication identity verify karti hai; authorization decide karti hai ki request allowed hai ya nahi.

027 IAM Users

Seedhi baat: IAM user account ke andar ek identity hai jiske credentials aur permissions configure kiye ja sakte hain. Human workforce ke liye federation aur temporary credentials generally prefer karo.

REAL-LIFE EXAMPLE
Agar legacy integration ko IAM user chahiye, permission scope aur access-key rotation/review plan karo.

YAAD RAKHO
Root credentials ko daily work mein use mat karo; unused users aur long-lived keys risk badhate hain.

028 IAM Groups

Seedhi baat: IAM group users ka collection hai jis par permissions policies attach karke multiple users ko manage kiya ja sakta hai. Group khud principal nahi hai.

REAL-LIFE EXAMPLE
Developers group ko development environment ki permissions do, individual users ko group membership se access mile.

YAAD RAKHO
Group nesting aur membership rules documentation ke hisaab se check karo; group permissions excessive na hon.

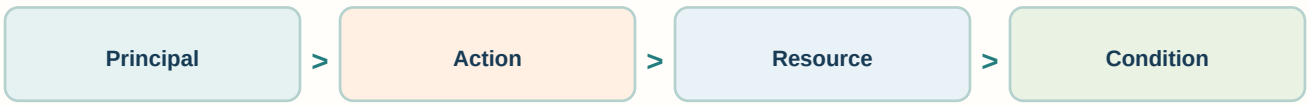
029 IAM Roles

Seedhi baat: Role ek assumable identity hai jiske permission policies aur separate trust policy hoti hai. AssumeRole ya supported service integration se temporary credentials mil sakte hain.

REAL-LIFE EXAMPLE
EC2 application role assume karke limited S3 access le; static access key instance par store na karo.

YAAD RAKHO
Trust policy who can assume define karti hai; permission policy role session kya kar sakta hai define karti hai.

030 Policies: JSON Explained



Seedhi baat: Policy document mein Statement, Effect, Action, Resource aur optional Condition use hote hain. Allow ya Deny specific requests par apply hota hai; policy type ke hisaab se Principal bhi ho sakta hai.

TASVEER SE SAMJHO

Policy: kaun, kya aur kis par

Visitor principal hai; checklist policy ka idea hai; locker resource hai. Guard yeh dekhta hai ki kaunsi action allow hai aur kya conditions puri hain. Explicit deny applicable ho to request deny hoti hai.

REAL-LIFE EXAMPLE

Ek statement S3 GetObject action ko selected bucket prefix par allow kare aur condition se context restrict kare.

YAAD RAKHO

JSON valid hone se permission safe nahi ho jaati. Wildcards aur resource ARN/action mismatch test karo.

```
{ "Effect": "Allow",
"Action": "s3:GetObject",
"Resource": "arn:aws:s3:::example-bucket/reports/*" }
```

031 Permissions Deep Dive

Seedhi baat: Evaluation request ki identity, resource, policies aur context par depend karti hai. Default implicit deny hota hai; applicable explicit deny allow par priority rakhta hai.

REAL-LIFE EXAMPLE

AccessDenied troubleshoot karte waqt identity policy, resource policy, SCP, boundary, session aur KMS permissions inspect karo.

YAAD RAKHO

Har principal/resource policy scenario ka ek universal one-line intersection formula sahi nahi; AWS evaluation rules dekho.

032 IAM Best Practices

Seedhi baat: Least privilege, temporary credentials, MFA, centralized workforce access aur regular access review ko default banao. Root user ko sirf required tasks ke liye protect karo.

REAL-LIFE EXAMPLE

Team members federation se role assume karein; app ko narrowly scoped role mile; unused access periodically remove karo.

YAAD RAKHO

Permissions grant karne se pehle business need, scope aur expiration define karo; policy simulator aur real tests useful hain.

033 Multi-Factor Authentication (MFA)

Seedhi baat: MFA password ke alawa ek additional authentication factor use karta hai. Supported methods aur account policy ke according device register karo.

REAL-LIFE EXAMPLE
Root user aur workforce sign-in par MFA lagao; lost device aur recovery procedure securely document karo.

YAAD RAKHO
MFA stolen active sessions ya overly broad permissions ka complete replacement nahi hai.

034 IAM Roles vs Users

Seedhi baat: User identity ke paas persistent credentials ho sakte hain; role assume hota hai aur temporary session credentials de sakta hai. Typical workforce aur workload patterns roles par rely karte hain.

REAL-LIFE EXAMPLE
Engineer federation se role assume kare; EC2 application instance role use kare.

YAAD RAKHO
Role trust policy aur permission policy dono verify karo. Role ki temporary credentials bhi sensitive hoti hain.

035 Temporary Credentials

Seedhi baat: Temporary access key, secret aur session token limited session duration ke liye milte hain, typically role assumption ke through. Expiration exposure window ko reduce kar sakti hai.

REAL-LIFE EXAMPLE
Workload SDK credential provider se role credentials obtain aur refresh kare instead of embedded static keys.

YAAD RAKHO
Temporary credentials leak ho sakti hain; least privilege, session duration aur monitoring phir bhi zaroori hain.

036 Identity Federation

Seedhi baat: Federation external identity provider se authentication aur AWS role-based access ko connect karti hai. IAM Identity Center supported workforce access centralise karne ka option hai.

REAL-LIFE EXAMPLE
Company directory se sign-in karo aur permitted account/role choose karo; employment end par identity disable karo.

YAAD RAKHO
Identity provider trust, role mappings, MFA aur offboarding regular review karo.

037 AWS Organizations

Seedhi baat: Organizations multiple AWS accounts ko centrally group aur govern karne mein help karta hai. Organisational units, consolidated billing aur policies supported capabilities hain.

REAL-LIFE EXAMPLE
Production, development aur security accounts ko appropriate OUs mein rakho aur centrally review karo.

YAAD RAKHO
Account isolation helpful hai lekin network, IAM aur resource policies ki individual configuration bhi chahiye.

038 Service Control Policies (SCP)

Seedhi baat: SCP member accounts ke principals ke available permissions par organisational guardrails define karte hain; SCP khud permission grant nahi karta. Applicable scope aur exceptions documentation ke according evaluate karo.

REAL-LIFE EXAMPLE
Organisation selected Regions ke bahar supported actions restrict kare; workload role ko alag IAM allow bhi chahiye.

YAAD RAKHO
Management account aur service-linked role behavior alag ho sakta hai; pehle non-production OU mein test karo.

039 IAM Security Risks

Seedhi baat: Long-lived leaked keys, broad wildcard permissions, unsafe trust policies aur unused identities common risks hain. Privilege escalation paths bhi assess karne chahiye.

REAL-LIFE EXAMPLE
Access key repository mein dikhe to incident process ke according revoke/rotate, activity review aur exposure scope investigate karo.

YAAD RAKHO
Secrets ko chat, tickets ya screenshots mein paste mat karo; log evidence preserve karo.

040 IAM Real-world Setup + Interview Questions

Seedhi baat: Workforce identity provider se sign-in, MFA aur role-based access; workloads ke liye separate service roles; account guardrails aur central audit logs. Break-glass access ko test aur monitor karo.

REAL-LIFE EXAMPLE
Interview: trust policy aur permissions policy mein farq? Trust assume karne wale principals batati hai; permissions allowed actions define karti hain.

YAAD RAKHO
AccessDenied ka reason identify kiye bina admin access dena solution nahi; request context aur explicit deny check karo.

EC2

041 What is EC2

Seedhi baat: Amazon EC2 resizable virtual compute instances provide karta hai. Instance type, AMI, storage, network aur security settings select karke workload run karte ho.

TASVEER SE SAMJHO
EC2 ke saath kaun se parts sochne hain

Server instance ke saath image, disk, network protection aur monitoring bhi plan karo. Chitra inko yaad karne ka visual map hai, deployment steps ka exact order nahi.

REAL-LIFE EXAMPLE
Online store ki backend service EC2 instance par chal sakti hai aur ALB se traffic le sakti hai.

YAAD RAKHO
AWS underlying infrastructure sambhalta hai; guest OS patches, app aur relevant security configuration customer ki responsibility hai.

042 EC2 Instance Types

Seedhi baat: Instance families different ratios of CPU, memory, storage aur networking provide karti hain; sizes capacity badalte hain. General-purpose, compute-optimized aur memory-optimized categories common hain.

REAL-LIFE EXAMPLE
Memory-heavy cache aur CPU-heavy batch processing ke liye alag families benchmark karo.

YAAD RAKHO
Family name se workload performance guarantee nahi; architecture, price, quotas aur Region support verify karo.

043 Amazon Machine Image (AMI)

Seedhi baat: AMI EC2 instance launch karne ka image template hai jisme supported boot configuration aur software setup ho sakta hai. Image ko version, access aur lifecycle ke saath manage karo.

REAL-LIFE EXAMPLE
Approved hardened AMI se identical application servers launch karke repeatable deployment banao.

YAAD RAKHO
Old image mein vulnerable software ya embedded secrets ho sakte hain; patch aur image provenance check karo.

044 Launching EC2 Step-by-step

Seedhi baat: Account aur Region confirm karo; AMI aur instance type choose karo; VPC/subnet, role, security group, storage aur access method configure karke launch karo. Phir status checks aur application health verify karo.

REAL-LIFE EXAMPLE
Sandbox mein temporary instance launch karke authorised access test karo; exercise ke baad instance aur billable resources clean up karo.

YAAD RAKHO
Public SSH ko duniya ke liye open karna avoid karo; Session Manager ya restricted access evaluate karo.

045 Key Pairs

Seedhi baat: EC2 key pair supported login flows mein public/private key cryptography use karta hai. Public key instance launch process mein place hoti hai aur private key ko owner securely protect karta hai.

REAL-LIFE EXAMPLE
Linux SSH login mein private key ka use ho sakta hai; team aur workload needs ke according access method choose karo.

YAAD RAKHO
Private key share, commit ya chat mein paste mat karo; lost-key recovery aur revocation procedure plan karo.

046 Security Groups

Seedhi baat: Security group instance/network interface ke liye stateful virtual firewall hai. Inbound aur outbound rules protocol, port aur source/destination define karte hain; response traffic statefully tracked hota hai.

REAL-LIFE EXAMPLE
ALB security group se app port allow karo; database port sirf application security group se accept karo.

YAAD RAKHO
Security groups allow rules use karte hain. Har port ko 0.0.0.0/0 dena least privilege nahi.

047 Elastic Block Storage (EBS)

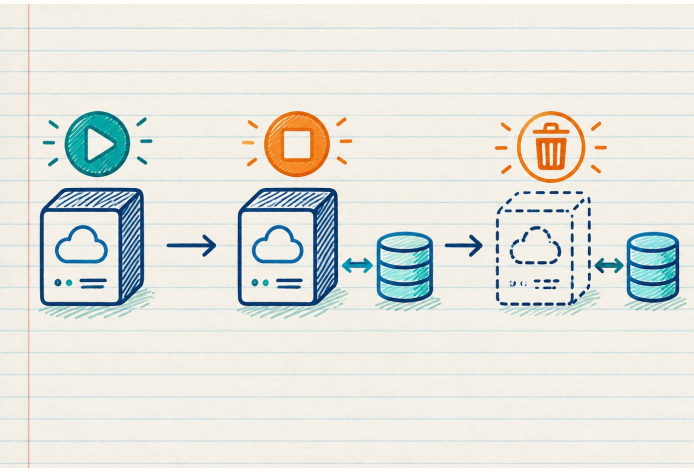
Seedhi baat: EBS EC2 ke liye persistent block volumes provide karta hai. Volume type, size, IOPS, throughput, encryption aur AZ placement configure hote hain.

REAL-LIFE EXAMPLE
EC2 par EBS volume attach karke filesystem banao; snapshots se recovery plan karo.

YAAD RAKHO
EBS volume ek AZ se associated hota hai; snapshot aur restore testing cross-AZ resilience planning ka hissa hain.

048 Instance Lifecycle

Seedhi baat: Instance pending, running, stopping, stopped, shutting-down aur terminated states se guzar sakta hai. Stop aur terminate ka storage, IP aur billing par alag effect ho sakta hai.



TASVEER SE SAMJHO
Start, stop aur terminate

Running instance kaam karta hai. Stopped EBS-backed instance ko dobara start kiya ja sakta hai; terminated instance ko waise wapas start nahi karte. Volume delete hoga ya bachega, uski setting par depend karta hai.

REAL-LIFE EXAMPLE
Maintenance ke liye EBS-backed instance stop karo, settings check karke start karo; termination se pehle backups verify karo.

YAAD RAKHO
Stopped instance ka EBS ya public IPv4 jaise resources chargeable reh sakte hain; instance store data persistence assume mat karo.

049 Auto Scaling Basics

Seedhi baat: EC2 Auto Scaling group desired capacity maintain aur configured policies ke according instances add/remove kar sakta hai. Launch template instance settings define karta hai.

REAL-LIFE EXAMPLE
Traffic badhne par additional app instances launch; demand kam hone par scale in, subject to policy and limits.

YAAD RAKHO
Scaling strategy ko application statelessness, boot time, health checks aur database capacity ke saath test karo.

050 Load Balancers Overview

Seedhi baat: Elastic Load Balancing incoming traffic ko healthy targets mein distribute karta hai. Application Load Balancer HTTP/HTTPS aur Network Load Balancer transport-layer use cases address karte hain.

REAL-LIFE EXAMPLE
Users ALB se connect karein; ALB application instances ko health checks ke basis par traffic bheje.

YAAD RAKHO
Load balancer unhealthy database ya poorly designed application ko magically reliable nahi bana deta; availability aur timeouts test karo.

051 EC2 Pricing Models

Seedhi baat: On-Demand flexible usage, Reserved Instances selected commitments, Savings Plans eligible compute commitments aur Spot spare capacity use karne ke pricing choices hain.

REAL-LIFE EXAMPLE
Steady baseline ke liye commitment evaluate karo, unpredictable burst On-Demand aur interruption-tolerant batch Spot par consider karo.

YAAD RAKHO
Actual rates, scope, commitment terms, interruption risk aur Region-specific price current pricing page se verify karo.

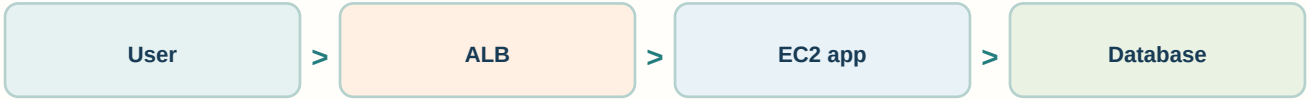
052 EC2 Monitoring: CloudWatch

Seedhi baat: CloudWatch EC2 metrics collect aur alarms trigger kar sakta hai; custom metrics/logs ke liye agent ya application configuration lag sakti hai.

REAL-LIFE EXAMPLE
CPU utilisation aur application error rate monitor karo; sustained failure par alert aur runbook trigger ho.

YAAD RAKHO
Default metrics har application symptom nahi batate. Memory/disk visibility aur alarm thresholds separately configure karo.

053 EC2 Architecture Diagram



Seedhi baat: Basic web path: user → public load balancer → application instances in multiple AZs → protected database. IAM roles, logs aur monitoring side controls hain.

REAL-LIFE EXAMPLE
Instance failure simulate karke health check, target replacement aur application response observe karo.

YAAD RAKHO
Har subnet ek AZ mein hota hai; app aur data layers ke failure modes aur deployment costs independently evaluate karo.

054 EC2 Real-world Example

Seedhi baat: Online shop sale ke time traffic badhta hai. ALB healthy app instances ko requests deta hai; Auto Scaling capacity adjust karta hai; CloudWatch operational signals deta hai.

REAL-LIFE EXAMPLE
Order state shared database mein rakho, local disk par solely depend na karo; workload tests se scaling thresholds choose karo.

YAAD RAKHO
More app instances database bottleneck solve nahi karte; load testing aur cost guardrails chahiye.

055 EC2 Common Mistakes + Revision

Seedhi baat: Common issues: public admin ports, hard-coded credentials, incorrect IAM role, outdated AMI, missing backup aur idle resources. Verify caller, account, Region aur resource state.

REAL-LIFE EXAMPLE
Interview: security group stateful hai? Haan. Kya stopped instance ke saare charges zero? Zaroori nahi.

YAAD RAKHO
Cleanup mein EBS volumes, snapshots, Elastic IPs aur other billable resources separately check karo.

S3

056 What is S3

Seedhi baat: Amazon S3 object storage hai jahan data objects buckets mein keys ke through store hota hai. APIs aur permissions ke through upload, retrieval aur management hota hai.

TASVEER SE SAMJHO
S3: bucket ke andar objects

Bucket ko container samjho, har photo ya document ek object. Clock version/retention ki yaad dilata hai aur shield protection ki. Archive aur permissions apni configuration se aate hain; picture automatic backup ka wada nahi karti.

REAL-LIFE EXAMPLE
Product photos, backups aur analytics datasets ko alag prefixes aur access policies se organise karo.

YAAD RAKHO
S3 object API traditional mounted filesystem jaisi semantics nahi deti; permissions aur charges assess karo.

057 Buckets & Objects

Seedhi baat: Bucket objects ka container hai; object mein data, key aur metadata hota hai. Prefix folder jaisa nazar aa sakta hai lekin object keys flat namespace ka part hain.

REAL-LIFE EXAMPLE
Bucket `shop-assets` mein `products/123.jpg` ek object key hai.

YAAD RAKHO
Bucket names aur Region rules check karo; sensitive data bucket names ya metadata mein unnecessarily na rakho.

058 Storage Classes

Seedhi baat: S3 classes access frequency, retrieval, latency aur resilience needs ke hisaab se differ karti hain. Standard, Intelligent-Tiering, infrequent access aur Glacier family ke options hain.

REAL-LIFE EXAMPLE
Frequently accessed photos Standard mein; infrequent archive ko suitable Glacier tier mein evaluate karo.

YAAD RAKHO
Minimum storage duration, retrieval fees aur One Zone trade-off estimate karo; availability current docs se check karo.

059 Versioning

Seedhi baat: Versioning enabled bucket mein object ki multiple versions maintain ho sakti hain. Accidental overwrite/delete ke baad relevant version recover karne mein help mil sakti hai.

TASVEER SE SAMJHO
Purani versions aur archive

Ek photo ki alag versions sambhalne se overwrite ke baad purani copy dhoondhne mein madad mil sakti hai. Lifecycle rules kuch versions transition ya expire kar sakti hain, agar sahi se configure ki gayi hon.

REAL-LIFE EXAMPLE
Product image ka naya upload galat ho to older version identify karke restore karo.

YAAD RAKHO
Versioning backup, cross-account isolation ya lifecycle policy ka substitute nahi; old versions billable ho sakte hain.

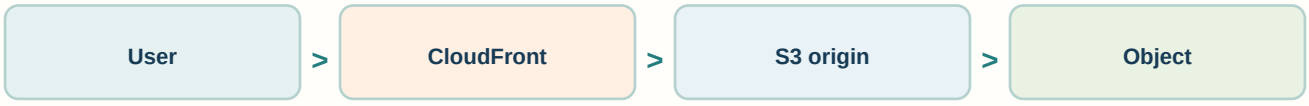
060 Lifecycle Policies

Seedhi baat: Lifecycle rules eligible objects aur versions ko transition ya expire karne mein help karti hain. Scope prefix, tags aur object characteristics se control ho sakta hai.

REAL-LIFE EXAMPLE
Logs ko defined period baad archive karo aur retention period complete hone par expire karo.

YAAD RAKHO
Legal retention, versioning, minimum-duration charges aur restore time pehle validate karo; rule test karo.

061 Static Website Hosting



Seedhi baat: S3 static website hosting HTML, CSS, JavaScript aur static assets deliver kar sakta hai; dynamic server-side code S3 website endpoint par execute nahi hota.

REAL-LIFE EXAMPLE
Portfolio site static files se host karo; HTTPS aur restricted origin ke liye CloudFront architecture evaluate karo.

YAAD RAKHO
S3 website endpoint aur REST endpoint alag behavior rakhte hain. Public access avoid karne ke liye appropriate CloudFront origin access setup samjho.

062 Permissions & Bucket Policies

Seedhi baat: IAM identity policies aur S3 bucket policies supported requests ko control kar sakti hain. Policy action, bucket/object ARN, principal aur conditions ko precisely scope karo.

REAL-LIFE EXAMPLE

App role ko sirf required prefix par GetObject do; bucket policy se authorised access restrict karo.

YAAD RAKHO

Block Public Access, explicit deny aur cross-account policy evaluation check karo; bucket aur object actions ke ARNs alag hote hain.

063 S3 Security

Seedhi baat: Block Public Access, least privilege, encryption, logging, versioning aur access review ko layered controls ki tarah use karo. Sensitive buckets ke liye data classification aur incident plan banao.

REAL-LIFE EXAMPLE

Private order-data bucket ko app role se access do; public download ki need na ho to public permissions na do.

YAAD RAKHO

Encryption public access ko rokne ka substitute nahi. Bucket policies, access points aur KMS permissions separately verify karo.

064 Data Encryption

Seedhi baat: S3 supported server-side encryption options object data at rest protect karte hain; TLS data in transit protect karta hai. SSE-S3 aur SSE-KMS key ownership/permissions mein differ karte hain.

REAL-LIFE EXAMPLE

Sensitive data ke liye KMS key policy aur IAM permissions review karo; encryption-required upload rules test karo.

YAAD RAKHO

Encryption access control aur backup replace nahi karti; key deletion/disable hone se data access impact ho sakta hai.

065 Transfer Acceleration

Seedhi baat: S3 Transfer Acceleration supported long-distance uploads/downloads ke liye AWS edge network ka use kar sakta hai. Benefit geography, network aur object patterns par depend karta hai.

REAL-LIFE EXAMPLE
Remote teams ka large-file upload acceleration on/off ke saath benchmark karke actual cost compare karo.

YAAD RAKHO
Feature aur bucket compatibility, pricing aur measurable performance check karo; har transfer faster hona guaranteed nahi.

066 S3 vs EBS vs EFS

Seedhi baat: S3 object API ke liye, EBS block disk ke liye, aur EFS shared NFS-style file system ke liye design hai. Latency, access pattern aur sharing requirements compare karo.

REAL-LIFE EXAMPLE
Photos S3, EC2 boot volume EBS aur multiple Linux instances ke shared files EFS par rakh sakte ho.

YAAD RAKHO
EBS single-AZ placement aur EFS mount/network requirements samjho; tools ko interchangeably assume mat karo.

067 S3 Real-world Use Cases

Seedhi baat: Static assets, application uploads, data lakes, backups aur archives S3 ke common use cases hain. Har use case ke liye access, retention, recovery aur cost define karo.

REAL-LIFE EXAMPLE
User upload ko private bucket mein rakho; scanning aur validation ke baad authorised application ke through expose karo.

YAAD RAKHO
Untrusted files ko process karne se pehle type/size, malware controls aur signed access design karo.

068 S3 Interview Questions + Revision

Seedhi baat: Question: S3 key kya hai? Bucket ke andar object ka identifier. Versioning kya karti hai? Object versions preserve karne ka mechanism. Glacier ka matlab instant access? Tier par depend karta hai.

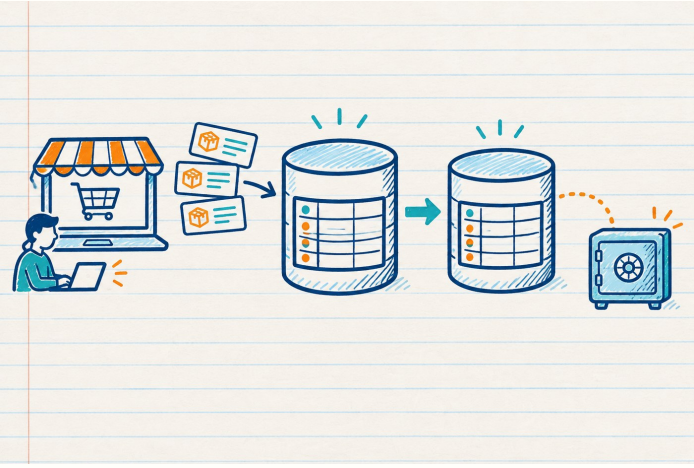
REAL-LIFE EXAMPLE
Exercise: private bucket design mein role permissions, encryption, lifecycle, versioning aur restore test list karo.

YAAD RAKHO
Durability, availability, backup aur access control alag concepts hain; ek ko doosre ka proof mat samjho.

DATABASES IN AWS

069 RDS Introduction

Seedhi baat: Amazon RDS supported relational database engines ko provision, maintain aur operate karne mein help karta hai. Backups, patching aur availability options engine/configuration par depend karte hain.



TASVEER SE SAMJHO

Order database aur recovery

Shop ke order cards database mein jaate hain; alag copy read scaling ya recovery planning mein kaam aa sakti hai. Read replica, high availability aur backup teen alag concepts hain; image conceptual hai, automatic configuration nahi.

REAL-LIFE EXAMPLE

Online store ke transactional order records ke liye RDS evaluate karo aur app ke saath connection limit test karo.

YAAD RAKHO

Managed ka matlab customer ke database users, schema, data aur network permissions automatic secure nahi.

070 Supported Databases

Seedhi baat: RDS supported engines mein PostgreSQL, MySQL, MariaDB, Oracle, SQL Server aur Db2 options ho sakte hain. Aurora MySQL/PostgreSQL-compatible separate managed offering hai.

REAL-LIFE EXAMPLE

Migration se pehle source engine, extension, licensing aur target version compatibility check karo.

YAAD RAKHO

Engine versions, Region availability, licensing aur feature support current AWS documentation mein verify karo.

071 RDS vs EC2 Database

Seedhi baat: RDS selected database operations abstract karta hai; EC2 par database chalane se OS/engine configuration par zyada control aur maintenance responsibility milti hai.

REAL-LIFE EXAMPLE
Managed backups aur patch scheduling helpful hon to RDS evaluate karo; special OS access requirement ho to EC2 compare karo.

YAAD RAKHO
Cost aur operational workload dono compare karo; managed service bhi query tuning aur security ownership demand karti hai.

072 Multi-AZ Deployment

Seedhi baat: RDS Multi-AZ supported deployment options high availability ke liye standby or multiple database instances use kar sakte hain. Failover behavior chosen engine aur deployment type par depend karta hai.

REAL-LIFE EXAMPLE
Primary AZ fail hone ki drill mein application reconnect, DNS behavior aur recovery time test karo.

YAAD RAKHO
Multi-AZ backup ya disaster recovery plan ka replacement nahi; failover zero interruption ki guarantee nahi.

073 Read Replicas

Seedhi baat: Read replica source database ke changes replicate karke supported read workloads serve kar sakti hai. Replication lag aur engine-specific capabilities samajhna zaroori hai.

REAL-LIFE EXAMPLE
Reporting queries ko replica par move karo taaki primary transaction workload par read pressure kam ho.

YAAD RAKHO
Replica stale data return kar sakti hai; Multi-AZ standby aur read replica identical concepts nahi.

074 DynamoDB: NoSQL

Seedhi baat: DynamoDB managed NoSQL service hai jo tables mein items aur attributes store karta hai. Partition key aur optional sort key access patterns design karne mein important hain.

REAL-LIFE EXAMPLE
Shopping cart ko customer key se read/write karo; expected queries aur item size pehle define karo.

YAAD RAKHO
Relational joins aur arbitrary query patterns direct equivalents assume mat karo; capacity, consistency aur hot keys test karo.

075 DynamoDB Use Cases

Seedhi baat: Low-latency key-based access, user sessions, shopping carts aur event-driven applications DynamoDB ke possible uses hain. Data model questions se start karo, sirf entities se nahi.

REAL-LIFE EXAMPLE
Customer ID ke saath current cart lookup; expiration ke liye TTL evaluate karo.

YAAD RAKHO
TTL deletion immediate guarantee nahi; hot partitions, permissions, backups aur capacity cost monitor karo.

076 Aurora Overview

Seedhi baat: Amazon Aurora MySQL- aur PostgreSQL-compatible relational database service hai jiska architecture distributed storage use karta hai. Supported readers aur recovery options configuration par depend karte hain.

REAL-LIFE EXAMPLE
Read-heavy app ke liye Aurora writer aur reader endpoints ke behavior ka test karo.

YAAD RAKHO
Compatibility har extension, engine version ya feature ki identity guarantee nahi. Actual limits aur pricing verify karo.

077 Backup & Restore

Seedhi baat: Backups data recovery points dete hain; restore test se data integrity, permissions aur application compatibility validate hoti hai. RPO aur RTO business requirements define karo.

TASVEER SE SAMJHO
Backup ka asli test: restore

Source database se backup banao, phir ek alag test environment mein restore karke data aur application check karo. Sirf backup successful notification dekhna sufficient nahi.

REAL-LIFE EXAMPLE
RDS point-in-time restore ko isolated environment mein test karo aur app se consistency verify karo.

YAAD RAKHO
Multi-AZ replication bad data ya accidental deletion se independent backup nahi; retention aur cross-account copies assess karo.

078 Scaling Databases

Seedhi baat: Database scale up se instance resources badh sakte hain; read replicas read workload distribute kar sakti hain; partitioning/sharding kuch workloads mein horizontal scale help karta hai.

REAL-LIFE EXAMPLE
Reporting traffic replica par shift karo aur write load ke liye schema/query optimization assess karo.

YAAD RAKHO
Replication lag, connection count, storage performance aur failover constraints ke bina scale decision adhura hai.

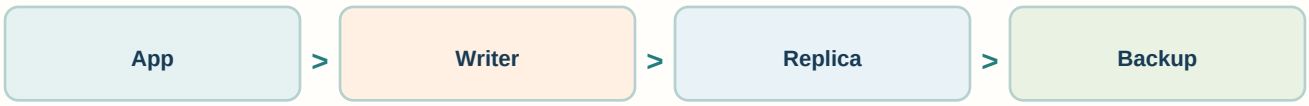
079 Performance Optimization

Seedhi baat: Query plans, indexes, slow queries, connection pooling, caching aur resource utilisation measure karo. Bottleneck identify karke change test karo.

REAL-LIFE EXAMPLE
Slow order lookup ke liye WHERE clause aur relevant index examine karo; before/after latency compare karo.

YAAD RAKHO
Extra indexes writes ko slow kar sakte hain; benchmark representative data aur monitor regressions.

080 Database Architecture Diagram



Seedhi baat: User → application → database connection; optional read traffic → replica; backup → recovery storage. Multi-AZ aur access controls workload-specific deployment mein separately show karo.

REAL-LIFE EXAMPLE
Application role/secret aur network rules set karo; restore drill se recovery path validate karo.

YAAD RAKHO
Replica arrows asynchronous ho sakte hain; diagrams mein replication lag aur primary write path clear rakho.

081 Database Real-world Example

Seedhi baat: Shop ke orders aur payment state relational database mein, read-only reports replica par, aur product images object storage mein rakhe ja sakte hain.

REAL-LIFE EXAMPLE
Payment write ko transactional logic aur duplicate request protection ke saath handle karo; backup restore test schedule karo.

YAAD RAKHO
Service selection transaction semantics, traffic pattern, availability aur compliance needs ke hisaab se karo.

082 Database Interview Questions + Revision

Seedhi baat: Read replica read scaling ke liye useful ho sakti hai; Multi-AZ availability-oriented deployment hai; backup historical recovery ke liye. DynamoDB key design access patterns se shuru hota hai.

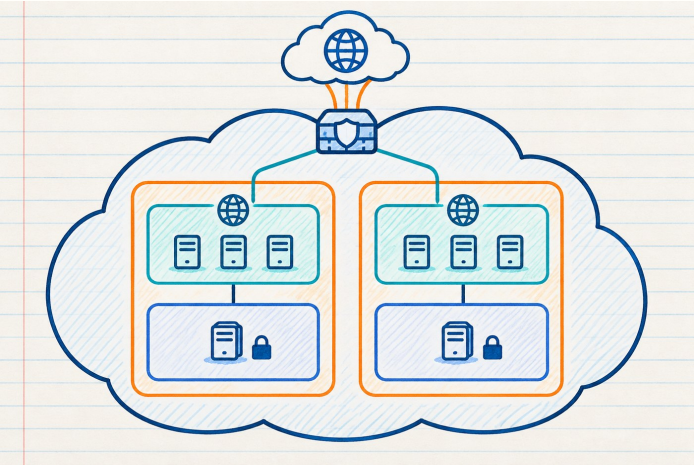
REAL-LIFE EXAMPLE
Question: replica always latest data dikhayegi? Nahi, replication lag possible hai. Multi-AZ backup replace karta hai? Nahi.

YAAD RAKHO
RDS features engine aur deployment type ke saath change hote hain; exact behavior documentation se verify karo.

VPC NETWORKING

083 What is VPC

Seedhi baat: VPC ek logically isolated virtual network hai jahan IP range, subnets, routes aur network controls define kiye jate hain. Resources selected subnets mein launch hote hain.



TASVEER SE SAMJHO
VPC ko do alag zones ki tarah dekho

Cloud ke andar do alag zones dikhaye hain. Har zone ke andar apne alag network areas hain; ek subnet do AZs mein nahi phailta. Yeh sirf concept sketch hai; actual routes aur gateways neeche ke VPC diagram se samjho.

REAL-LIFE EXAMPLE
Shop ke web, app aur database tiers ke liye VPC banao aur necessary traffic paths explicitly define karo.

YAAD RAKHO
VPC isolate karta hai, lekin overly permissive rules aur exposed resources ka risk configuration se aata hai.

084 Subnets

Seedhi baat: Subnet VPC ke CIDR ka ek IP range hai aur ek hi Availability Zone mein belong karta hai. Public/private classification effective routes par depend karti hai.

REAL-LIFE EXAMPLE
Do AZs mein alag public aur private subnets banao; har subnet ke route table aur IP allocation review karo.

YAAD RAKHO
Public subnet mein instance automatically internet-reachable nahi: public IP, route aur security rules bhi matter karte hain.

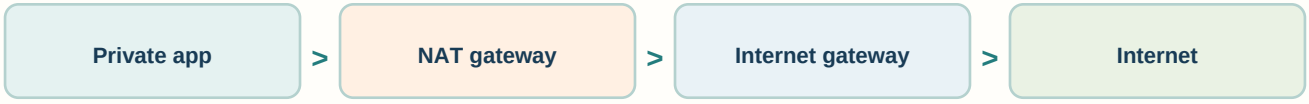
085 Internet Gateway

Seedhi baat: Internet gateway VPC aur internet ke beech supported connectivity provide karta hai. Public IPv4 traffic ke liye route, suitable public IP aur security settings bhi chahiye.

REAL-LIFE EXAMPLE
Public subnet route table mein `0.0.0.0/0` Internet Gateway ki taraf point kare aur ALB authorised traffic receive kare.

YAAD RAKHO
Gateway attach karna akela instance ko public nahi banata; IPv6 routing aur controls separately verify karo.

086 NAT Gateway



Seedhi baat: Public NAT gateway private subnet resources ko outbound IPv4 connectivity provide karne mein help karta hai, jab return traffic established connection ka hissa ho. NAT public subnet mein deploy hota hai.

REAL-LIFE EXAMPLE
Private app instance software update download kare through NAT; unsolicited internet inbound connection directly open nahi hoti.

YAAD RAKHO
Routes, Elastic IP, AZ resilience aur hourly/data processing charges plan karo. NAT security group replacement nahi.

087 Route Tables

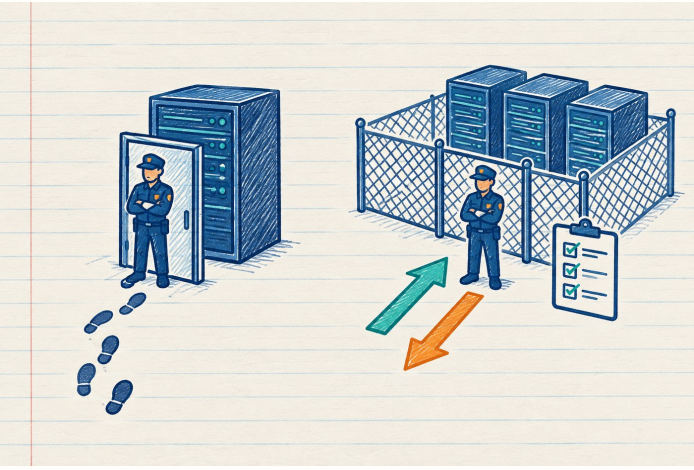
Seedhi baat: Route table destination CIDR/prefix aur next target define karti hai. Subnet association decide karta hai resource traffic ke liye kaunsi table apply hogi.

REAL-LIFE EXAMPLE
Public subnet default route internet gateway ko, private subnet IPv4 outbound default NAT gateway ko point kare.

YAAD RAKHO
Longest-prefix match aur relevant gateway/endpoint routes check karo; security group route table ka replacement nahi.

088 Security Groups vs NACL

Seedhi baat: Security groups interface-level stateful allow rules dete hain. Network ACL subnet-level stateless allow/deny rules dete hain aur numbered rules order mein evaluate hote hain.



TASVEER SE SAMJHO
Security group vs NACL

Baayen darwaze ka guard security group ki tarah interface par rule check karta hai. Daayen boundary guard subnet-level NACL ka idea hai. SG stateful hai; NACL stateless hai, isliye return traffic ke rules bhi check hote hain.

REAL-LIFE EXAMPLE
App SG database SG se specific port allow kare; subnet NACL inbound aur return ephemeral ports ko appropriate tarah permit kare.

YAAD RAKHO
Stateful SG return traffic aur stateless NACL return-path rules ko confuse mat karo; effective network path test karo.

089 Elastic IP

Seedhi baat: Elastic IP Region-scoped static public IPv4 address hai jo eligible resources se associate ho sakta hai. Supported failover aur fixed addressing workflows mein useful hai.

REAL-LIFE EXAMPLE
NAT gateway ke public IPv4 ke liye Elastic IP use karo aur allocation track karo.

YAAD RAKHO
Public IPv4 pricing aur association limits current docs se check karo; static IP alone identity ya security control nahi.

090 VPC Peering

Seedhi baat: VPC peering do VPCs ke beech private IP routing connect karne ka option hai. CIDR overlap, route tables, DNS settings aur security rules compatibility check karo.

REAL-LIFE EXAMPLE
Application VPC ko shared service VPC se peer karo aur dono sides par required routes configure karo.

YAAD RAKHO
Peering transitive routing provide nahi karta: A-B aur B-C se A-C connectivity automatically nahi milti.

091 VPN & Direct Connect

Seedhi baat: Site-to-Site VPN encrypted tunnel ke through supported network connection banata hai; Direct Connect dedicated network connection option provide karta hai. Dono ka operational setup alag hai.

REAL-LIFE EXAMPLE
Office data center ko VPC se connect karo; failover, routing aur application dependencies test karo.

YAAD RAKHO
Direct Connect by itself VPN-style encryption guarantee nahi karta; encryption requirements aur provider connectivity plan karo.

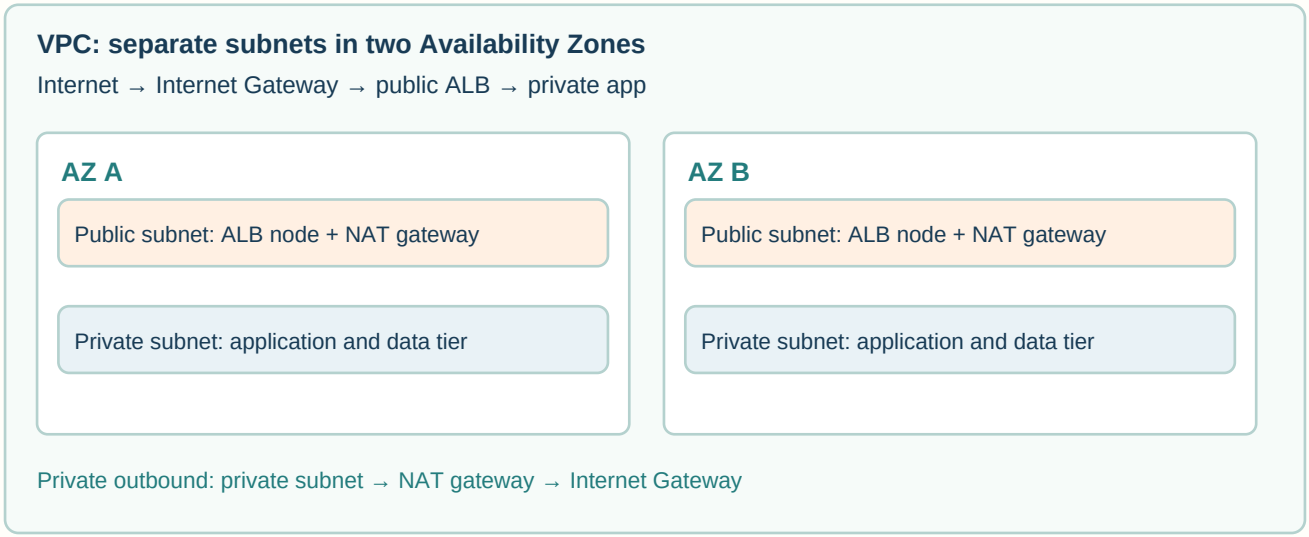
092 Network Architecture Design

Seedhi baat: Network design IP ranges, account boundaries, AZ placement, ingress/egress, DNS, private endpoints aur observability se shuru hota hai. Trust boundaries ko explicit banao.

REAL-LIFE EXAMPLE
Two-AZ layout mein public ALB aur private app/database subnets choose karo; per-tier security groups define karo.

YAAD RAKHO
CIDR overlap, single NAT dependency, asymmetric routes aur cost pehle assess karo.

093 Full VPC Diagram



Seedhi baat: Region → VPC → AZ-A aur AZ-B; har AZ mein separate public aur private subnet. Internet → IGW → public ALB → app; private outbound → NAT → IGW; database only authorised app se.

REAL-LIFE EXAMPLE
Diagram trace: external browser kis path se app tak jayega? App updates ke liye outbound traffic ka return path kya hoga?

YAAD RAKHO
Subnet AZs ko span nahi karta. NAT inbound internet connection ke liye reverse proxy nahi hai.

094 VPC Real-world Setup

Seedhi baat: Ek online shop ke liye separate AZ subnets, internet-facing ALB, private app instances aur restricted database configure karo. Logs, routes, DNS aur IAM roles bhi design ka hissa hain.

REAL-LIFE EXAMPLE
Availability drill mein ek AZ ke targets fail karo aur remaining capacity/application behavior observe karo.

YAAD RAKHO
Production setup mein threat model, deployment automation, costs aur recovery objectives independently validate karo.

095 VPC Troubleshooting

Seedhi baat: Connectivity fail ho to DNS, destination IP/port, source route, return route, security group, NACL, gateway aur application listener check karo. Layer-by-layer isolate karo.

REAL-LIFE EXAMPLE
Private instance package download nahi kar paa rahi: subnet route, NAT status, public route aur DNS resolution verify karo.

YAAD RAKHO
Diagnosis ke liye blindly all ports open mat karo; approved tools aur temporary narrowly scoped tests use karo.

096 VPC Best Practices

Seedhi baat: CIDR planning, multiple AZs, least-privilege security groups, private data tiers, flow logs aur endpoint usage workload ke according design karo.

REAL-LIFE EXAMPLE
Network access review se unnecessary public database path hatao aur VPC Flow Logs se relevant traffic investigate karo.

YAAD RAKHO
Private network compromised identity ya app vulnerability se automatic protection nahi deta; defense in depth rakho.

097 VPC Interview Questions + Revision

Seedhi baat: Public subnet ka matlab IGW route available hona; public IP aur rules alag check. SG stateful hai, NACL stateless. Peering transitive nahi hoti.

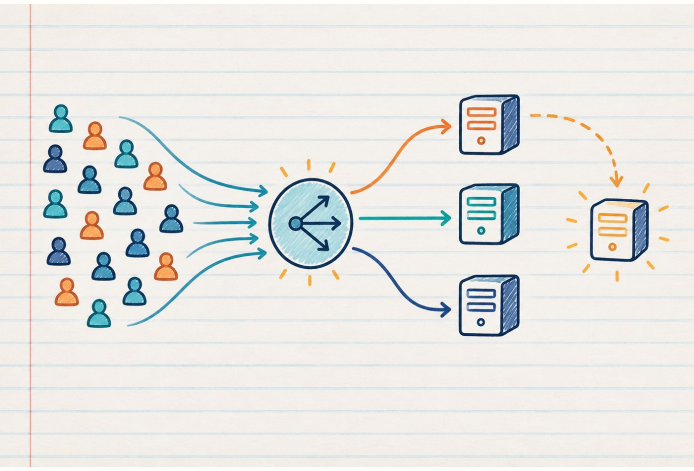
REAL-LIFE EXAMPLE
Question: NAT gateway inbound unsolicited request accept karta hai? Nahi. Kya subnet do AZs mein extend ho sakta hai? Nahi.

YAAD RAKHO
Networking answers ke saath route, source/destination aur allowed return path draw karke samjho.

LOAD BALANCING & AUTO SCALING

098 Load Balancer Introduction

Seedhi baat: Load balancer clients se traffic receive karke configured targets tak distribute karta hai. Listeners, target groups aur health checks traffic routing ke core pieces hain.



TASVEER SE SAMJHO
Traffic ko baantna

Users ki requests ek traffic distributor tak aati hain aur healthy servers ko bheji jaati hain. Extra server scale-out ka idea hai. Actual scaling aur health checks configure aur test karne hote hain.

REAL-LIFE EXAMPLE
Public endpoint par traffic accept karke multiple app instances ko requests bhejo; unhealthy target ko routing se remove karo.

YAAD RAKHO
Backend capacity, app timeouts aur single dependencies ki failure planning zaroori hai.

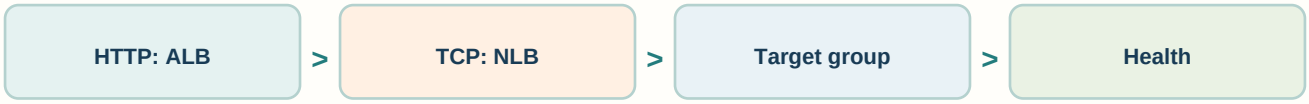
099 Load Balancer Types: ALB, NLB, CLB

Seedhi baat: ALB HTTP/HTTPS application-layer routing ke liye; NLB TCP/UDP aur supported transport-level workloads ke liye; Classic Load Balancer legacy option hai.

REAL-LIFE EXAMPLE
Path-based web routing ke liye ALB aur low-level TCP use case ke liye NLB compare karo.

YAAD RAKHO
Features aur protocol support evolve hote hain; legacy CLB ki current AWS migration guidance check karo.

100 ALB vs NLB



Seedhi baat: ALB request attributes jaise host/path ke basis par HTTP routing support karta hai; NLB transport-layer connections aur supported static IP needs ko address kar sakta hai.

REAL-LIFE EXAMPLE
`/api` aur `/shop` ko different target groups bhejna ALB use case; TCP service ko NLB se expose karna doosra example.

YAAD RAKHO
TLS termination, target types, client IP aur cost ko actual configuration se verify karo.

101 Health Checks

Seedhi baat: Load balancer periodic checks se targets ki health assess karta hai aur configured thresholds ke baad traffic routing adjust karta hai.

REAL-LIFE EXAMPLE
`/health` endpoint application readiness reflect kare; failure case aur recovery threshold test karo.

YAAD RAKHO
Health endpoint itna shallow na ho ki broken app healthy dikhe, aur na itna strict ki transient dependency failure sab targets remove kar de.

102 Auto Scaling Groups

Seedhi baat: Auto Scaling group launch template aur min, desired, max capacity se EC2 fleet manage karta hai. Unhealthy instances replace aur supported policies ke according capacity adjust ho sakti hai.

REAL-LIFE EXAMPLE
Do AZs mein app instances distribute karo; deployment aur scale-out ke baad ALB target health check karo.

YAAD RAKHO
Instance startup time, quotas, capacity availability aur app shared state ki design test karni hogi.

103 Scaling Policies

Seedhi baat: Target tracking desired metric level maintain karne ki koshish karta hai; step aur scheduled scaling alag patterns ke options hain.

REAL-LIFE EXAMPLE
Predictable morning traffic se pehle scheduled scale-out, phir demand ke according target tracking evaluate karo.

YAAD RAKHO
Metric lag, cooldown/warm-up, scale-in safety aur min/max boundaries test karo; aggressive policy oscillation la sakti hai.

104 High Availability

Seedhi baat: High availability service ko component failures ke bawajood accessible rakhne ka design goal hai. Redundant resources ko independent failure domains mein place karna useful hota hai.

REAL-LIFE EXAMPLE
ALB aur application targets ko multiple AZs mein design karo; database failover path aur capacity bhi review karo.

YAAD RAKHO
Multiple instances ek hi AZ mein rakhne se AZ failure tolerate hona guaranteed nahi.

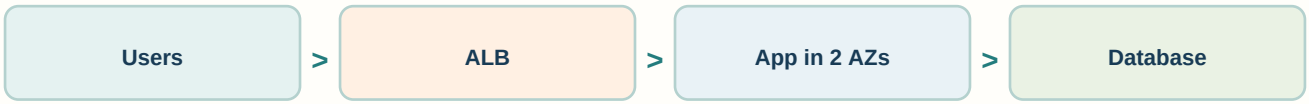
105 Fault Tolerance

Seedhi baat: Fault tolerance failure hone par service continue karne ki ability hai; graceful degradation aur recovery mechanisms workload ke hisaab se alag ho sakte hain.

REAL-LIFE EXAMPLE
Ek instance fail hone par healthy targets traffic sambhalein; payment dependency down ho to safe retry aur customer messaging plan karo.

YAAD RAKHO
Redundancy duplicate side effects, shared dependency failures ya data corruption automatically solve nahi karti.

106 Load Balancing Real-world Architecture



Seedhi baat: User → DNS → ALB → multiple AZs mein app instances → database. Monitoring, role-based access aur backups side controls hain.

REAL-LIFE EXAMPLE
Shop sale ke load test mein per-target latency, error rate, instance capacity aur database connections observe karo.

YAAD RAKHO
Ye educational diagram hai; production mein TLS, routing, availability objectives aur failure drills detail mein design karo.

107 Load Balancing Cost Optimization

Seedhi baat: Load balancer hours/usage, processed traffic, data transfer aur underlying compute costs evaluate karo. Idle services aur oversized instance fleets identify karo.

REAL-LIFE EXAMPLE
Off-hours lower demand ke liye safe scale-in aur right-sized instance benchmark karo.

YAAD RAKHO
Cost kam karte waqt resilience, traffic spikes aur minimum healthy capacity compromise na karo.

108 Load Balancing Interview Questions + Revision

Seedhi baat: ALB application-layer aur NLB transport-layer use cases address karte hain; target health checks determine karte hain configured routing eligibility. ASG desired capacity maintain karta hai.

REAL-LIFE EXAMPLE
Question: kya ASG ek database ko automatically scale karta hai? Nahi. Kya unhealthy target ka reason investigate karna chahiye? Haan.

YAAD RAKHO
Scaling, load distribution aur failover related but distinct functions hain.

MONITORING & LOGGING

109 CloudWatch Overview

Seedhi baat: CloudWatch metrics, logs, alarms aur dashboards se supported AWS resources aur applications observe karne mein help karta hai.

TASVEER SE SAMJHO

Dikkat ka pata kaise chalega?

Server se metrics aur logs collect karo; alarm condition par on-call person ko notify karo. Picture sab monitoring cheezein saath dikhati hai; metrics se logs banne ka fixed sequence zaroori nahi.

REAL-LIFE EXAMPLE

App latency, errors aur EC2 CPU dashboard par dikhao; critical thresholds par notification configure karo.

YAAD RAKHO

Monitoring signal collection, retention aur charges configure/verify karo; observability automatic incident resolution nahi.

110 Metrics & Logs

Seedhi baat: Metric numeric time-series measurement hai; log event/detail record karta hai. Dimensions, timestamps, aggregation aur retention ka design useful analysis ke liye important hai.

REAL-LIFE EXAMPLE

HTTP error count metric par alert; corresponding request logs se error detail investigate karo.

YAAD RAKHO

Secrets aur personal data logs mein unnecessarily na likho; access aur retention restrict karo.

111 Alarms

Seedhi baat: CloudWatch alarm configured metric aur evaluation periods ke basis par state change kar sakta hai aur supported action initiate kar sakta hai.

REAL-LIFE EXAMPLE

5-minute periods mein high error rate sustained rahe to SNS notification bhejo aur runbook follow karo.

YAAD RAKHO

Threshold, missing-data behavior, action permissions aur notification delivery test karo; one spike aur sustained failure alag hain.

112 CloudTrail

Seedhi baat: CloudTrail supported AWS API/account activity record karta hai. Event history, trail aur event data configurations ke scope aur retention differ karte hain.

TASVEER SE SAMJHO

Monitoring aur audit alag sawaal hain

Baayen performance aur alarm: system kaisa chal raha hai? Daayen audit ledger: kis identity ne kaunsa supported API action kiya? CloudWatch aur CloudTrail in alag sawalon ko address karne mein help karte hain.

REAL-LIFE EXAMPLE

Unexpected IAM policy change investigate karne ke liye principal, event time, action aur source information review karo.

YAAD RAKHO

Management aur data events automatically same scope mein nahi hote; logging, storage, security aur cost plan karo.

113 AWS Config

Seedhi baat: AWS Config supported resource configurations record aur evaluate karne mein help karta hai. Rules compliance findings generate kar sakti hain based on configured checks.

REAL-LIFE EXAMPLE

Publicly accessible resource rule trigger ho to resource owner investigate aur approved remediation apply kare.

YAAD RAKHO

Config rule result security ya regulatory compliance ka complete guarantee nahi; recording scope aur rule logic test karo.

114 Logging Strategy

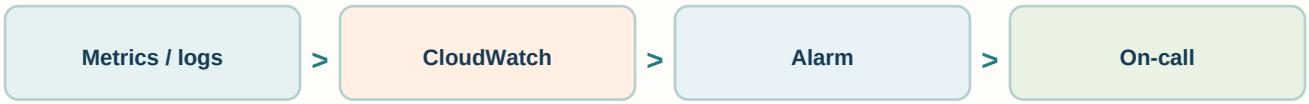
Seedhi baat: Collect, centralise, protect, retain aur search logs according to incident and compliance needs. Application, access, network aur audit logs alag questions answer karte hain.

REAL-LIFE EXAMPLE

Accounts se central log destination design karo; ingestion failure aur retention deletion ke alerts plan karo.

YAAD RAKHO
Log sensitive data minimise karo, integrity aur access restrict karo, and lifecycle costs estimate karo.

115 Monitoring Architecture Diagram



Seedhi baat: Application/resources → metrics and logs → CloudWatch dashboards/alarms → notification/on-call response. API activity → CloudTrail → protected audit storage; configuration → AWS Config.

REAL-LIFE EXAMPLE
Checkout failures badhein to alarm se owner identify, logs inspect aur relevant deployment events correlate karo.

YAAD RAKHO
Data paths aur required permissions/configuration explicit rakho; diagram implicit automatic integration assume nahi karta.

116 Debugging Issues

Seedhi baat: Symptom, impact aur start time establish karo; dashboards, logs, recent changes aur dependency health correlate karo. Hypothesis-by-hypothesis narrow tests run karo.

REAL-LIFE EXAMPLE
High latency par ALB metrics, app traces, database slow queries aur resource saturation compare karo.

YAAD RAKHO
Production mein broad destructive diagnostics avoid karo; evidence aur incident timeline preserve karo.

117 Real-world Monitoring Setup

Seedhi baat: Shop ke checkout error rate, latency, order success aur queue age track karo. Infrastructure metrics ke saath user-facing signals aur alert ownership define karo.

REAL-LIFE EXAMPLE
Alarm trigger hone par on-call runbook, dashboard link aur escalation route available ho; controlled test se notification verify karo.

YAAD RAKHO
Too many noisy alerts incident response slow karte hain; thresholds aur service objectives review karo.

118 Monitoring Interview Questions + Revision

Seedhi baat: CloudWatch operational telemetry ke liye; CloudTrail supported AWS API activity ke liye; AWS Config resource configuration/compliance evaluations ke liye.

REAL-LIFE EXAMPLE

Question: CloudTrail app error logs ka replacement hai? Nahi. Alarm hard budget cap hai? Nahi.

YAAD RAKHO

Har tool ka data scope, retention aur permissions separately verify karo.

SECURITY & COMPLIANCE

119 AWS Security Basics

Seedhi baat: Identity, network, encryption, monitoring, backups aur response multiple defensive layers create karte hain. Security workload ke lifecycle mein continuously operate hoti hai.

TASVEER SE SAMJHO

Security ek taala nahi, layers hain

Identity check, permission, encryption, monitoring aur incident response ko saath socho. Ek shield ka chitra sab controls ka replacement nahi: har layer ko configure aur regularly test karna padta hai.

REAL-LIFE EXAMPLE

Role-based access, private data tier, encrypted storage aur logged administrative actions se starting baseline banao.

YAAD RAKHO

Ek control failure ka risk reduce karne ke liye layered design aur regular testing karo.

120 Shared Responsibility Model: Security

Seedhi baat: AWS underlying cloud infrastructure security ka owner hai; customer data, access aur service-specific configuration sambhalta hai. Managed service mein exact division different hoti hai.

REAL-LIFE EXAMPLE

EC2 par guest OS patching customer karta hai; RDS mein provider selected operations manage karta hai.

YAAD RAKHO
Responsibility matrix service aur deployment ke basis par verify karo; shared ka matlab equal split nahi.

121 Encryption

Seedhi baat: Encryption plaintext ko cryptographic key se protected form mein badalti hai. At-rest protection storage par aur in-transit protection network connection par apply hoti hai.

REAL-LIFE EXAMPLE
S3 data at rest encrypt karo aur client-to-service communication mein validated TLS use karo.

YAAD RAKHO
Encryption key policy, rotation, recovery, access control aur key deletion risks bhi plan karo.

122 Key Management Service

Seedhi baat: AWS KMS encryption keys manage aur supported cryptographic operations ke liye use hota hai. Key policies, IAM policies aur grants authorised usage influence kar sakte hain.

REAL-LIFE EXAMPLE
Application S3 SSE-KMS object read kare to S3 permissions ke saath relevant KMS decrypt permission bhi verify karo.

YAAD RAKHO
Key disable/delete ya incorrect policy se application access fail ho sakta hai; key lifecycle aur audit plan karo.

123 Web Application Firewall

Seedhi baat: AWS WAF supported web-facing resources ke HTTP(S) requests par configured filtering rules evaluate karta hai. Managed rules aur custom conditions supported deployment options hain.

TASVEER SE SAMJHO
WAF: web requests ka filter

Web application tak aane wali HTTP requests ko rules se inspect karo. Kuch patterns block ya count ho sakte hain; valid traffic bhi galti se block na ho, isliye rules test karo. WAF application login aur authorisation ka replacement nahi.

REAL-LIFE EXAMPLE
ALB ya CloudFront ke saamne abusive request patterns detect/filter karne ke rules evaluate karo; first count mode se false positives measure karo.

YAAD RAKHO
WAF app authorization ya secure code ka replacement nahi; rule scope, cost aur regional availability check karo.

124 AWS Shield

Seedhi baat: AWS Shield distributed denial-of-service protection capabilities provide karta hai. Standard aur Advanced options ka coverage aur operational model alag hai.

REAL-LIFE EXAMPLE
Public application ke DDoS response plan mein CloudFront, Shield capabilities aur escalation process evaluate karo.

YAAD RAKHO
Protection har application-layer issue ya all attack outcomes ke against guarantee nahi; current terms and costs review karo.

125 Secrets Manager

Seedhi baat: Secrets Manager supported secrets store, retrieve aur selected rotation workflows configure karne mein help karta hai. Workloads ko IAM permission ke through authorised access milta hai.

REAL-LIFE EXAMPLE
Database credentials source code mein rakhne ke bajay service se retrieve karo; rotation ke baad app connection behavior test karo.

YAAD RAKHO
Secret access logs, caching, KMS policy, rotation compatibility aur exposure response plan karo.

126 Security Best Practices

Seedhi baat: Federated workforce access, MFA, least privilege, logging, encryption, patching aur tested backups ko baseline banao. Account isolation aur continuous review add karo.

REAL-LIFE EXAMPLE
Sensitive workloads mein separate accounts, narrowly scoped roles, central audit aur incident response ownership define karo.

YAAD RAKHO
Default deny aur explicit allow samjho; admin access as a troubleshooting shortcut avoid karo.

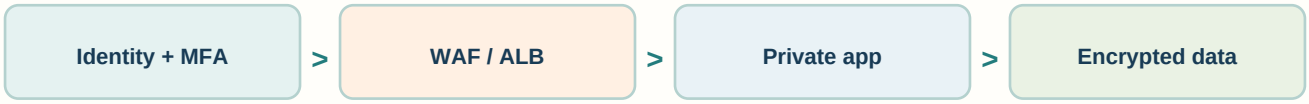
127 Compliance Standards

Seedhi baat: Standards aur regulations organisation, geography aur workload par depend karte hain. AWS assurance reports infrastructure controls ka evidence ho sakte hain; customer controls separately assess hote hain.

REAL-LIFE EXAMPLE
Auditor ke liye relevant SOC/ISO/PCI report scope, selected Region/service aur internal access procedures document karo.

YAAD RAKHO
AWS ka certification automatically tumhari application ko certified ya compliant nahi banata.

128 Security Architecture Diagram



Seedhi baat: Identity provider → role/MFA → application; internet traffic → WAF/ALB → private app; app → encrypted data; audit events → protected logs. KMS aur secrets controls authorised calls protect karte hain.

REAL-LIFE EXAMPLE
Incident drill: suspicious role activity par affected access limit karo, logs preserve karo aur response owner ko notify karo.

YAAD RAKHO
Diagram conceptual hai; each arrow ki policy, network path aur service support explicitly implement karo.

129 Real-world Security Setup

Seedhi baat: Online shop ke liye role-based app access, private database, controlled ingress, encryption aur central logs configure karo. Backup restore aur incident response test karo.

REAL-LIFE EXAMPLE
New engineer joins: federation group assign karo, least-privilege role test karo, access review date set karo.

YAAD RAKHO
Credentials ko chat ya repository mein share mat karo; guardrails plus application-level authorization dono chahiye.

130 Security Interview Questions + Revision

Seedhi baat: Authentication identity verify karti hai; authorization allowed action decide karti hai. IAM role trust aur permissions alag; KMS keys access policies se governed; WAF network firewall ka full replacement nahi.

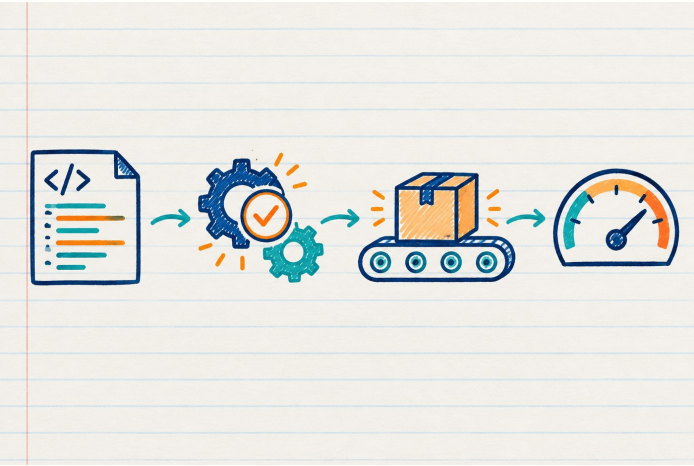
REAL-LIFE EXAMPLE
Question: encryption ke baad public data safe? Nahi, authorised/public read policy aur key access matter karte hain.

YAAD RAKHO
Explicit deny aur policy evaluation ko request-specific context ke saath samjho.

DEVOPS & DEPLOYMENT

131 CI/CD Basics

Seedhi baat: Continuous integration code changes build aur test karti hai; continuous delivery/deployment release process automate karta hai. Approval aur production deployment rules team policy se define hote hain.



TASVEER SE SAMJHO
Code se running application tak

Pehle code, phir build aur test, uske baad deploy aur monitor. Team ki policy ke hisaab se manual approval bhi lag sakta hai. Failed test ke baad seedha production release nahi karna chahiye.

REAL-LIFE EXAMPLE
Commit → build → unit test → security scan → staging → approval → production → monitoring.

YAAD RAKHO
Automation speed badhati hai, lekin secrets, artifacts, permissions aur rollback controls safe rakhna zaroori hai.

132 CodePipeline

Seedhi baat: CodePipeline release workflow ke stages aur supported actions coordinate karta hai. Source, build, test, approval aur deployment integrate kiye ja sakte hain.

REAL-LIFE EXAMPLE
Repository commit se pipeline trigger ho, build artifact produce ho, test pass hone ke baad approved deployment ho.

YAAD RAKHO
Pipeline roles, artifact encryption, source triggers aur retry/failure notifications review karo.

133 CodeBuild

Seedhi baat: CodeBuild managed build service hai jo configured build environments mein source compile, tests aur commands run kar sakta hai.

REAL-LIFE EXAMPLE
Buildspec mein dependencies install, tests run aur deployable artifact package karo; logs se failure diagnose karo.

YAAD RAKHO
Build script untrusted code execute kar sakta hai; role permissions, dependency integrity aur secret exposure limit karo.

134 CodeDeploy

Seedhi baat: CodeDeploy supported compute platforms par application deployment coordinate karne mein help karta hai; deployment strategies platform-specific hoti hain.

REAL-LIFE EXAMPLE
EC2 fleet par new version gradually deploy karo aur health failure par configured rollback evaluate karo.

YAAD RAKHO
Agent/platform requirements, deployment hooks, health checks aur rollback behavior actual environment mein test karo.

135 Elastic Beanstalk

Seedhi baat: Elastic Beanstalk supported applications deploy karne ke liye environment provisioning aur selected operational orchestration simplify karta hai. Underlying resources still exist aur billable ho sakte hain.

REAL-LIFE EXAMPLE
Web application package upload karo; environment configuration, capacity aur deployment health monitor karo.

YAAD RAKHO
Platform versions, service roles, updates aur underlying resource costs review karo; abstraction zero operations nahi.

136 Docker on AWS

Seedhi baat: Container image application aur dependencies package karti hai; runtime container image se isolated process chalata hai. AWS par ECR registry aur ECS/EKS execution options hain.

REAL-LIFE EXAMPLE
App build → image scan → ECR push → ECS deployment → service health checks.

YAAD RAKHO
Image scan, base-image updates, secrets handling aur least-privilege runtime important hain; container VM security boundary ka direct substitute nahi.

137 ECS vs EKS

Seedhi baat: ECS AWS-native container orchestration hai; EKS managed Kubernetes control-plane option hai. Supported workloads EC2 ya Fargate compute choices evaluate kar sakte hain.

REAL-LIFE EXAMPLE
Existing Kubernetes tooling aur expertise ho to EKS evaluate; simpler AWS-native orchestration ke liye ECS compare karo.

YAAD RAKHO
Operations, add-ons, networking, IAM, portability aur total cost compare karo; dono automatically simpler nahi.

138 AWS Lambda

Seedhi baat: Lambda event-driven code execution service hai jahan infrastructure operations ka kuch hissa abstract hota hai. Function runtime, IAM role, timeout, memory aur triggers configure hote hain.

REAL-LIFE EXAMPLE
S3 object event par image processing function run ho; failure/retry aur duplicate events safely handle karo.

YAAD RAKHO
Execution limits, cold starts, concurrency, cost aur idempotency ka impact benchmark karo; long-running state externalise karo.

139 Real-world Deployment Pipeline

Commit > **Build / test** > **Deploy** > **Observe**

Seedhi baat: Developer commit se automated build/test, security checks aur versioned artifact; staging test ke baad controlled production deployment aur health observation.

TASVEER SE SAMJHO
Pipeline mein approval aur observation

Code change test hone ke baad approval aur deployment hota hai; deployment ke baad health dekhni padti hai. Failure aaye to rollback aur incident response ka plan pehle se rakho.

REAL-LIFE EXAMPLE
Agar error rate release ke baad badhe, approved rollback run karo; incident timeline aur deployment ID correlate karo.

YAAD RAKHO
Deployment automation ko least privilege do; production approvals, secrets aur reproducible artifacts protect karo.

140 DevOps Revision Sheet

Seedhi baat: CI build/test; CD release/deploy; CodePipeline orchestration; CodeBuild build/test; CodeDeploy deployment; ECS/EKS container orchestration; Lambda event-driven compute.

REAL-LIFE EXAMPLE
Question: image registry aur orchestrator same? Nahi: ECR image store karta hai, ECS/EKS containers manage karte hain.

YAAD RAKHO
IaC, automated testing, monitoring aur rollback saath rakho; current feature/lifecycle details check karo.

BILLING & COST OPTIMIZATION

141 AWS Pricing Model

Seedhi baat: Pricing service ke hisaab se usage, allocated capacity, requests, storage, transfer aur commitments par based hoti hai. Exact rate Region aur configuration se badalti hai.

TASVEER SE SAMJHO
Bill kin cheezon se banta hai?

Compute, storage, requests aur transfer jaise resources bill mein contribute kar sakte hain. Bell budget alert aur magnifying glass cost review ki yaad dilate hain. Alert apne aap spending stop nahi karta.

REAL-LIFE EXAMPLE
Website estimate mein compute hours ke saath EBS, load balancer, requests aur data transfer bhi add karo.

YAAD RAKHO
Pay-as-you-go ka matlab har idle resource free nahi; pricing calculator aur actual usage compare karo.

142 Billing Dashboard

Seedhi baat: Billing console invoices, charges, account settings aur supported cost-management views dikhata hai. Organisational access aur billing permissions separately configure hote hain.

REAL-LIFE EXAMPLE
Unexpected bill dekhkar service, Region aur linked account breakdown identify karo.

YAAD RAKHO
Sensitive billing access least privilege se do; dashboard visibility aur cost data timing/refresh limitations check karo.

143 Cost Explorer

Seedhi baat: Cost Explorer historical costs aur supported usage trends analyse karne mein help karta hai. Filters aur groupings accounts, services aur activated tags se useful ho sakti hain.

REAL-LIFE EXAMPLE
Month-over-month EC2 aur S3 costs compare karo; unusually high usage metric identify karke owner se investigate karo.

YAAD RAKHO
Attribution incomplete tags ya shared costs se misleading ho sakti hai; forecast actual bill ki guarantee nahi.

144 Budgets & Alerts

Seedhi baat: AWS Budgets configured cost/usage thresholds par alerts provide kar sakta hai; supported budget actions specific configuration aur permissions require karte hain.

REAL-LIFE EXAMPLE
Monthly threshold par email alert configure karo aur receiving address test karo; action ke liye owner/runbook assign karo.

YAAD RAKHO
Alert alone automatic hard spending cap nahi. Notifications, budgets aur optional enforcement alag cheezein hain.

145 Cost Optimization Techniques

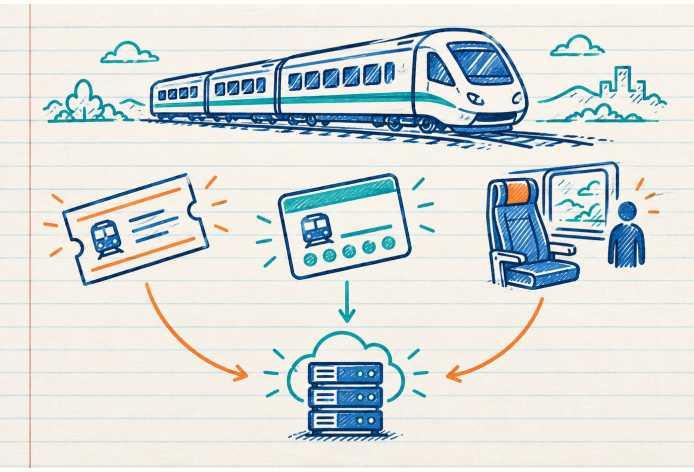
Seedhi baat: Right-size after measurement, unused resources remove karo, suitable storage tier select karo, lifecycle configure karo aur commitment coverage evaluate karo. Allocate cost to owners aur review routinely.

REAL-LIFE EXAMPLE
Non-production instances approved schedule par stop karo; retained EBS aur IP costs still check karo.

YAAD RAKHO
Reliability, performance, security aur recovery requirements violate karke cost cut mat karo.

146 Reserved Instances

Seedhi baat: Reserved Instances eligible EC2 capacity/pricing commitments ki family hain; scope, instance attributes, term aur payment options terms define karte hain. Savings Plans alag commitment model hai.



TASVEER SE SAMJHO
Pricing ko tickets ki tarah socho

Regular ticket flexible On-Demand ka idea deta hai; prepaid pass commitment ka; standby seat interruptible Spot ka. Analogy sirf yaad rakhne ke liye hai: AWS ki actual pricing, rules aur interruptions official terms se verify karo.

REAL-LIFE EXAMPLE
Stable baseline workload ke liye historical usage se commitment coverage assess karo; uncertain burst ke liye flexible capacity rakho.

YAAD RAKHO
RI aur Savings Plans interchangeable nahi; utilisation, applicable terms aur current pricing confirm karo.

147 Spot Instances

Seedhi baat: Spot spare EC2 capacity use karta hai at variable pricing; AWS capacity need ke hisaab se instance interrupt kar sakta hai. Appropriate workloads interruption-tolerant hone chahiye.

REAL-LIFE EXAMPLE
Batch image processing jobs checkpoints rakhein aur interruption par safely retry karein.

YAAD RAKHO
Critical single-instance stateful workload ko unprotected Spot par mat rakho; interruption notice/timing aur availability verify karo.

148 Free Tier Strategy

Seedhi baat: Applicable account plan aur current service-specific limits samjho; lab se pehle expected costs calculate karo aur spending alerts set karo. Time limits aur eligibility vary kar sakte hain.

REAL-LIFE EXAMPLE
Sandbox mein small resources launch karo, usage track karo aur lab ke baad instances, volumes, gateways aur storage cleanup karo.

YAAD RAKHO
Free Tier unlimited nahi; account plan aur eligible services check karo. Budget notification alone spending stop nahi karti.

149 Real-world Cost Saving Example

Seedhi baat: Shop ne monthly bill mein idle development instances aur unnecessary old storage versions identify kiye. Owner approval ke baad schedule, rightsizing aur lifecycle changes test kiye.

REAL-LIFE EXAMPLE
Pehle baseline cost aur usage measure karo; change ke baad comparable period mein savings aur application reliability compare karo.

YAAD RAKHO
Yeh example illustrative hai; exact savings claim mat karo bina real price, workload aur measurement ke.

150 Final Revision Sheet

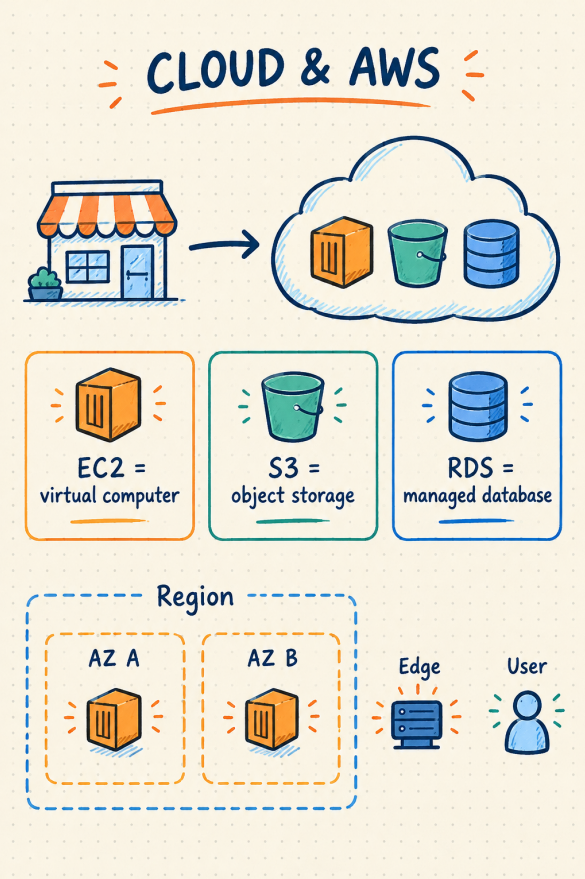
Seedhi baat: Compute chalaata hai, storage data rakhta hai, database queries handle karta hai, VPC connectivity define karta hai aur IAM access decide karta hai. Monitor, secure, backup aur cost-review har layer par chahiye.

REAL-LIFE EXAMPLE
Self-test: Multi-AZ backup hai? Nahi. Budget alert hard cap hai? Nahi. SG stateful hai? Haan. Subnet multiple AZs mein? Nahi.

YAAD RAKHO
Official docs se current support, Region, pricing aur lifecycle verify karo; architecture ko workload aur recovery needs se validate karo.

EXPANDED SERVICE HANDBOOK

Ab service-wise details, engineering examples aur hands-on drills padho. Ye part upar diye gaye 150 topics se aage AWS ke additional areas bhi cover karta hai.



Cloud aur AWS: shuru se samjho

Need > Cloud resource > Configure > Use / monitor

Ek online shop ki kahani

Socho Riya apni online shop shuru karti hai. Usko website chalane ke liye computer chahiye, product photos ke liye storage aur orders ke liye database. Agar woh sab hardware khud kharide, to demand badhne se pehle hi planning, kharid, delivery, setup, power, cooling aur maintenance ke liye time aur paisa lagta hai. Demand kam nikli to machines khaali baith sakti hain; zyada nikli to website slow ya unavailable ho sakti hai.

Cloud mein Riya zarurat ke hisaab se computing, storage, databases aur doosre services provision kar sakti hai. Provision ka matlab hai resource ko apne workload ke liye configure karke use ke liye available karna. Kuch resources ki capacity baad mein badhai ya ghatai ja sakti hai. Cloud ka matlab yeh nahi ki kharch apne aap kam hoga: selected pricing, continuously running resources aur architecture ka design bill ko affect karte hain.

Pehle ki cost aur usage ke saath badalne wali cost

• Traditional setup mein hardware kharid par upfront capital expenditure ho sakta hai; uske baad power aur maintenance bhi lagte hain.

• Cloud mein many services consumption ya allocated capacity ke hisaab se charge karti hain. Exact metric service ke anusaar hota hai: instance time, stored data, requests, transfer, etc.

• Pay-as-you-go ke saath reserved commitments ya fixed charges bhi ho sakte hain. Isliye 'jitna use utna hi bill' har product aur contract ke liye bina qualification ke sahi nahi hai.

AWS ne 2006 mein internet ke through IT infrastructure services dena shuru kiya. Aaj startups, enterprises, public-sector organisations aur chhote businesses in building blocks ko apni requirements ke anusaar combine kar sakte hain. Availability, account quotas aur prices alag Regions aur services mein alag ho sakte hain.

Cloud computing kya hai?

Cloud computing internet ke through on-demand compute, storage, database, applications aur other IT resources ka access hai. Cloud provider network-connected physical hardware aur facilities own/manage karta hai; customer selected resources provision karke apne authorised workloads chalata hai. Customer ke paas application, identity, data aur configuration ki relevant responsibilities rehti hain.

Example: festival sale mein Riya ko website capacity badhani hai. Appropriate architecture ke saath woh extra instances launch kar sakti hai aur sale ke baad capacity reduce kar sakti hai. Lekin uske database, application aur quotas ko bhi extra traffic handle karna hoga; sirf server count badhana har bottleneck solve nahi karta.

Yaad rakho: scalability capacity badhane ya ghatane ki ability hai. Elasticity demand ke saath capacity ko adjust karne ka pattern hai. Aapko safe scaling policy, limits aur monitoring design karni hoti hai.

Cloud ke chhe practical advantages

• Upfront expenditure se variable model ki taraf: kuch infrastructure purchase karne ke bajay usage ya allocated resources ke pricing model mein move kar sakte ho. Lekin commitments aur unused resources ka cost bhi assess karo.

• Economies of scale ka potential: provider large aggregated demand ke liye infrastructure kharidta aur operate karta hai. Is scale ka benefit pricing mein aa sakta hai, par kisi specific workload ke liye lower cost guaranteed nahi.

• Capacity ka guess kam: past mein peak ke liye hardware buy karna padta tha; cloud mein demand observe karke supported services ki capacity adjust kar sakte ho. Quotas, warm-up aur application design phir bhi matter karte hain.

• Speed aur experimentation: authorised team console, CLI ya automation se resources jaldi create kar sakti hai. Provisioning minutes mein ho sakti hai, lekin security approval, tests aur governance steps skip nahi karne chahiye.

• Data-center heavy lifting kam: physical servers rack karna, power aur facilities maintain karna provider ka kaam hota hai. Customer app, permissions, data aur applicable service-specific operations ka owner rehta hai.

• Global reach: eligible services aur Regions ke through users ke paas deployment plan kar sakte ho. Region choose karte waqt latency, residency, service availability, resilience aur actual costs check karo; multi-Region automatically correct ya cheap nahi hota.

Practice: sale khatam hui aur instance 24/7 chalta raha. Kya 'elasticity available hai' ka matlab bill khud kam ho jayega? Nahi. Resource ko scale down karne ki policy ya explicit cleanup zaruri hai.

Deployment models: cloud, private aur hybrid

Cloud deployment: application ke components cloud environment mein deploy hote hain. Application nayi cloud-native ho sakti hai ya existing infrastructure se migrate ki gayi ho sakti hai. Aap low-level components select karke zyada configuration manage kar sakte ho, ya higher-level managed services choose karke kuch operational burden provider ko de sakte ho.

Private/on-premises: organisation dedicated environment aur facilities mein workloads operate karti hai. Virtualisation aur management software se hardware utilisation improve ho sakta hai. Ye approach dedicated control ya particular constraints ke liye relevant ho sakti hai, lekin facility aur hardware operations ka burden bhi consider karna hoga.

Hybrid: cloud resources existing non-cloud infrastructure se connect hote hain. Example: existing on-premises inventory database se cloud-hosted storefront connect karna. Network connectivity, routing, DNS, identity, encryption, monitoring aur failure recovery dono sides par design karni hogi.

Socho: agar on-premises connection fail ho gaya aur storefront ko har request par wahi database chahiye, to cloud-hosted website ka behavior kya hoga? Cache, degraded mode aur recovery requirements pehle decide karo.

Regions, Availability Zones aur edge

Region geographic area hai jahan AWS infrastructure aur eligible services available hoti hain. Region ke andar multiple Availability Zones (AZs) hoti hain. AZs isolated failure domains ke roop mein design hoti hain aur network se interconnected hoti hain. Ek AZ ko ek single building samajhna zaruri nahi; service-specific documentation dekho.

Region choose karte waqt customer latency, data residency, regulatory requirements, available services/features, price aur recovery plan compare karo. Do app instances ek hi AZ mein hon to AZ failure dono ko affect kar sakta hai; separate AZs mein placement resilience improve kar sakta hai. Database aur dependencies bhi resilient honi chahiye.

Edge locations users ke paas supported delivery services ko help kar sakti hain. Edge location, Region aur AZ ek hi cheez nahi hain. VPC ka subnet sirf ek AZ mein hota hai.

Security: cloud mein kaun kya sambhalta hai?

Physical facilities aur underlying cloud infrastructure ki security AWS ki responsibility hai: 'security of the cloud'. Customer apne workload ke andar access, data, configuration aur application security ka owner hai: 'security in the cloud'. Exact split service ke saath change hota hai.

EC2 example: provider physical hardware sambhalta hai; customer ko guest operating-system patching, app code, network security rules, identity permissions aur data protection manage karni hoti hai. Managed database mein provider aur operational tasks handle kar sakta hai, lekin database users, access policies, data classification aur settings customer ko design karni hoti hain.

Cloud security software-based controls se implement ho sakti hai: network filtering, identity and access control, encryption, configuration monitoring aur security findings. AWS guidance, documentation, partners aur advisory information available kara sakta hai; incident response plan aur customer-specific controls ka owner customer hi hai.

Provider ki data-center safeguards aur audited controls ek foundation provide karte hain. Data access permissions galat ho to strong physical security public bucket ko private nahi bana degi.

Security monitoring, protection aur response continually operate karne honge; automatic perfect security assume mat karo.

Security benefits ko balanced tareeke se samjho

• Data protection: physical safeguards aur encryption/access-control tools milte hain; correct configuration aur key access important hai.

• Compliance support: provider ke certifications aur reports assessment evidence ka hissa ban sakte hain; customer ke apne controls bhi audit honge.

• Operational cost: physical facilities na chalane se kuch costs shift ho sakte hain; total cost workload aur controls par depend karti hai.

• Scale: policies aur tools multiple resources tak apply karne mein help karte hain, lekin continuously validation aur remediation chahiye.

Compliance aur evidence

Compliance bhi shared responsibility hai. AWS apni infrastructure controls ka evidence provide karta hai; customer apne data handling, workload settings, identity, incident response aur relevant business obligations ke controls demonstrate karta hai. Listed assurance examples mein SOC 1/ISAE 3402, SOC 2, SOC 3, FISMA, DIACAP, FedRAMP, PCI DSS Level 1 aur ISO 9001, 27001, 27017, 27018 jaise frameworks hain. Framework name dekhkar current scope ya kisi individual service ki certification assume mat karo: latest report, Region, service aur applicable version verify karo.

Audit evidence mein applicable reports, certifications aur independent attestations useful ho sakte hain. AWS Artifact relevant reports aur agreements access karne ka starting point ho sakta hai. Organisation ko apne procedures, control implementation aur evidence bhi maintain karne chahiye.

AWS services ka access: console, CLI, SDK

Management Console: browser-based graphical interface. Resource settings inspect karne, available service features navigate karne aur suitable actions perform karne mein help karta hai. Selected account aur Region check karo. Mobile console application supported information aur actions dekhne ka option ho sakti hai.

AWS CLI: terminal mein commands ke zariye multiple services manage karne aur scripts automate karne ka unified tool. Command chalane se pehle caller identity, account, Region, permissions aur target resource confirm karo.

CloudShell: console se browser-based shell access mil sakta hai. Shell ka authentication aur current permissions use-case ke hisaab se verify karo; credentials ya sensitive output public screenshots mein share mat karo.

SDK: programming-language/platform ke liye API libraries, jisse application AWS service APIs call kar sakti hai. SDK role-based temporary credentials use karne ke patterns support kar sakte hain; long-lived secrets code mein hard-code mat karo.

Quick revision

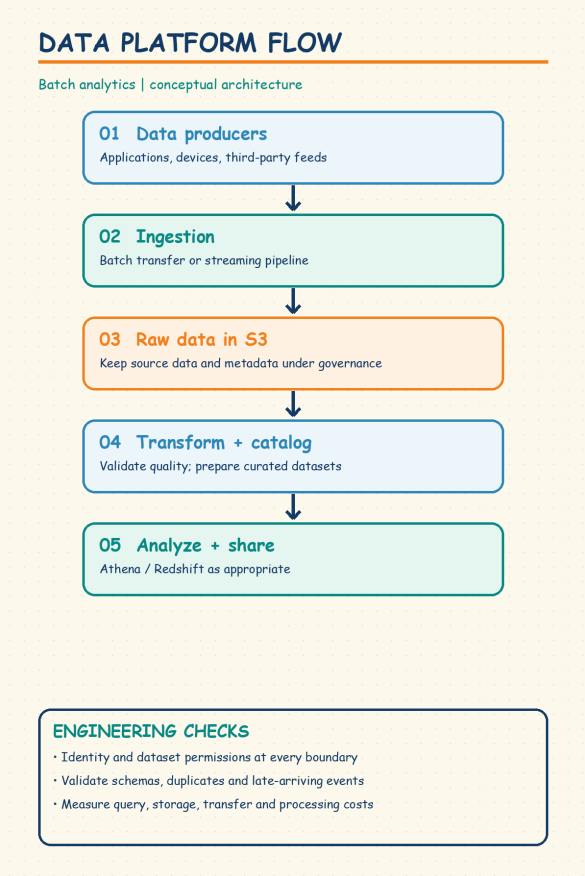
• Cloud provider ka data center secure hona kya customer ki bucket policy ko safe banata hai? Nahi. Customer access configuration ka owner hai.

• Region aur AZ same cheez hain? Nahi. Region mein multiple AZs hoti hain.

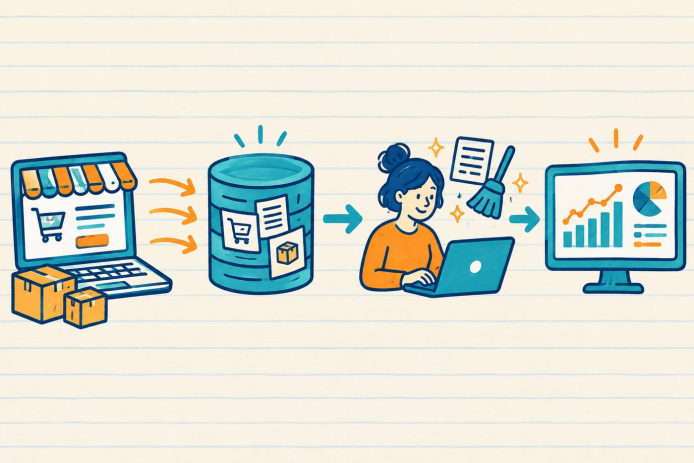
• Hybrid ka matlab sirf cloud mein sab deploy karna hai? Nahi. Cloud aur existing non-cloud resources connected hote hain.

• Budget alert lagate hi spending automatically band ho jayegi? Zaroori nahi. Alert aur automatic enforcement alag cheezein hain.

• Compliance report dekh lene se application certified ho jata hai? Nahi. Report scope aur customer ke apne controls dono assess karne hote hain.



Analytics: data se decision tak



TASVEER SE SAMJHO
Analytics: data se decision

Shop ke orders jama karo, zarurat ke hisaab se saaf aur process karo, phir dashboard par pattern dekho. Har step ke liye access aur data quality check alag se zaroori hai.

Data > Ingest > Process > Insight

Pehle poori picture dekho

Socho ek online store ke paas sales orders, website clicks, product returns, delivery times aur customer support messages hain. Agar ye alag systems mein locked rahen to ek sawaal — 'kaunse products ki return rate zyada hai?' — ka jawab nikalna mushkil ho sakta hai. Analytics workflow ko paanch steps mein dekho: collect → store → prepare → analyse → share. Security aur governance har step ke around rehte hain.

Batch vs streaming: batch mein data pehle jama hota hai aur phir scheduled analysis chalti hai. Streaming mein records aate hi process kiye ja sakte hain. Streaming faster insight de sakti hai, lekin duplicates, late events aur failure recovery ka design important hai.

1. Amazon Athena: S3 par SQL

Athena interactive query service hai jo S3 mein rakhe compatible data par SQL chalane deti hai. Isse samajhne ke liye maan lo reports bucket mein order files hain. Data ka schema — columns aur unke types — describe karo; phir SQL se SELECT product, SUM(amount) ... jaisi query run karo. Serverless model mein query infrastructure manually manage karna zaroori nahi, lekin S3 data, query results aur queries se associated pricing alag-alag check karni chahiye.

Data ko analyse karne ke liye har baar pehle complex ETL pipeline banana zaroori nahi. Kabhi raw ya existing files par directly query useful hoti hai; doosre cases mein data quality ya performance ke liye transformation zaroori ho sakti hai. Results ka time data size, format aur query par depend karta hai; seconds ka promise mat samjho.

Glue Data Catalog connection: metadata ka matlab data ke baare mein information, jaise table name, column schema aur partition definitions. Glue crawler supported data sources ka structure discover karke catalog update karne mein help kar sakta hai. New ya modified table/partition definitions aur schema versions ko manage karke same metadata multiple services mein use ki ja sakti hai.

Decision: ek analyst S3 data par occasional SQL chalana chahta hai — Athena evaluate karo. Large warehouse-centric reporting, workload management aur defined warehouse model chahiye to Redshift bhi compare karo.

2. Amazon CloudSearch: website par search

CloudSearch managed search solution set up, manage aur scale karne ka option hai. Website mein product search chahiye, to documents/index aur searchable fields define karne honge. Search features mein autocomplete, matching text highlighting aur geospatial searches jaise use cases aate hain; language support service configuration aur current documentation se verify karo. 34 languages ka historical figure milta hai, lekin current support list independently check karo.

Example: user 'wirel' type karta hai; autocomplete 'wireless headphones' suggest kar sakta hai. Search result mein match hua word highlight hota hai. Location query se nearby matching items discover karne ka scenario alag design hai. Search index permissions aur data freshness plan karo.

3. Amazon DataZone: data ka organised catalogue

DataZone data publication, discovery aur governed access workflows organise karne mein help karta hai. Socho alag teams ke paas sales data, inventory aur customer interactions hain; ek business catalogue mein dataset ka naam, meaning, owner aur access process milta hai. Users personally relevant web experience se available data discover kar sakte hain.

Data AWS par, on-premises ya supported SaaS applications mein ho sakta hai; integrations aur supported paths check karna zaroori hai. Athena, Redshift, Glue, Lake Formation aur BI workflows se use karne ka scenario hai. Catalogue entry visible hona access grant ke barabar nahi: owner approval, underlying permissions aur data protection rules separately validate karo.

4. Amazon EMR: bade data par open-source processing

EMR big-data processing environments manage karne ka platform hai. Supported open-source tools mein Spark, Hive, HBase, Flink, Hudi aur Presto jaise technologies ka use-case aa sakta hai. Example: saalon ke clickstream logs ko process karke customer journey ka aggregate banana. Platform cluster capacity provisioning aur selected operational tasks simplify kar sakta hai; job logic, dependencies, security, data quality aur cost ab bhi design karne hote hain.

Workload deployment options mein EC2, supported EKS-based deployments aur Outposts ka use-case ho sakta hai. Engine version, location aur available features separately verify karo. Marketing performance comparisons ya fixed percentage ko apne workload par guaranteed result mat samjho: representative benchmark chalao.

5. Amazon FinSpace: financial data ko ready karna

Financial organisations internal portfolio, actuarial aur risk systems se data use karti hain; third-party market feeds, jaise historical security prices, bhi jodti hain. Data dhoondhna, permission lena, compliance check karna aur analysis ke format mein prepare karna time leta hai.

FinSpace financial-services data management aur analytics use cases ke liye design kiya gaya tha. Data ko asset class, risk classification ya geographic region jaise business concepts se catalogue karna, discovery/sharing, centrally defined access policies aur audit logs ka use-case tha. Financial-data preparation functions ke examples time bars aur Bollinger bands hain. Ye historical service description hai: current availability, new-account eligibility aur suitable alternative evaluate karo; preparation time ka specific reduction guarantee nahi hai.

6. Kinesis family: data aate hi use karo

Streaming mein continuous records aate hain, jaise video, audio, app logs, website clicks, IoT telemetry, financial transactions ya location updates. Pura din ka dataset collect hone ka wait karne ke bajay consumer new information par react kar sakta hai. Chaar alag roles samjho: Firehose delivery, Managed Flink stream processing, Data Streams event stream, aur Video Streams video ingestion/playback workflows.

Worked example: store ke clicks stream mein aate hain. Processing component per-minute product views count karta hai, aur delivery component selected destination mein output store karta hai. Late events, duplicate records, data retention aur permission boundaries design ka part hain.

Data Firehose: stream se destination

Firehose streaming data capture karke configured destination tak deliver karne mein help karta hai. Examples mein S3, Redshift, OpenSearch aur supported third-party destinations jaise Splunk aate hain; exact integration current docs se verify karo. Delivery stream create karke producers connect karte hain; service buffering/batching aur supported transformations handle kar sakti hai.

Delivery se pehle supported configurations mein compression, transformation aur encryption configure ki ja sakti hai. S3 delivery ke liye compatible incoming records ko Parquet ya ORC columnar format mein convert karne ke workflows available ho sakte hain. Columnar layout

analytical scans aur storage efficiency mein help kar sakta hai; actual benefit data/query par depend karta hai. Buffering ki wajah se near-real-time ka matlab zero latency nahi.

Managed Service for Apache Flink: stream processing

Jab incoming records par continuously stateful processing chahiye — jaise last 10 minutes ki moving average aur anomaly signal — Apache Flink-based application evaluate karo. Managed service supported Flink applications ke provisioning aur operation ko simplify karta hai. Java/open-source library integrations aur SQL-oriented workflows ka zikr milta hai; actual APIs aur product versions current docs se confirm karo.

Event time, windowing, checkpoints, backpressure, retries aur late records ka correct behavior application designer ko plan karna hota hai. Managed scaling ko infinite throughput ya zero operations ka promise mat samjho.

Kinesis Data Streams: record stream and consumers

Data Streams continuously records ingest karne aur consumers ko processing ke liye available karne ka option hai. Use cases clickstreams, database change events, financial events, application logs, dashboards, anomalies aur dynamic pricing ho sakte hain. Throughput, latency, retention, ordering aur retry behavior configuration aur workload par depend karte hain; per-shard/stream design aur current limits check karo.

Example: order events stream mein aaye. Fraud-check consumer aur reporting consumer alag rates se process kar sakte hain. Consumer fail/restart ho to repeated processing se extra charge ya duplicate order create na ho: idempotency design karo.

Kinesis Video Streams: connected devices ka video

Connected devices se video ingest, secure storage/indexing aur supported APIs se access/playback workflows mein help karta hai. Live aur on-demand viewing aur supported Rekognition Video ya ML/computer-vision libraries ke saath integration use cases ho sakte hain. Camera consent, restricted access, retention aur network interruption plan karo. Hardware counts aur platform support current documentation se verify karo.

7. OpenSearch Service: search aur operational analytics

OpenSearch deployments search, near-real-time analysis aur visualisation use cases support karte hain. Examples: application log analytics, full-text document search, application monitoring aur clickstream analysis. OpenSearch Dashboards aur Logstash jaise open-source tools ke saath supported ingestion/visualisation workflows ho sakte hain.

Networking aur security mein VPC, KMS encryption, IAM/Cognito access controls aur monitoring ke liye CloudWatch jaise integrations ka use-case hai. Firehose aur Lambda ingestion pipelines ka hissa ho sakte hain. Index mapping, retention, access permissions, node capacity aur ingestion failures team ko plan karne honge.

OpenSearch Serverless aur vector search

Serverless option mein supported workloads ke liye cluster infrastructure ka manual configuration aur scaling responsibility kam hoti hai; capacity, latency aur cost ko still monitor karo. Vector engine embedding-based similarity search ke liye relevant hai: image/document search, product recommendation, audio/video retrieval, location-based discovery, fraud/anomaly patterns jaise applications. Vector search relevance, permissions aur model behavior test karna zaroori hai; har query ka response time guaranteed nahi.

8. Amazon Redshift: warehouse analytics

Redshift data warehouse analytics ke liye SQL aur BI workflows support karta hai. Structured aur semi-structured datasets par large analytical queries ko query optimisation, columnar storage aur parallel processing approaches se serve kiya ja sakta hai. Sales, inventory aur returns ko model karke monthly profitability report banana example hai. Actual query time data layout, scale, concurrency aur workload par depend karega.

Historical pricing examples fixed amount de sakte hain, lekin inhe current rate mat samjho. Region, capacity model, storage, data movement aur workload configuration ke current pricing rules check karo.

Redshift Serverless

Warehouse capacity provisioning/scaling ka operational model simplify karta hai. Analysts aur developers reporting, dashboards, supported data-sharing aur data-lake/warehouse workflows chala sakte hain. Query Editor ya compatible BI tool se query ki ja sakti hai. 'Serverless' ka matlab zero cost, zero configuration ya every workload ka automatic performance guarantee nahi hai.

9. Quick: BI aur dashboards

Business intelligence tool ka kaam data se interactive reports banana aur users tak pahunchana hai. Dashboard browser ya supported mobile experience mein view ho sakta hai; embedded analytics se application ke andar bhi dikhaya ja sakta hai. Example: sales manager region-wise revenue filter kare. Viewer ke permissions, source dataset refresh, metrics definition aur current product naming/features check karo. Dashboards share karne se pehle customer-level data exposure test karo.

10. AWS Clean Rooms: raw data share kiye bina collaboration

Do organisations collective data ka supported analysis karna chahti hain, par apna underlying raw dataset directly ek-doosre ko hand over nahi karna chahti. Clean Rooms analysis rules aur supported collaboration workflow provide karta hai. Use cases advertising campaign measurement, investment research aur joint R&D ho sakte hain.

Example: retailer aur advertiser permitted aggregate overlap measure karte hain. Allowed query, result disclosure, minimum aggregation aur contractual privacy requirements pehle define karo. Clean room apne aap consent, legal compliance ya all privacy risks solve nahi karta.

11. AWS Data Exchange: third-party datasets

Data Exchange third-party data products discover aur subscribe karne ka workflow hai. Examples ke taur par market news, health claims aggregates, business records ya location datasets relevant ho sakte hain. Provider claims aur dataset statistics ko timeless fact mat samjho: license, provenance, updates, terms, permissions aur quality verify karo.

Subscription ke baad supported API workflows se dataset S3 mein laakar analytics ya ML tools mein use kiya ja sakta hai. Insurer historical weather se coverage analysis kare, restaurant population/location data se expansion evaluate kare, researcher emissions data study kare, ya healthcare team aggregated trial datasets analyse kare — ye illustrative use cases hain.

Dataset providers ke liye distribution, entitlement aur billing workflows ko platform help kar sakta hai. Sensitive data ke liye permitted use, privacy rules aur access control clear hone chahiye.

12. AWS Data Pipeline: scheduled movement and processing

Data Pipeline historical scheduling/movement service tha jisse AWS compute/storage aur on-premises sources ke beech scheduled processing workflows banaye ja sakte the. Sources aur destinations mein S3, RDS, DynamoDB aur EMR use cases aate hain. Task dependencies, retry/failure handling aur notifications complex workflow mein useful concepts hain.

Nayi architecture mein pehle service lifecycle aur supported alternatives verify karo; schedule, dependencies aur failure handling ka plan kisi bhi replacement mein zaroori rahega.

13. AWS Entity Resolution: related records jodna

Ek customer ka record web app, loyalty program aur e-commerce database mein alag IDs se ho sakta hai. Entity Resolution supported rule-based aur ML-based matching workflows se related records identify/link karne mein help karta hai. Examples: duplicates detect karna, customer interactions connect karke profile banana, aur marketing ya loyalty use cases.

Example: ad click, abandoned cart aur purchase ko appropriate matching rules se ek match ID ke under relate kiya ja sakta hai. False positives se galat logon ka data merge ho sakta hai; false negatives se same person multiple profiles mein reh sakta hai. Consent, identifiers ki protection aur match quality evaluate karo.

14. AWS Glue: catalogue, transformation aur quality

ETL ka simple matlab: Extract data nikalo, Transform clean/reshape karo, Load suitable destination mein rakho. Glue supported data integration jobs aur Data Catalog provide karta hai. Data store discover karke table definitions aur schema jaise metadata catalogue kiye ja sakte hain; catalogued data ko compatible tools query ya transform kar sakte hain.

Glue data integration engines mein Spark, PySpark aur Python-based workflows relevant hain; Ray-based option bhi described hai. Supported engine/version, execution model aur costs current docs se verify karo.

Glue Data Quality: dataset ki statistics aur quality rules se missing, stale ya invalid data detect/monitor karne mein help mil sakti hai. Example rule: order_id blank na ho, amount negative na ho, aur latest ingestion expected interval mein aaye. Alerts ka owner aur failed records ka quarantine process define karo. Automated quality checks business correctness ka complete replacement nahi hain.

15. Lake Formation: governed data lake

Data lake central repository hai jahan original/raw aur analysis-ready datasets store kiye ja sakte hain. Objective: data silos todna aur alag analytics/ML tools ko governed data dena. Data-lake setup mein diverse source ingestion, flow monitoring, partitions, encryption/key management, transforms, columnar layout, deduplication, record matching, access grants aur audit jaise tasks involve ho sakte hain.

Lake Formation data-lake setup aur fine-grained governance workflows mein help karta hai. Data locations aur policies define karne ke baad supported catalogue, data preparation aur access-management integrations use hote hain. Users approved data ko Athena, EMR, Redshift, SageMaker AI ya BI tools se analyse karne ka design bana sakte hain. S3/IAM/KMS aur Lake Formation permissions ka interaction verify karna zaroori hai; service choose karte hi saare controls automatically correct nahi ho jate.

16. Amazon MSK: managed Kafka

Apache Kafka streaming pipelines aur applications ke liye platform hai. MSK Kafka-based infrastructure ko run karne ka managed option provide karta hai. Kafka APIs se event pipelines, data-lake population, database changes aur analytics/ML workflows banaye ja sakte hain.

Self-managed Kafka mein servers provision karna, clusters configure karna, failed servers replace karna, patch/upgrade, high availability, secure/durable data, monitoring aur scaling manage karne padte hain. Managed approach inmein se kuch operational tasks simplify karti hai. Topic partitions, producer/consumer semantics, retention, application failures, schema evolution aur access policies team ko design karni padti hain.

Sabko ek real system mein jodo

• Website activity aur order events aate hain.

• Relevant events stream ya delivery workflow mein jaate hain; destination S3 ya doosra supported system hai.

• Glue metadata catalogue aur optional ETL/data-quality checks lagata hai; Lake Formation governed access workflows provide kar sakta hai.

• Athena SQL se S3 data query karti hai; Redshift warehouse-centric analytics ka alternative ya complementary component ho sakta hai.

• BI dashboard approved aggregated results share karta hai. Search/log use cases ke liye OpenSearch separate component ho sakta hai.

Is flow mein ek bhi service compulsory nahi: latency, data access, team skill, Region, failure recovery, privacy aur price ke hisaab se architecture badlega.

Self-check

• S3 files par ad-hoc SQL: Athena; long-running large Spark processing: EMR evaluate karo.

• Stream delivery aur stateful stream processing same cheez nahi: Firehose aur Managed Flink ke roles compare karo.

• Catalogue entry visible hone se sensitive data read permission automatically milti hai? Nahi.

• Clean-room collaboration se privacy aur contracts ki responsibility khatam? Nahi.

• Cloud bill ya query latency ka old marketing figure production estimate hai? Nahi; current pricing aur actual benchmark use karo.

Application integration: systems ko baat karna sikhao

TASVEER SE SAMJHO
Integration: order se alag-alag kaam

Order event se queue mein kaam ruk sakta hai aur notification ya analytics ko alag event mil sakta hai. SQS, SNS aur EventBridge ke roles same nahi; agle notes unka farq samjhate hain.

Event > Route / queue > Worker > Result

Story: online store ka order

Order aate hi payment, warehouse, email aur reporting teams ko alag kaam karna hota hai. Agar checkout service har doosri service ke response ka wait kare, ek failure poore checkout ko slow ya unavailable kar sakta hai. Decoupling ka matlab components ko aise connect karna hai ki ek component ko har doosre component ki internal details par tightly depend na karna pade.

Puri application ko ek saath rewrite karna zaroori nahi: ek boundary, jaise order notification, se shuru kar sakte ho. Isse components ko independently update aur release karne mein help mil sakti hai. Trade-off: asynchronous systems mein duplicate messages, retries, delayed processing, monitoring aur data consistency ko carefully design karna hota hai.

Map: workflow steps coordinate karne ke liye Step Functions; SaaS data flows ke liye AppFlow; business documents ke liye B2B Data Interchange; event routing ke liye EventBridge; scheduled task graphs ke liye MWAA; broker compatibility ke liye MQ; fan-out ke liye SNS; queue aur workers ke liye SQS; background task coordination ke liye SWF.

AWS Step Functions: workflow ka traffic controller

Distributed application ko individual steps mein break karo. Step Functions state machine in steps ko coordinate karti hai; visual console se flow dekh aur edit kar sakte ho. Har step ka input, output, transitions, failures aur timing define hota hai. Execution history aur configured logs diagnosis mein help karte hain. Retries aur error handling design ki ja sakti hai; automatic correct business outcome ka assumption mat karo.

Example: Order received → inventory reserve → payment charge → shipment request. Payment fail ho to inventory release karne ka compensation step add karo. Agar shipment request fail hone ke baad refund bhi fail ho, to separate retry aur human review path chahiye. Payment API ko duplicate attempt se bachane ke liye idempotency key rakho.

Visual composition se kuch changes code likhe bina kiye ja sakte hain, par underlying functions aur integration code may still need changes. Standard/Express workflow choice, timeouts, role permissions, sensitive state data aur charges requirements ke hisaab se check karo.

Amazon AppFlow: SaaS aur AWS ke beech data flow

AppFlow supported SaaS applications — jaise Salesforce, Zendesk, Slack ya ServiceNow — aur destinations jaise S3/Redshift ke beech data-transfer workflows configure karne mein help karta hai. Connectors aur supported source/destination combinations time ke saath change ho sakte hain.

Flow schedule par, supported business event ke response mein, ya on demand run ho sakta hai. Field filtering, mapping, transformation aur validation se data ko downstream use ke liye prepare kiya ja sakta hai. Transit encryption aur supported PrivateLink integration se selected flows ka

network exposure reduce karne ke options ho sakte hain; supported SaaS aur specific setup verify karo.

Example: support system se closed tickets har raat S3 mein aate hain; sirf team, category aur closure duration fields rakhte ho; personal message content unnecessary ho to transfer mat karo. Credentials, authorisation, retention, failure notification aur schema changes ka plan banao.

AWS B2B Data Interchange: business documents ko readable data mein badlo

Companies suppliers aur customers se purchase orders, invoices aur shipping notices electronic data interchange (EDI) standards, jaise X12, mein exchange kar sakti hain. EDI documents ko direct business applications consume karna difficult ho sakta hai; B2B Data Interchange supported EDI ko JSON/XML jaise formats mein transform karne aur trading-partner configurations manage karne mein help karta hai.

Flow: supplier ka EDI purchase order → partner identification → format validation/transformation → JSON → order processing ya data lake. Low-code configuration integration effort reduce kar sakti hai, lekin partner onboarding, mapping correctness, rejected documents aur access policy review remain. Structured output ko analytics ya AI workflow mein bhejne se pehle sensitive data aur business rules validate karo.

Amazon EventBridge: event ke hisaab se routing

Event = koi ghatna, jaise OrderPlaced. EventBridge event bus supported application, AWS service aur SaaS events le sakta hai. Rules event ke fields match karke event ko targets — jaise Lambda ya supported SaaS integration — ki taraf route karte hain. Publisher ko har consumer ke internals jaanne ki zarurat nahi.

Example: OrderPlaced se inventory updater, analytics workflow aur customer notification alag targets ban sakte hain. Har target independent failure face kar sakta hai. Event ID, type, schema version, source aur event time define karo; rule patterns test karo; target permission, retry, dead-letter handling aur idempotency configure/verify karo. 'Real time' ko guaranteed instant delivery mat samjho.

Amazon MWAA: scheduled task graph

Apache Airflow open-source tool hai jisme workflows code se author, schedule aur monitor kiye jate hain. Workflow aksar DAG hota hai: directed acyclic graph, jahan tasks aur dependencies defined hain. Amazon Managed Workflows for Apache Airflow underlying environment operate karne ka managed option hai; Python se compatible workflows create kiye ja sakte hain.

Example: 1 baje sales file ingest → data quality check → transform → warehouse load → report refresh. Quality check fail ho to dependent steps ka behavior aur alerts specify karo. Managed

environment ke bawajood workflow code, secrets, IAM permissions, dependency versions aur data correctness owner ko handle karna hota hai.

Amazon MQ: existing broker protocols ke saath messaging

Different platforms ya programming languages ke applications message broker ke through information exchange kar sakte hain. Amazon MQ supported Apache ActiveMQ Classic aur RabbitMQ brokers ki setup, provisioning aur maintenance simplify karne ke liye managed option hai.

Existing applications relevant standard protocols/APIs — jaise JMS, NMS, AMQP, STOMP, MQTT aur WebSocket — use kar sakti hain, lekin broker aur selected protocol ke actual supported combination verify karo. Compatibility migration effort kam kar sakti hai; har application bina changes migrate hogi, aisa guarantee nahi. Authentication, network access, durability, broker limits aur failure behavior test karo.

Amazon SNS: ek event, multiple subscribers

SNS publish/subscribe pattern mein publisher topic par message bhejta hai aur supported subscribers ko push-based fan-out hota hai. Subscribers SQS queues, Lambda functions, HTTP/S endpoints ya supported end-user channels — mobile push, SMS, email — ho sakte hain. Each subscription ke delivery aur permissions check karo.

Example: payment complete event ek topic par publish hota hai. Warehouse queue packing start karti hai; notification function customer ko message bhejti hai; analytics subscriber record store karta hai. Har downstream system alag speed aur failure behavior rakhta hai. Retry, duplicate handling aur confidential payload minimisation design karo.

Amazon SQS: waiting line aur workers

SQS managed message queue provide karta hai. Producer message bhejta hai; worker receive karta hai; processing success hone par message delete karta hai. Receive ke baad visibility timeout message ko temporarily dusre consumers se hide karta hai; timeout khatam hone ya processing fail hone par message dobara aa sakta hai. Queue producer aur worker ko temporarily unavailable hone par decouple kar sakti hai, lekin retention, limits aur failure behavior configure/verify karne honge.

Standard queue: high-throughput use cases; at-least-once delivery aur best-effort ordering. Isliye duplicate aur out-of-order messages handle karo.

FIFO queue: message group ke andar ordering aur deduplication features provide karti hai. Ye application ke external side effects ke liye blanket exactly-once guarantee nahi: worker payment commit karne ke baad delete se pehle crash kar sakta hai. Stable business idempotency key aur conditional state update se duplicate payment guard karo.

Failure drill: queue mein poison message repeat ho raha hai. Dead-letter queue/redrive policy explicitly configure karo, alarm lagao, root cause fix karo; messages replay karne se pehle side effects ka duplicate risk check karo. Queue console, CLI ya SDK se operate kar sakte ho.

Amazon SWF: background steps aur state tracking

Simple Workflow Service background workflows ke sequential ya parallel steps coordinate karne aur state track karne mein help karta hai. Long-running processes mein har stage ki progress, retries aur recovery track karna useful ho sakta hai. Isko Step Functions se choose karte waqt existing application design, programming model, supported integrations aur operational requirements compare karo; kisi fixed millisecond threshold ko universal rule mat samjho.

Decision drill: kaunsi service kyon?

• Ek order event se multiple independent actions: SNS fan-out ya EventBridge routing evaluate karo; target model aur requirements compare karo.

• Worker temporarily unavailable ho aur work queue mein wait kare: SQS evaluate karo.

• Payment aur inventory ke multiple sequential steps plus compensation: Step Functions consider karo.

• Existing RabbitMQ-compatible broker requirement: Amazon MQ evaluate karo.

• Nightly data pipeline with task dependencies: MWAA evaluate karo.

• SaaS se scheduled data copy: AppFlow; partner EDI ko business-readable format mein transform: B2B Data Interchange.

Blockchain: jab multiple parties ka shared ledger chahiye

Blockchain distributed ledger architecture ka ek family hai, jahan multiple parties agreed protocol ke through transactions record/validate kar sakti hain. Isse har scenario mein central trusted database ki zarurat khatam nahi hoti: network governance, identity, legal trust aur data accuracy ab bhi business decisions hain. Trusted single owner wale use case mein normal database simpler ho sakta hai.

Managed Blockchain supported blockchain/Web3 network capabilities ko operate karne mein help karne ke liye managed service family hai. Historical examples mein Hyperledger Fabric aur Ethereum ka zikr aata hai; aaj ke supported network, product lifecycle aur new deployment options verify karo.

Self-managed setup mein member hardware provision karna, software install karna, certificates aur permissions manage karna, network configure karna, transaction demand monitor karna aur members joining/leaving handle karna pad sakta hai. Managed capabilities in selected operational tasks ko simplify kar sakti hain; exact responsibilities current feature par depend karti hain. Compute, memory, storage aur application security monitor karo.

Example: supply-chain participants agreed events record karte hain. Agar wrong shipment event enter hua, ledger ki permanence us input ko correct nahi bana deti. Data validation, correction protocol, confidentiality aur participants ki identity define karo.

Business applications: team aur customers ko connect karo

TASVEER SE SAMJHO

Business applications: logon ko jodo

Chat, meeting aur customer-support tools alag needs solve karte hain. User access, privacy aur product availability ko har service ke liye alag check karo.

People > Workflow > Measure > Improve

Story: ek growing business

Riya ki online shop badh rahi hai. Ab employees ko email/calendar, customer support ko contact center, developers ko transactional email aur security team ko SaaS activity visibility chahiye. Har kaam ke liye custom system banana zaroori nahi; managed applications ya APIs evaluate kar sakte hain. Product availability, permissions, data residency, subscription aur actual costs separately verify karo.

AWS AppFabric: SaaS security activity ko ek common language

Organisation ki sales, support aur collaboration teams alag SaaS apps use kar sakti hain. Pehle har app ko security tooling se jodne ke liye separate point-to-point integration banana aur maintain karna pad sakta tha. AppFabric supported authorised SaaS apps se security data ingest/aggregate karke normalise karne mein help karta hai.

OCSF (Open Cybersecurity Schema Framework) common security event schema ka example hai: alag apps ke logs ko consistent structure mein lana analysis aur alerting easier kar sakta hai. Security teams supported event activity, policies aur user-access workflows evaluate kar sakti hain. App ko connect karne se pehle permissions, collected data, supported connectors, retention aur current service status check karo. Normalised log alone incident investigation complete nahi karta.

Amazon Chime aur Chime SDK: application vs building block

Amazon Chime meeting/communication application tha jisme online meetings, video calls, chat aur content sharing jaise uses the. Device compatibility aur internal/external participants application configuration par depend karte hain. Historical integration example Alexa for Business ke through voice se meeting join/start karna tha. Dono products/integrations ki present availability aur support ko deploy ya recommend karne se pehle verify karo.

Chime SDK alag hai: developers supported voice, video aur messaging capabilities apni application mein build kar sakte hain. Ready-made meeting app aur app-banane ke SDK ko confuse mat karo. User consent, identity, recording, regional availability aur communication cost design decisions hain.

Customer contact center: contact flow samjho

Amazon Connect customer-contact workflows ke liye cloud contact-center capabilities provide karta hai. Omnichannel ka matlab business configured channels par interactions manage kar sakta hai; exact channel support confirm karo. Graphical flows se incoming contact ko appropriate queue/agent tak route karna, agents manage karna aur performance metrics dekhna possible ho sakta hai.

Example: customer order delay ke liye contact karta hai → flow language choose karata hai → order-support queue assign hoti hai → agent interaction handle karta hai → manager wait time aur resolution metrics dekhta hai. Telephony aur service usage ke pricing components alag ho sakte hain. Security, call recording consent, staffing aur customer data permissions organisation ko define karni hongi.

Amazon Pinpoint: targeted messages aur campaigns

Pinpoint customer engagement aur messaging workflows ka historical example hai. Audience segments create karke promotional, educational ya retention campaigns ko schedule karna aur email, SMS ya push jaise channels use karna possible tha. Order confirmation aur password reset transactional messages ke examples hain; supported transactional channels mein voice use-case bhi ho sakta tha.

Application integration se usage events aur delivery, open/click jaise metrics collect kiye ja sakte the; console/API se supported campaigns aur custom applications build karne ke options the. Messaging consent, sender reputation, opt-out aur analytics privacy important hain. Service ke current lifecycle aur recommended replacement deployment se pehle verify karo; feature/availability ko permanent mat samjho.

Amazon SES: application email bhejna

Simple Email Service application se transactional, marketing ya bulk email send karne ke liye evaluate kiya ja sakta hai. Authentication aur supported IP deployment choices sender reputation aur deliverability affect karte hain; sending analytics se deliveries, bounces aur other signals monitor kiye ja sakte hain.

Example: checkout complete hone par order confirmation. Domain/sender identity verify karo, email content mein unnecessary personal data mat bhejo, bounce/complaint handling set karo aur applicable consent requirements follow karo. Har message inbox tak pahunchna guaranteed nahi; email deliverability ongoing operational responsibility hai.

Amazon WorkDocs: document collaboration ka historical case

WorkDocs managed enterprise document storage/sharing aur administrative controls ka example tha. Users files par comments de sakte, feedback share kar sakte aur nayi versions upload kar sakte the — multiple email attachments ke bajay central version control jaisa workflow. PCs, Macs, tablets aur phones se supported access; corporate directory integration, sharing policies aur data location controls ka use-case tha.

Naye customer sign-ups aur account upgrades ke liye restrictions reported hain. Current lifecycle, migration instructions aur replacement options verify karo. Confidential documents par sharing aur directory permissions review karna zaroori hai.

Amazon WorkMail: email, calendar aur contacts

Managed business email/calendar service supported desktop aur mobile email clients ke saath use ki ja sakti hai. Outlook, supported native mobile apps, IMAP clients aur browser access jaise options evaluate karo; actual client/version support confirm karo. Existing corporate directory integration, supported journaling, data encryption keys/location settings aur Exchange interoperability relevant enterprise concerns hain.

Users, groups aur resources ke programmatic management ke liye supported APIs/SDK use ho sakte hain. Migration se pehle mailbox scope, authentication, retention, licensing, cutover aur rollback plan banao.

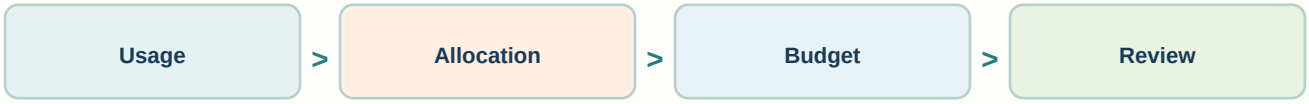
Decision drill

• App se order email: SES evaluate karo. Team meetings application aur communication SDK ek cheez nahi.

• Customer-support queue aur contact routing: contact-center capability evaluate karo, email service se confuse mat karo.

• Multiple SaaS security logs: integration/normalisation capabilities evaluate karo. Access approvals aur incident ownership still required hain.

Cloud financial management: bill samjho, surprise kam karo



Story: □1,000 budget ka matlab stop button nahi

Shop ne app deploy ki; next month compute, storage, requests, network transfer aur other services ka bill bana. Sabka owner aur driver samajhna zaroori hai. Price per resource aur usage duration se estimate banta hai; actual charges pricing rules, Region, commitments, taxes aur transfer par depend karte hain. Budget alert notification hai, automatic hard spending cap nahi.

AWS Billing Conductor: showback aur chargeback

Showback mein team ko dikhaya jata hai ki unke resources ka cost kitna attributed hai; chargeback mein business accounting ke through team/customer se cost charge ki ja sakti hai. Billing Conductor billing groups aur pricing plans ke through customised pro forma billing view create karne mein help karta hai.

Example: company do business units ko custom rates ke saath monthly cost report dena chahti hai. Billing Conductor group-specific view bana sakta hai aur custom rate ko actual AWS rate se compare karne mein help kar sakta hai. Important: ye actual AWS bill ki calculation automatically change nahi karta. Console ya API se supported group/plan settings, quotas aur reporting configuration manage ki ja sakti hai.

AWS Cost Explorer: bill ko slice karke dekho

Cost Explorer historical cost/usage ko charts ya tables mein explore karne aur custom reports banane mein help karta hai. High-level organisation totals dekho ya service, linked account, time aur activated allocation tags jaise filters use karo. Example: project tag ke basis par development team ka EC2 spend trend dekho.

Tagging consistent nahi to 'untagged' costs miss ho sakte hain. Reports, update timings aur available filters current documentation se check karo. Cost spike dikhe to recent deployments, resource duration aur usage type correlate karo; dashboard dekh lena cost reduction nahi hai.

AWS Budgets: thresholds aur alerts

Budgets cost ya usage thresholds par alerts set karne ka tool hai. Actual spend ya forecast threshold cross kare to configured email/SNS notification mil sakti hai; alert delivery aur owner action verify karo. Monthly, quarterly ya yearly tracking aur applicable start/end dates configure ki ja sakti hain. Service, linked account, tag aur other supported dimensions se scope narrow karo.

Reservation utilisation/coverage targets bhi monitor kiye ja sakte hain. Eligible EC2, RDS, Redshift aur ElastiCache reservation reporting support configuration/version ke anusaar check karo. Console/dashboard ya API se budgets manage ho sakte hain. Supported budget actions alag configuration hain; sirf budget create karne se all resources automatically stop nahi hote.

Cost and Usage Report: detailed line items

Cost and Usage Report (CUR) detailed usage/cost data ka export provide karta hai. Usage hourly ya daily line items, service categories aur activated cost-allocation tags ke saath analyse ki ja sakti hai; supported settings se daily/monthly aggregation choose kar sakte ho. Current report options aur naming check karo.

Example: finance team export ko S3-based analysis pipeline mein use karti hai, costs ko account/team/tag ke saath reconcile karti hai, aur shared networking costs ke allocation rules separately document karti hai. Tag values, data freshness aur data access controls matter karte hain.

Reserved Instance reporting: utilisation vs coverage

Utilisation: aapki purchased eligible reservation kitni use hui? Coverage: eligible usage ka kitna portion reservation se covered hai? Dono same metric nahi. Cost Explorer RI reports aggregated ya particular subscription views mein help karte hain; detailed line items ke liye CUR use ho sakta hai. AWS Budgets se utilisation threshold alerts bhi configure kiye ja sakte hain.

Low utilisation unused commitment risk dikha sakti hai; low coverage ka matlab eligible steady usage ka significant portion on-demand ho sakta hai. Metrics ko usage volatility, terms aur business plans ke saath interpret karo.

Savings Plans: commitment vs flexibility

Savings Plans eligible usage par On-Demand rates ke comparison mein pricing option dete hain, usually ek specified hourly spend commitment ke badle one- ya three-year term ke saath. Examples: Compute Savings Plans eligible EC2/Lambda/Fargate usage cover kar sakte hain; EC2 Instance Savings Plans selected EC2 scope ke liye; SageMaker AI Savings Plans eligible SageMaker AI usage ke liye. Current eligibility, regional/family scope, terms aur rates verify karo.

Example: steady baseline ka measurement kiya aur conservative commitment consider ki. Traffic ya architecture badal gaya to commitment unused reh sakta hai; isliye full peak demand ko blindly commit mat karo. Cost Explorer recommendations, performance reporting aur budgets decision support karte hain, guarantee nahi.

Practice: bill-reduction plan

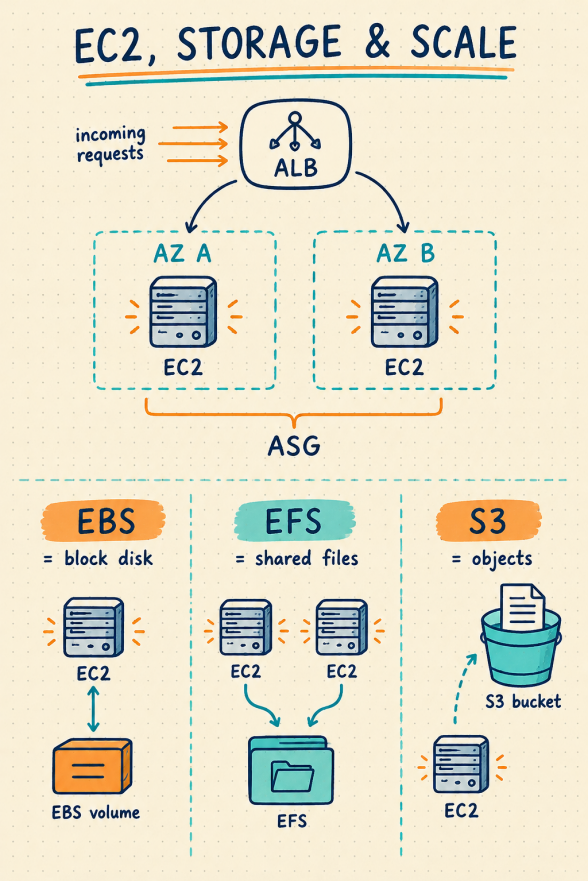
• Pehle current bill aur usage drivers identify karo: instance-hours, storage, requests aur data transfer.

• Ownership/tags aur shared-cost allocation set karo; nonproduction idle resources review karo.

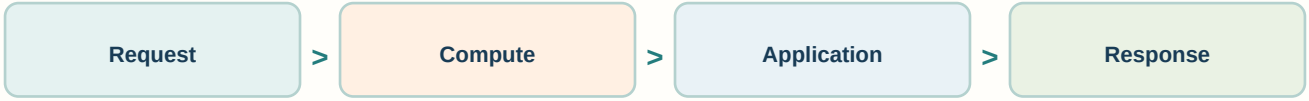
• Budget threshold, recipients aur escalation define karke test karo.

• Stable usage baseline par reservation/Savings Plans ka scenario compare karo.

• Performance, resilience aur security impact test karke changes deploy karo; next billing cycle mein verify karo.



Compute: application ko chalane ki taqat



Kahani: online shop ko kitni machines chahiye?

Riya ki shop par subah kam log aate hain, shaam ko traffic badhta hai aur festival sale par demand spike hoti hai. Usko code chalane ke liye compute capacity chahiye. Lekin har kaam ke liye same virtual machine choose karna zaroori nahi: long-running web server, short image-processing task, large batch job aur mobile-edge application ki needs alag hain.

Choose karne ke pehle chaar sawaal: (1) kitna server/OS control chahiye? (2) workload continuously chalega ya events par? (3) interruption tolerate kar sakta hai? (4) operations aur cost ka owner kaun hoga?

Compute landscape: map yaad rakho

• Virtual machines: EC2 configurable virtual servers deta hai; Lightsail simpler bundled experience hai; Batch batch jobs ko schedule aur compute capacity ke saath run karta hai; EC2 Auto Scaling instance fleet ki capacity aur health manage karne mein help karta hai.

• Containers: ECR images store karta hai; ECS/EKS orchestrate karte hain; Fargate supported ECS/EKS tasks/pods ke liye serverless compute option hai; App Runner supported source/image se web apps chalane ka simplified approach hai. ECS Anywhere aur EKS Anywhere customer-managed infrastructure par respective orchestration use cases address karte hain; support aur responsibility model check karo.

• Functions: Lambda event-driven code execution ke liye hai. Infrastructure ka server administration nahi karte, lekin code, roles, configuration, retries aur downstream dependencies design karte ho.

• Hybrid/edge: Outposts customer site par supported AWS infrastructure/services; Snow Family physical transfer/edge scenarios; Wavelength selected telecom edge locations; Local Zones selected cities ke paas AWS infrastructure; VMware Cloud on AWS VMware-based hybrid workloads ka option. Availability aur lifecycle region/product ke hisaab se check karo.

• Support tools: Savings Plans pricing commitment; Compute Optimizer recommendations; Elastic Beanstalk application environment deployment; Image Builder repeatable image pipeline; ELB incoming traffic distribute karta hai. Ye sab ek hi kaam ke tools nahi.

Amazon EC2: virtual server ka control

EC2 se selected Region/AZ mein virtual server instance launch kar sakte ho. Instance start karte waqt Amazon Machine Image (AMI), instance type, network/subnet, security group, IAM role aur storage jaise choices hoti hain. Application aur OS ko customise karne ka control milta hai, par guest OS patches, access permissions, security aur app reliability tumhe design karni hoti hai.

Capacity minutes mein provision ho sakti hai aur compatible architecture ke saath badhai/ghatai ja sakti hai. EC2 tools fault-resilient application build karne mein help karte hain; ek server launch

karna automatic high availability nahi banata. Pricing instance, purchase option, storage, network aur Region ke hisaab se vary karti hai.

Instance type ka naam: label ko decode karo

Instance name family, generation, processor family/additional capability aur size se milkar ban sakta hai. Family ka purpose samjho aur actual workload benchmark karo:

• C7g: compute-intensive family; AWS Graviton3 processor. Example: CPU-intensive batch, scientific modelling, gaming server, video encoding, ad serving, distributed analytics ya suitable CPU inference. ARM compatibility verify karo.

• M7g: general-purpose family, Graviton3; application servers, microservices, medium data stores ya caches jaise balanced workloads evaluate kar sakte hain.

• R7g: memory-intensive family, Graviton3; in-memory cache, memory-heavy databases aur near-real-time analytics ke candidates.

• Inf2: AWS Inferentia2 accelerator-based instances, deep-learning inference ke liye. Model/runtime support aur accelerator deployment ka testing karo.

• Trn1: AWS Trainium-based instances, deep-learning training use cases, including supported generative models, ke liye. Framework support, model size aur actual training economics compare karo.

'Best' ya historical percentage discount har workload ke liye guaranteed nahi. CPU architecture, binaries, licensing, networking, memory, storage throughput aur software compatibility measure karo.

EC2 purchase options: flexibility aur commitment

On-Demand: eligible compute capacity ke liye usage-based model, without a long-term purchase commitment. Billing granularity OS/instance type aur pricing rules par depend karti hai. Short-term, spiky, unpredictable, non-interruptible ya initial development workloads mein useful ho sakta hai; current price check karo.

Spot: spare capacity ke pricing option ke through discounts mil sakte hain, lekin instance interrupt ho sakta hai. Flexible start/end, fault-tolerant batch, stateless fleets aur checkpointing wale jobs evaluate karo. Historical 'up to' discount har Region/instance/time par guarantee nahi. Urgent work ke liye fallback capacity plan rakho; Spot available hoga hi, assume mat karo.

Reserved Instances: eligible instance usage ke liye term-based commitment aur pricing option. Standard aur Convertible types ke terms/flexibility alag hote hain; Convertible exchange rules current documentation se verify karo. Reservation purchase underlying running instance ko automatically create nahi karti.

Savings Plans: eligible compute use ke liye specified hourly commitment on a one- or three-year term, subject to terms. Compute, EC2 Instance aur other applicable plan types ki coverage/flexibility alag hai. Steady measured baseline se start karo; demand kam ho to unused commitment risk hai.

Dedicated Hosts: physical EC2 server dedicated to a customer, relevant host placement aur certain server-bound software licensing scenarios ke liye. Windows Server, SQL Server ya SUSE

licenses ke use ka permission actual contracts aur current provider rules par depend karta hai. Dedicated hardware ka use apne aap compliance ya lower cost guarantee nahi karta.

Worked example: photo-processing workers

Shop ke paas 10,000 photos resize karne ki queue hai. Work 5 minutes ruk sakta hai aur har image individually retry ho sakti hai. Spot workers cost option ho sakte hain; queue aur idempotent output design karo. Agar a payment API cannot stop, On-Demand ya appropriate resilient baseline consider karo. Dono scenarios mein total cost mein storage, data transfer, logs aur idle resources bhi add karo.

EC2 Auto Scaling: fleet ko healthy aur responsive rakho

Auto Scaling group mein min, desired aur max capacity define ki ja sakti hai. Fleet management unhealthy instances replace karne mein help karta hai; dynamic scaling measured demand ke basis par adjust kar sakti hai; predictive scaling historical patterns ke basis par forecast-driven capacity schedule kar sakti hai. Dono policies compatible configuration mein saath use ho sakti hain.

Example: usual traffic par 2 instances; metric threshold cross hone par capacity increase; demand normal hone par scale in. Health-check path, warmup, launch template, AZ placement, cooldown aur capacity quota test karo. Database saturated ho to extra web servers har baar solution nahi honge.

Practice: maximum capacity 20 set hai. Load balancer healthy targets nahi dekh raha: application port, security group, health endpoint, startup time aur deployment logs check karo; sirf desired count badhana kaafi nahi.

EC2 Image Builder: machine images repeatable banao

Image Builder VM aur supported container image build/test/distribution pipelines mein help karta hai. Manual patch → snapshot → distribute process mein version drift aur missed updates aa sakte hain. Pipeline mein base image choose karo, components/updates apply karo, tests chalao aur approved image distribute karo; AWS-provided components aur security settings ke scope ko review karo.

Use case AWS ya supported on-premises environments ke images ho sakte hain. Image Builder ka service charge model aur underlying build instances, storage, image sharing aur other resource charges current pricing mein check karo. Automation hone se vulnerability scanning aur patch ownership khatam nahi hoti.

Amazon Lightsail: simpler starting point

Lightsail virtual private server aur related bundled plan experience provide karta hai. Selected plans mein VM, SSD-backed storage, a data-transfer allowance, DNS management aur static IP jaise components ho sakte hain. Actual bundle aur excess transfer charges current plan/Region par depend karte hain.

Example: small portfolio site ko predictable setup chahiye. Growth, backup/restore, private networking, scaling aur operational limits assess karo. Lightsail simple hai, lekin IAM, OS patching, application security aur unused-resource cleanup still matter karte hain.

Amazon Linux 2023: server operating system

Amazon Linux 2023 AWS workloads ke liye Linux distribution hai, EC2 aur supported AWS tooling ke saath integrate hota hai; Graviton workloads ke liye compatibility/performance evaluate kar sakte ho. Support lifecycle aur newer Amazon Linux release schedule current documentation se check karo; kisi date ya fixed support duration ko permanently assume mat karo.

AL2023 security-by-default design, SELinux configuration, IMDSv2 defaults aur kernel live-patching availability jaise features provide kar sakta hai. Exact AMI/version settings verify karo; permissive SELinux ko enforcing ke equal mat samjho. Versioned repositories se update version control aur consistent deployments mein help milti hai. Lock karna useful hai, par patches indefinitely avoid karne ka reason nahi. Supported Regions, including specialised Regions, latest list se validate karo.

AWS App Runner: source ya container se web service

App Runner supported source code ya container image se web application/API deploy karne ka managed experience deta hai. Build/deploy, encrypted traffic routing/load balancing aur automatic scaling jaise supported capabilities configure ki ja sakti hain. Isse server fleet management kam hota hai, par application logic, IAM, networking, secret handling, observability aur cost ka design tumhe karna hai.

Example: container image registry se service deploy; request traffic badhe to configured scaling policy response de. Image provenance, runtime health, connection limits, ingress/egress aur current feature availability check karo.

AWS Batch: background job ki factory

Batch submitted jobs, queues aur compute environments ko coordinate karta hai. CPU/memory/resource requirements aur workload volume ke hisaab se supported capacity provision/schedule ho sakti hai; EC2 aur Spot jaise compute options scenario ke according evaluate karo. Team ko apna batch scheduler/cluster manually build karne ka kuch operational burden kam hota hai.

Example: 50,000 independent video thumbnails; har job image key leta hai aur resized output write karta hai. Failed jobs retry, duplicates idempotently handle aur input/output permissions scope karo. Job duration, available capacity, storage I/O aur budget monitor karo.

AWS Elastic Beanstalk: application environment deploy

Elastic Beanstalk web apps/services deploy aur scale karne ke environment workflows provide karta hai. Supported platform/runtime examples Java, .NET, PHP, Node.js, Python, Ruby, Go aur Docker hain; platform versions aur web servers jaise Apache, Nginx, Passenger ya IIS current support matrix se check karo.

Code upload ke baad supported environment setup capacity provisioning, load balancing, scaling aur health monitoring automate kar sakta hai. Underlying AWS resources ka inspection/control aur associated charges remain. Database migration, deployment rollback, secrets aur app dependencies separately plan karo.

AWS Fargate: container run karo, host fleet manage mat karo

Fargate supported ECS aur EKS container workloads ke liye managed compute option hai. ECS Fargate workflow mein application ko image mein package karo; task CPU/memory, networking, IAM task/execution roles aur logs define karo. Underlying server cluster provision/patch/scale ka task customer ke paas nahi hota, but container image, app, access, networking aur resource settings owner ko manage karni hain.

ECS on EC2 comparison: EC2 launch type mein servers choose, provision, patch aur scale karne hote hain. ECS task placement mein cluster CPU/memory aur declared requirements use karta hai. Host-level customisation aur placement needs relevant hon to EC2 option useful ho sakta hai, with more operations. Compliance ke liye particular host settings chahiye hon to specialist review zaroori hai; kisi model ko automatic compliant mat assume karo.

AWS Lambda: events par function

Lambda function supported events ya direct invocation par code run karti hai, server administration manage kiye bina. Billing request/execution aur applicable features ke pricing rules ke hisaab se hoti hai; log storage, provisioned concurrency, downstream services ya network jaise costs separately ho sakte hain. 'Code run nahi ho raha to total AWS bill zero' galat assumption hai.

Example: user photo upload kare → event → Lambda thumbnail banaye → output different prefix mein save kare. Input/output prefixes separate rakho taaki output se infinite recursive trigger na bane. Timeout, memory, concurrency, execution role, retries, idempotency aur DLQ/destination source-specific hain; DLQ automatically enabled nahi hoti.

Serverless Application Repository: ready component ka safe reuse

Repository mein shared code samples, components aur applications discover/deploy karne ke workflows ho sakte hain. Examples web/mobile backend, event/data processing, monitoring, logging aur IoT. Application SAM template describe kar sakta hai ki kaunse AWS resources create honge; public application ke saath source code link available ho sakta hai.

Deployment se pehle template aur code review karo: IAM permissions, data access, network configuration, lifecycle aur costs. Service fee aur deployed resources ke actual costs current pricing se verify karo. Teams ya organisations ke saath approved apps share/publish karne ke options evaluate karo.

Outposts, Wavelength, Local Zones aur hybrid

Outposts: supported AWS infrastructure/service operating model ko eligible customer data center, colocation ya on-premises site tak extend karta hai. Low-latency local processing aur data-location requirements relevant ho sakti hain. Earlier offerings described AWS-native aur VMware-oriented options; current product/partner availability and support verify karo. Depending on offering, servers/rack sizes aur connectivity/site requirements vary karte hain. Same API ka matlab saare cloud services aur features har Outpost par available nahi.

Wavelength: selected communications providers ke 5G network edge mein AWS infrastructure placement ka option. Supported mobile traffic ka network path edge application tak shorter ho sakta

hai, but end-to-end latency device, carrier, location aur app par depend karti hai. Region/zone availability, carrier integration aur cost verify karo.

Local Zones: selected geographic areas ke near supported AWS infrastructure provide karte hain. Low-latency application placement ke liye evaluate karo; service support, parent Region, availability, data residency aur price check karo.

VMware Cloud on AWS: VMware-based environments ko AWS infrastructure par extend/migrate karne ka hybrid use case, with technologies such as vSphere, vSAN, NSX and vCenter in supported offerings. AWS services ke saath integration, disaster recovery aur operational consistency relevant ho sakte hain. Commercial relationship, Region availability, partner support, compatibility aur migration terms time ke saath change ho sakte hain; current providers se directly confirm karo. Automatic portability ya zero-change migration guarantee nahi.

Customer enablement: expertise aur operational support

AWS Managed Services: infrastructure operations ke selected services — change processes, monitoring, patching, security operations aur backup workflows — provide kar sakta hai. Contract scope aur customer ki remaining permissions, data, application aur incident responsibilities clearly agree karo. Automation se risk reduce ho sakta hai, risk zero nahi hota.

AWS re:Post Private: eligible enterprise support contexts mein organisation-specific knowledge/community aur private technical discussions ka option. Supported plans, access controls, current lifecycle aur what data may be shared verify karo. Team community se knowledge share kar sakti hai, lekin confidential information exposure policies follow kare.

Chapter recap

• EC2 host/OS control deta hai; Lambda function execution aur Fargate container compute mein provider ki operational responsibility zyada hoti hai.

• Spot interruptible capacity hai; On-Demand, Reservations aur Savings Plans ke terms alag hain.

• Scaling ke saath application state, database capacity, health checks aur cost control ka plan bhi chahiye.

• Edge/hybrid options ki Region, network path aur feature support check karo.

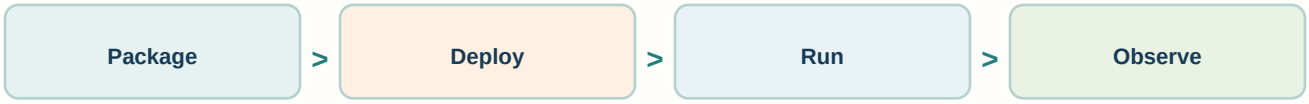
• Managed service ka matlab zero customer responsibility nahi hota.

Containers: ek baar package karo, controlled tareeke se chalao

TASVEER SE SAMJHO

Containers: app ka portable package

Ek box image ko aur server par chalne wale boxes containers ko yaad dilate hain. Orchestrator deployment aur health manage karne mein help karta hai; images aur security policies ko bhi maintain karna hota hai.



Kahani: Riya ka app mere laptop par chalta tha!

Developer ke laptop par app chalti hai, server par dependency version alag hone se fail ho jati hai. Container image mein application aur uski required dependencies/package configuration bundle karte hain. Image ek blueprint hai; running container us blueprint ka execution hai. Ye virtual machine se same concept nahi: isolation, kernel aur runtime ka model alag hai.

Journey: code → build image → registry mein store → orchestrator se deploy → containers run → logs/health monitor. Registry, orchestrator aur compute engine alag kaam karte hain. Har stage par image provenance, credentials, network aur security important hai.

Amazon ECR: image ki registry

Elastic Container Registry container images store, manage aur pull karne ka managed registry option hai. ECR aur ECS ko combine karke build se deployment pipeline bana sakte ho. Repository-level IAM permissions se decide karo kaun image push/pull kar sakta hai. Image scanning, immutable tags/digests aur lifecycle policies supply-chain hygiene mein help kar sakte hain; applicable features configure/verify karo.

Example: CI build shop-api image push kare, deployment approved image digest pull kare. latest mutable tag par blindly depend karne se reproducibility weak ho sakti hai. Storage, data transfer aur applicable scanning/request pricing current docs se check karo; 'managed registry' ka matlab free ya automatically secure nahi.

Amazon ECS: container orchestrator

ECS containerised applications ke tasks aur services manage karne ka AWS orchestration service hai. Task definition image, CPU/memory, ports, roles aur other runtime settings describe kar sakti hai. Service desired number of tasks maintain karne aur deployments coordinate karne mein help karti hai.

ECS APIs ke through tasks start/stop aur application state inspect ki ja sakti hai. IAM roles, security groups, load balancer, CloudWatch, CloudFormation aur CloudTrail jaise services ke saath supported integrations use kiye ja sakte hain. Fargate capacity option mein underlying host fleet customer manage nahi karta; EC2 launch type mein servers provision, patch aur scale karne hote hain. Ye boundary samajhna essential hai.

Example: shop API ke do tasks alag failure domains mein, ALB traffic healthy tasks ko route kare. Container port, target health, image pull permissions, secrets aur task role validate karo. Task failure par stopped reason aur logs dekho.

Amazon EKS: Kubernetes ke saath managed control plane

EKS Kubernetes applications deploy, manage aur scale karne ke liye managed service hai. Kubernetes primitives jaise pods, deployments, services aur namespaces use hote hain. Managed control plane architecture multiple AZs mein designed hoti hai; Kubernetes conformance se familiar tooling/plugins use karne mein help mil sakti hai.

Compatibility ka matlab every plugin, version, custom controller ya on-prem deployment unchanged run kare, aisa nahi. Worker nodes/capacity, workloads, Kubernetes RBAC, identity, networking, upgrades, observability aur data protection ka responsibility split selected EKS mode par depend karta hai. Current platform and add-on versions verify karo.

Debug: pod pending ho to events, requests/limits, capacity, node selectors/taints aur networking inspect karo. CrashLoop ho to container logs, probes aur application configuration dekho.

AWS App2Container: existing apps ko modernise karna

App2Container command-line tooling supported .NET aur Java applications ke containerisation workflows mein help karta hai. VM/on-prem/cloud servers par running apps aur dependencies discover/inventory karo, selected app ko container image mein package karo, network ports identify karo, aur supported ECS task/Kubernetes pod definitions generate karo.

CloudFormation aur supported CI/CD infrastructure generation deployment pipeline mein help kar sakti hai. Auto-generated artifacts production ready hone ki unconditional guarantee nahi: stateful storage, external dependencies, secrets, compatibility, security aur test coverage review karo. App data ko image ke andar permanent store karna risky ho sakta hai.

Red Hat OpenShift Service on AWS (ROSA)

ROSA AWS par managed OpenShift experience provide karta hai. Existing OpenShift APIs aur workflows use karne wali teams container deployment ke liye evaluate kar sakti hain. AWS compute, storage, networking, database aur other services ke saath integrate karne ke options application needs par depend karte hain.

Cluster lifecycle management mein Red Hat/AWS ka role selected deployment/service agreement se defined hota hai. Billing plans, support model, availability aur SLA current contractual terms se verify karo; old percentage ya price ko universal guarantee mat samjho. Workload code, access, image security aur data remain customer concerns.

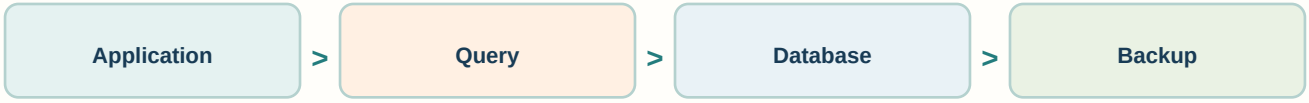
Container choice mini-drill

• Image store karni hai: ECR. Container schedule karne hain: ECS ya EKS. Underlying compute model: Fargate ya supported EC2-based capacity.

• Existing Kubernetes ecosystem aur Kubernetes APIs chahiye: EKS evaluate karo. Existing OpenShift operations important hain: ROSA compare karo.

• Java/.NET VM app ko package karna hai: App2Container analyse karo; dependencies aur state migration separately test karo.

Databases: data ke shape se database choose karo



Story: shop ko alag sawalon ke alag answers chahiye

Orders aur payments ke liye structured transactions, product catalogue ke liye documents, customer sessions ke liye fast key lookups, product recommendations ke liye relationships, aur sensor readings ke liye timestamps ho sakte hain. Ek hi database har case mein best nahi. Pehle access patterns, consistency, latency, recovery, data residency aur budget likho.

Relational: tables, relationships aur SQL; ERP, CRM, transactional ecommerce. RDS/Aurora examples; Redshift analytic warehouse hai, transactional database ka direct substitute nahi.

Key-value: known key se fast item access; DynamoDB example. In-memory: cache, sessions, leaderboards; ElastiCache aur durability-oriented MemoryDB compare karo. Document: JSON-like

nested records ke liye DocumentDB. Wide-column: Cassandra-compatible Keyspaces. Graph: connected relationships ke liye Neptune. Time series: timestamped events ke liye Timestream.

Amazon Aurora: relational database aur managed storage

Aurora managed relational engine family hai with MySQL-compatible aur PostgreSQL-compatible editions. Compatibility ka scope engine version aur features par depend karta hai; exact migration assessment zaroori hai. AWS management automation provisioning, setup, selected patching aur backup operations simplify karti hai. Schema design, queries, database users, IAM/network access aur data governance customer ki responsibilities hain.

Aurora architecture distributed fault-tolerant storage aur supported replication/recovery features use karti hai. Depending on configuration, read replicas, point-in-time recovery, continuous backup aur multi-AZ storage/recovery capabilities evaluate karo. Historical figures for storage size, replica count, speed aur cost change ho sakte hain: current limits aur measured workload benchmark verify karo.

Aurora I/O-Optimized: I/O-heavy workload ke liye alternate cluster pricing configuration. Example ecommerce ya payment database mein I/O cost meaningful ho to standard vs I/O-Optimized total estimate compare karo. Savings percentage guaranteed nahi; compute, storage aur request mix matter karta hai.

Zero-ETL integration example: supported Aurora-to-Redshift integration transactional data se analytics ke use cases simplify kar sakti hai. Compatibility, setup, lag, access policies aur current release status verify karo; 'zero ETL' ka matlab data modelling ya governance zero nahi.

Amazon DynamoDB: access pattern se key design

DynamoDB managed key-value/document database hai, low-latency request patterns ke liye. Table ko partition key aur optional sort key ke around design karo. Backups, restore, encryption, global-table options aur other features configuration/version ke hisaab se available hote hain. Mobile, web, gaming, IoT aur ad-tech examples mein predictable key access useful ho sakta hai.

Example: PK=CUSTOMER#42, SK=ORDER#2026-01-15#123 se customer order history query model ki ja sakti hai. Product ID se orders chahiye to alternate index/data model may be needed. Hot partitions, table/index costs, conditional writes, TTL semantics aur multi-Region conflict behavior assess karo. Claimed request rates ya single-digit latency har workload par guarantee nahi.

Amazon ElastiCache: fast memory cache

ElastiCache managed in-memory cache options provide karta hai. Example: product page ka frequently read information cache karo; miss par authoritative database read karo. Redis-compatible/Redis OSS aur Memcached protocol-based options ka feature set alag hai. Redis-like data structures, caching ya other supported workloads; Memcached widely used object cache pattern — choose with current compatibility and documentation.

Serverless cache option infrastructure capacity planning simplify kar sakta hai, with supported redundancy/availability model and usage-based charges. Current SLA, supported engine, AZ placement, availability and pricing verify karo; old fixed shard/size limits assume mat karo. Cache is not automatically source of truth: stale data, eviction, invalidation, TTL, failover aur cache stampede design karo.

Amazon Keyspaces: Cassandra-compatible wide-column

Keyspaces managed Cassandra-compatible database option hai. Existing Cassandra application code, APIs aur tools ke some workflows reuse ho sakte hain; supported operations/version compatibility test karo. Serverless capacity model, encryption-at-rest and supported point-in-time recovery features evaluate karo; selected settings and pricing confirm karo.

Example: industrial fleet device status aur timestamp-keyed maintenance records. Partition design aur query patterns explicitly model karo. 'Virtually unlimited' marketing ko quota, workload distribution, throughput aur cost planning se replace karo.

Amazon MemoryDB: in-memory plus durability

MemoryDB Redis-compatible in-memory database option hai, performance-sensitive applications ke liye. Familiar data structures/APIs ka use-case ho sakta hai; exact compatibility/version verify karo. Distributed transactional log aur multi-AZ architecture durability/failover support kar sakte hain, subject to configuration. Isse cache-only pattern se alag primary-database requirements evaluate ki ja sakti hain.

Example: microservice ko fast data access aur persistence chahiye. Backup/restore, multi-AZ availability, latency targets aur data loss tolerance test karo. Historical microsecond/millisecond figures ko performance guarantee mat samjho.

Amazon Neptune: relationships aur graph

Neptune graph database highly connected data ke relationships aur traversals handle karne ke liye designed hai. Nodes = entities, edges = relationships. Property Graph aur RDF models, Gremlin aur SPARQL jaise query languages ke supported versions evaluate karo. Fraud networks, recommendations, knowledge graphs, drug discovery aur network-security relationships possible use cases hain.

Example: customer → device → payment method relationships se suspicious shared-device cluster explore karna. Graph model aur access permissions early design karo. Supported read replicas, backups, point-in-time recovery, multi-AZ replication aur at-rest encryption options ka review karo.

Neptune Analytics: graph analysis engine supported S3 ya Neptune graph data ke liye algorithms, in-memory computing aur vector search workflows offer kar sakta hai. Data scale, import path, algorithm availability, pricing aur actual latency verify karo.

Amazon RDS: managed relational engines

RDS relational databases setup/operations simplify karta hai; supported provisioning, backups, patching aur scaling capabilities engine/deployment ke hisaab se vary karti hain. Examples MySQL, MariaDB, PostgreSQL, Oracle Database, SQL Server aur supported Db2; Aurora related managed family hai. RDS on Outposts ek deployment option hai, database engine nahi. Engine version, license aur Region availability check karo.

Application owner schema, queries, database users, credentials, network access aur business recovery plan maintain karta hai. Supported Multi-AZ options high availability mein help karti hain; standby backup ka replacement nahi. DMS selected migration/replication scenarios support karta hai, but compatibility, validation, cutover aur rollback plan zaroori hain.

RDS for Db2

Managed Db2 deployments ke provisioning, backups, patching aur monitoring support kar sakte hain. Supported Multi-AZ deployment, cross-Region backup aur security controls business continuity planning mein relevant ho sakte hain. Db2 licenses, engine versions, IBM/AWS integrations aur cost verify karo. Isolated restore drill se recovery success measure karo.

RDS on VMware: historical hybrid pattern

On-premises VMware environments mein RDS-like management aur optional AWS replication/backup use cases historically describe kiye gaye the. Hybrid operations, read replicas, disaster recovery aur long-term backup placement ke ideas useful hain, but product lifecycle, commercial support aur migration options current provider documentation se verify karo. Network interruption aur data-residency constraints plan karo.

Amazon Timestream: waqt ke saath badalta data

Time-series data timestamp se indexed measurements/events hain: IoT sensor readings, operations logs, industrial telemetry. Data often time-ordered/append-heavy hota hai aur queries time intervals filter karte hain, although actual workload differ kar sakta hai. Timestream time-series-oriented storage/query workflows aur supported serverless operation provide karta hai.

Example: factory temperature sensor har minute value bhejta hai; last hour average aur abnormal high readings query karo. Supported retention, tiering, rollups aur compression features data lifecycle manage karne mein help kar sakte hain. Ingest rate, late data, query range, retention settings aur costs measure karo; old price/performance comparisons guarantee nahi.

Amazon DocumentDB: MongoDB-compatible document workloads

DocumentDB JSON-like document workloads aur MongoDB-compatible APIs/tools ka managed option hai. Client compatibility ka matlab every MongoDB server feature, driver/version ya behavior identical nahi. Specific application queries, transactions, indexing, supported APIs aur engine versions migration se pehle test karo.

Example: product catalogue documents jisme optional attributes category ke hisaab se vary karte hain. Schema flexibility data validation aur access control ki need eliminate nahi karti. Backup, restore, network and instance cost separately assess karo.

Lightsail managed databases: simplified bundled option

Lightsail managed databases app compute se separate managed database option hain. Supported MySQL/PostgreSQL choices aur standard/high-availability configurations plan/Region par depend karte hain. Bundled monthly pricing mein selected compute, SSD storage aur transfer allowance ho sakti hai; excess charges aur configuration specifics verify karo.

Console, CLI, Lightsail API ya SDK se supported management actions kiye ja sakte hain. Credentials, network exposure, backup retention, migration, maintenance aur restore test still plan karna hai.

Database decision practice

• Financial order with multi-table relational queries? RDS/Aurora assess karo.

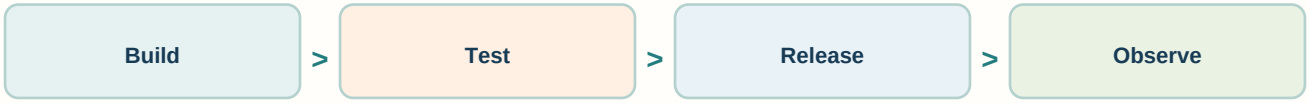
• Well-known key par large request volume? DynamoDB model karo; hot keys monitor karo.

• Fast frequently accessed product data? ElastiCache consider karo; authoritative data aur invalidation clear karo.

• Entity relationships traverse karni hain? Neptune; time-window sensor readings? Timestream.

• Managed database choose karne ke baad backup restore aur permissions kiski responsibility? Customer ko configure, validate aur operate karna hoga.

Developer tools: code se release tak



Ek feature ka safar

Riya ki team ko checkout mein coupon feature launch karna hai. Developer code likhta hai, team review karti hai, dependencies fetch hoti hain, build aur tests chalte hain, approved artifact deploy hota hai aur production performance monitor hoti hai. In tasks ko manual copy-paste se karne ke bajay controlled pipeline banao: design → code → dependencies → build/test → release → observe.

Infrastructure Composer: diagram se infrastructure code

Infrastructure Composer browser-based visual canvas mein supported serverless resources compose/configure karne aur unke connections dekhne mein help karta hai. Resources group karke larger component ke roop mein organise kiye ja sakte hain. Supported output CloudFormation aur AWS SAM configuration/artifacts ho sakte hain.

Example: API → Lambda → data store canvas par arrange karo; generated template ko version control mein rakho. Default configuration blindly production mein mat daalo: roles, network exposure, event permissions, logging aur cost review karo. Visual layout architecture ki security proof nahi hai.

Cloud9: browser IDE ka historical example

Cloud9 browser-based IDE tha jisme code editor, terminal, debugger aur popular language tools ke workflows the. Internet-connected machine se supported environment access, team collaboration/pair programming aur serverless development/testing jaisi capabilities described thi. New-customer availability aur current lifecycle/decommissioning instructions current AWS documentation se check karo; secrets aur team access secure rakho.

CloudShell: browser ke andar shell

CloudShell browser-based shell hai jahan common development/operations tools preinstalled ho sakte hain. Console sign-in credentials ke context se CLI commands, scripts aur supported SDK/API experiments run kar sakte ho without local setup. Service charge aur underlying resource/API charges current pricing se verify karo.

Safety: sts get-caller-identity aur selected Region check karo, tabhi destructive command chalana; sirf authorised account mein. Shell output mein tokens ya customer data leak mat karo. CloudShell IDE ke complete substitute ya persistent backup guarantee ke roop mein assume mat karo.

CodeArtifact: dependencies ka managed store

CodeArtifact software packages publish, store aur share karne ka repository service hai. Supported upstream repositories se dependencies fetch karne ka configuration ho sakta hai. Maven, Gradle, npm, yarn, twine/pip aur NuGet jaise tools ke supported formats/integration current docs se verify karo.

Example: build server approved private library aur public dependency repository se packages resolve karta hai. Upstream supply-chain compromise, dependency pinning, package provenance, repository access aur retention review karo. Latest version automatically fetch karna har baar secure choice nahi hai.

CodeBuild: code se tested artifact

CodeBuild source compile, automated tests run aur deployable packages create karta hai. Managed build environments ke through build servers khud provision karne ka burden kam hota hai. Supported prepackaged environments ya custom image/toolchain choose kar sakte ho; concurrency, queue limits aur build duration/price account setup par depend karte hain.

Example: pull request → compile → unit tests → dependency/security checks → artifact. Test fail ho to production deployment rok do. Build service role least privilege ho aur environment variables/logs mein passwords na chhapen.

CodeCatalyst: team workspace aur CI/CD

CodeCatalyst planning, code collaboration aur build/test/deploy workflows ko integrated environment mein laane ka product example hai. Supported project configuration ke through AWS accounts/resources connect kiye ja sakte hain. IAM integration, lifecycle, billing aur current product availability verify karo; platform team ki review/security ownership replace nahi karta.

CodeCommit: private Git repository

CodeCommit managed private Git source-control option hai. Existing Git workflows se supported source ya other repository content store kar sakte ho; permissions, branch protection aur backup policies design karo. New-customer/repository eligibility aur lifecycle current documentation se check karo. Repository mein credentials mat commit karo.

CodeDeploy: rollout automate karo

CodeDeploy supported EC2/on-premises aur other integration targets par application deployment automate karne mein help karta hai. One server ya larger fleet par controlled updates, deployment configuration aur health monitoring use ki ja sakti hai. Actual no-downtime result application, deployment strategy, database changes aur target setup par depend karta hai.

Example: small set of instances par new version → health test → broader rollout → failure par configured rollback. Database schema backward compatibility aur rollback behavior plan karo.

CodePipeline: release stages coordinate karo

CodePipeline release pipeline mein source, build, test aur deploy actions ko defined model ke according coordinate karta hai. Third-party GitHub aur supported custom integrations ka use-case possible hai; connector permissions aur support current docs se verify karo. Code change se pipeline trigger ho sakti hai; production approval gates team requirements se add karo.

Comparison: CodeBuild ka kaam build/test job run karna; CodeDeploy deployment perform karna; CodePipeline stages coordinate karna. Teeno ke roles distinct hain. Price, execution count aur connected resource cost assess karo.

Amazon Corretto: Java runtime distribution

Corretto OpenJDK distribution hai, supported platforms par Java applications develop/run karne ke liye. Selected versions ke security updates aur support lifecycle published policy ke anusaar check karo. Platform examples Amazon Linux, Windows aur macOS ho sakte hain; Java version, dependencies aur production compatibility test karo. 'No-cost distribution' ka matlab application hosting ka AWS cost zero nahi.

Fault Injection Service: failure ko safely rehearse karo

Chaos engineering mein controlled disruption se system response observe kiya jata hai. Example: nonproduction workers par CPU stress ya instance failure simulate karo; dekho alert aaya, workload recover hua aur customer-facing error budget impact kya tha. Fault Injection Service supported actions/templates aur stop conditions se experiments organise karne mein help karta hai.

Production experiment se pehle approval, blast radius, hypothesis, monitoring, stop condition aur rollback/abort owner define karo. Automatic stop ka configuration safety net hai, perfect protection nahi. Experiment ka objective learning aur remediation hai, random breakage nahi.

Amazon Q Developer: AI-assisted development

Q Developer coding, test, troubleshooting, security suggestions, application upgrade aur AWS resource assistance jaise tasks mein help kar sakta hai. Supported multi-step code-change and language-upgrade workflows product capability/version par depend karte hain. Generated code ya remediation ko human review, tests, least privilege aur security scanning se validate karo; AI output authoritative guarantee nahi.

AWS X-Ray: request ka route aur delay

Distributed application mein ek user request API, app aur database se guzar sakti hai. X-Ray tracing se supported segments/spans aur service map dekhe ja sakte hain; slow component aur error path investigate karne mein help milti hai. Sampling, instrumentation gaps aur privacy settings ke karan har request/field visible hona guaranteed nahi. Trace aur logs mein sensitive secrets record mat karo.

Scenario: checkout 4 seconds le raha hai. Trace dekhkar app step fast but downstream call slow dikhe, to dependency metrics/logs inspect karo. Correlation diagnosis assist karta hai; prove karne ke liye additional evidence zaroori hai.

End-user computing: desktop ya app remotely

TASVEER SE SAMJHO

Remote desktop: screen kahin aur, user kahin aur

User apne device se remote workspace access karta hai. Identity, device permissions aur connection security protect karna zaroori hai.

User > Identity > Desktop / app > Data

WorkSpaces Applications: ek app stream karo

Application centrally manage hoti hai aur supported user devices tak streaming se deliver ho sakti hai. Example: education institute students ko engineering/design application accessible banaye, ya software vendor demo/trial de bina full local install ke. GPU-heavy app ke liye appropriate supported capacity aur network latency assess karo. Capacity scaling, licensing, session policy, data location aur endpoint controls plan karna zaroori hai.

WorkSpaces: cloud desktop

WorkSpaces supported Windows/Linux desktop environments provision karne ka option hai. Organisation ko hardware inventory aur VDI infrastructure ke kuch operations simplify ho sakte hain; users supported devices se remotely connect kar sakte hain. Monthly/hourly billing modes aur desktop options supported configurations ke hisaab se vary karte hain. Identity, OS/application patch responsibilities, licensing, latency, backup aur actual cost verify karo.

WorkSpaces Core: third-party VDI management

WorkSpaces Core supported third-party VDI management solution ke saath AWS-based virtual desktop infrastructure ka option hai. Existing management software, migration approach aur workforce location relevant hain. Supported integration, billing model aur uptime SLA ke latest contractual terms check karo; uptime number ko universal promise na samjho.

WorkSpaces Thin Client: access device

Thin Client ek compact endpoint device ka option hai jo supported AWS virtual desktop experiences se connect hota hai. Compatible configuration mein monitors aur keyboard/mouse/headset/webcam jaise USB peripherals use ho sakte hain. Device management, allowed application policy aur local-data restrictions ki current capabilities verify karo. Endpoint security ke saath user identity aur remote-session permissions bhi protect karo.

WorkSpaces Web: browser se web apps

Supported browser-based access pattern internal websites aur SaaS applications ke liye managed workspace provide kar sakta hai. Policies user interactions aur access controls manage karne mein help karti hain; provider supported browser image/capacity management ka kuch burden sambhal sakta hai. Identity, web-app permissions, data transfer policy aur current product naming/features check karo.

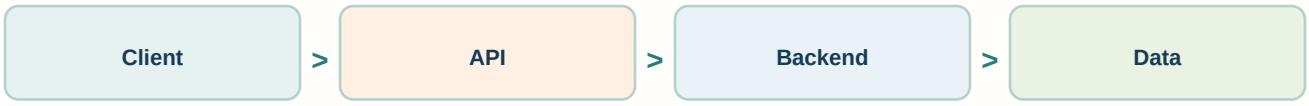
Choose: full desktop chahiye to WorkSpaces; selected app stream karni ho to WorkSpaces Applications; third-party VDI management integration ho to Core; browser-based internal web app use case ke liye Web evaluate karo.

Frontend, mobile aur game technology

TASVEER SE SAMJHO

Alag screens, backend ke saath baatcheet

Phone, browser aur game client backend se requests bhej sakte hain. Authentication, application logic aur data access ko separately secure karo; sab apps ka architecture ek jaisa nahi hota.



Amplify: front end aur backend connect karo

Amplify web/mobile application build, deploy aur supported backend integration workflows simplify karta hai. iOS, Android, web aur React Native ke supported libraries/components available ho sakte hain. Authentication, storage aur selected data-sync capabilities ko setup karne ke patterns offer karta hai; framework/version and feature support verify karo.

Example: mobile shopping app mein user sign-in aur product images integrate karo; release pipeline se front-end deploy karo. Generated infrastructure, user authorisation, offline conflict behavior, secrets aur cost review karo; framework app ki business security automatically design nahi karta.

AppSync: application data API

AppSync supported GraphQL-based APIs aur backend integrations provide karta hai. GraphQL mein client requested data shape describe kar sakta hai; resolvers data sources se information laate hain. Supported data synchronisation/offline patterns and other API capabilities configuration/client framework par depend karte hain.

Example: product detail screen ko title aur price chahiye; API resolver authorised fields fetch kare. Authentication ke baad bhi per-user authorisation, resolver permissions, data exposure aur caching validate karo.

Device Farm: real devices par test

Android, iOS aur web apps ko supported real devices/browsers par test ya interactive investigation kar sakte ho. Video, screenshots, logs aur performance signals se device-specific bugs reproduce karne mein help milti hai. Representative device matrix, OS versions, accessibility, synthetic test accounts aur test-data cleanup plan karo.

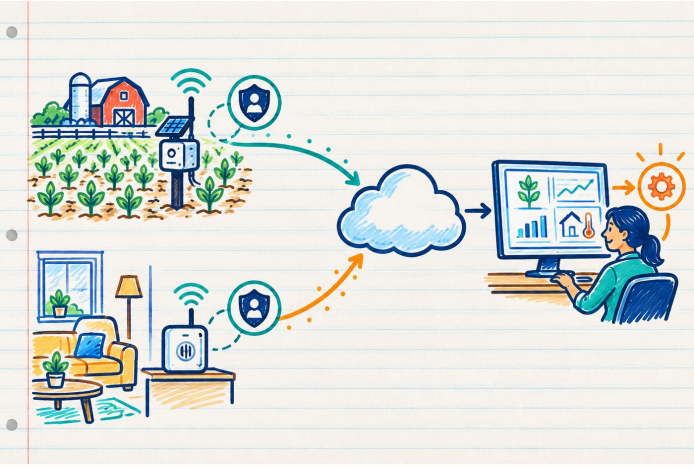
Amazon Location Service: maps aur location

Supported location functionality mein maps, geofencing, tracking aur location-aware application workflows aa sakte hain; providers aur current features verify karo. Example: delivery fleet ka location track karke depot geofence event detect karo. Location data sensitive hai: consent, access, retention, request pricing aur data sharing terms explicitly define karo. Geofence event safety-critical decision ka sole control na banao.

GameLift Servers: multiplayer sessions

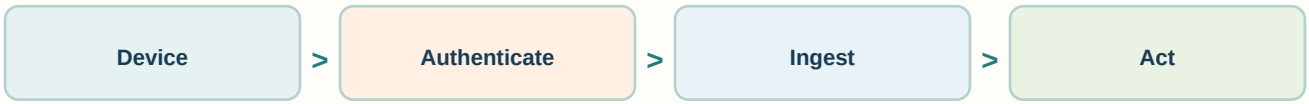
GameLift Servers dedicated session-based multiplayer game-server deployment aur scaling workflows ke liye evaluate hota hai. Server fleet capacity, available game sessions, player placement, geographic latency, bandwidth aur game-specific security important hain. Supported DDoS protection and matchmaking features ki exact scope check karo. Pay-as-you-use compute/bandwidth aur contracts ki current pricing/terms compare karo; low latency har player ke liye guaranteed nahi.

IoT: device se cloud tak



TASVEER SE SAMJHO
IoT: device se action tak

Khet ya ghar ka sensor data bhejta hai; cloud us data ko receive/process karta hai; operator dashboard dekhkar action leta hai. Device identity aur network security important hain.



Kahani: factory ka smart sensor

Temperature sensor ek resource-constrained device hai; woh readings bhejta hai, kabhi network offline hota hai aur kabhi battery low. Complete design mein device identity, secure communication, telemetry, edge logic, fleet updates, data analysis aur device retirement aate hain.

Flow: sensor identity → IoT Core connection → selected events → edge/local decision ya cloud storage → analysis → alert. Har stage par security, data freshness, offline behavior aur duplicate message handling plan karo.

IoT Analytics: messy sensor data ko useful banana

IoT telemetry noisy ho sakti hai: missing data, corrupted packets, gaps ya inaccurate readings. Irrigation example mein moisture sensor ko rainfall context ke saath combine karke watering decision improve kiya ja sakta hai; real-world safety aur agronomy rules separately validate karo.

IoT Analytics historically supported collection, filtering, transformation, enrichment with device type/location, time-series storage aur scheduled/ad-hoc SQL analysis. Containerised custom analytics, notebook/tool-based workflows aur ML use cases bhi described hain. Current service lifecycle aur alternatives verify karo; source data quality aur appropriate sampling app owner ko design karni hai.

IoT Button: event trigger ka example

Programmable Wi-Fi button ka historical example button click ko Lambda, DynamoDB, SNS ya other workflows se connect karta tha. Use cases feedback, check-in, alert, appliance trigger ya ordering jaise ho sakte hain. Third-party integrations, hardware availability aur lifecycle current docs se verify karo. Remote unlock ya safety-critical action mein strong authentication, physical safety aur explicit approval controls zaroori hain.

IoT Core: devices aur cloud ka bridge

IoT Core connected devices se supported secure communication, message routing aur device/application interaction facilitate karta hai. Devices offline ho sakte hain; state representation aur message behavior selected features/configuration par depend karte hain. Supported integrations Lambda, Kinesis, S3, DynamoDB, SageMaker, CloudWatch aur CloudTrail jaise services ke saath workflows enable kar sakte hain.

Per-device identity/certificates aur narrowly scoped policy use karo. High device count ke marketing numbers ko blanket quota ya guaranteed reliability na samjho; limits, message size, retries aur cost verify karo.

IoT Device Defender: security audit aur unusual behavior

Device Defender selected IoT configurations ko security audit checks se compare karta hai. Shared identity certificates, revoked certificate connection attempts, overly broad policies ya unusual metric behavior jaise signals investigate karne layak ho sakte hain. Device/Core metrics ko expected profile ke against monitor kar sakte ho; spikes potentially suspicious hain, automatically proof of attack nahi.

Alerts supported IoT console, CloudWatch ya SNS workflows se route ho sakte hain; Device Management se compatible remediation actions, jaise authorised security update, orchestrate ho sakte hain. Greengrass/FreeRTOS integration aur available metrics current docs se check karo. Alert owner aur incident response define karo.

IoT Device Management: fleet lifecycle

Large fleet mein registration, grouping, health, troubleshooting, permissions aur software/firmware update essential hain. Devices individually ya bulk onboard karne, group/state queries aur supported over-the-air jobs use kiye ja sakte hain. Microcontroller se vehicle tak devices varied hain; exact compatibility/protocols check karo.

Update drill: signed firmware → small pilot group → health verification → staged fleet rollout → failure par rollback/hold. Device loss/compromise par certificate revoke aur fleet record update karo.

IoT Events: telemetry pattern se action

Historical IoT Events workflow multiple telemetry sources par state/if-then logic evaluate karke alerts ya actions trigger kar sakta tha. Example: freezer temperature threshold ke upar rahe AND door closed ho → operator alert. Equipment belt speed, humidity aur motion sensors other examples hain. Device delays, false positives aur rule hysteresis plan karo. Product lifecycle aur current alternatives deployment se pehle verify karo.

IoT ExpressLink: partner connectivity modules

Partner hardware modules supported networking/security software aur provisioned credentials ke saath device-cloud connectivity simplify kar sakte hain. MQTT publish/subscribe, Device Shadow updates, supported OTA updates aur Device Defender metrics ke workflows described hain. Hardware vendor/module, certificate ownership, cryptography, supply chain aur firmware support period verify karo. Offloading networking ka matlab system security ka ownership outsource ho gaya, ye nahi.

IoT FleetWise: vehicle signals

FleetWise vehicle data collect/standardise aur selected rules ke basis par cloud transfer workflows ke liye designed tha. Example: temperature, speed ya model ke context mein telemetry collect karo, taaki unnecessary data transfer kam ho aur fleet health analyse ho. Maintenance insights, infotainment ya ML/ADAS research use cases possible hain. Driver privacy, consent, network, data quality, lifecycle aur applicable safety regulations evaluate karo; ML output ko driving safety ka sole guarantee mat samjho.

IoT Greengrass: edge par local action

Greengrass supported device components aur selected Lambda/ML inference/local messaging workflows edge par run karne mein help karta hai. Cloud management aur local operation combine kar sakte ho; internet unavailable ho to sirf appropriately designed local logic kaam karegi. Data filtering se cloud ko relevant records bhejne, supported connectors/protocol adapters aur Secrets Manager integration evaluate kar sakte ho.

Example: factory temperature high ho to local alarm; link return hone par buffered events upload. Local identity, offline storage encryption, update rollout aur physical safety interlocks design karo.

IoT SiteWise: industrial assets aur metrics

SiteWise industrial equipment telemetry collect, organise, model aur monitor karne ke workflows support karta hai. Facility gateway on-premises data sources se supported protocols ke through information collect karke cloud bhej sakta hai; MQTT/API pathways use case par depend karte hain. Asset models, processes aur site hierarchy se performance metrics aur industrial dashboards build kiye ja sakte hain.

Example: multi-site production line uptime compare karo, lekin sensor timestamps/units align karo. Remote monitoring ke insights maintenance team ko help karte hain; human/independent safety system ki jagah nahi lete.

IoT TwinMaker: real system ki digital representation

TwinMaker buildings, factories, equipment ya production lines ke digital twins create karne mein help kar sakta hai. Existing 3D models aur multiple data sources ko combine karke virtual view banaya ja sakta hai. Model, live data and permissions aligned rakho; visual twin stale ya incomplete data ko automatically correct nahi karta.

Partner Device Catalog: hardware discovery

Supported partner catalog mein development kits, embedded devices, gateways, edge servers, sensors aur cameras discover karne ka workflow ho sakta hai. Devices ki certification, current catalogue/product availability, firmware support aur purchase terms independently verify karo. Listed device ka hona har workload/security requirement meet karne ka proof nahi.

FreeRTOS: chhote devices ka operating system

Microcontroller (MCU) ek chip ho sakti hai jisme limited processor aur memory ho; sensors, appliances, fitness trackers, industrial controllers aur vehicles mein examples milte hain. FreeRTOS ek open-source real-time operating-system ecosystem hai jo small embedded devices ke scheduling aur supported connectivity/security libraries mein help karta hai.

Smart electricity meter cloud ko usage bhej sakta hai; building access device local controller se baat kar sakta hai. Exact hardware support, network stack, secure provisioning, updates, memory footprint aur real-time safety requirements verify karo. RTOS hone ka matlab device automatically secure ya always connected nahi.

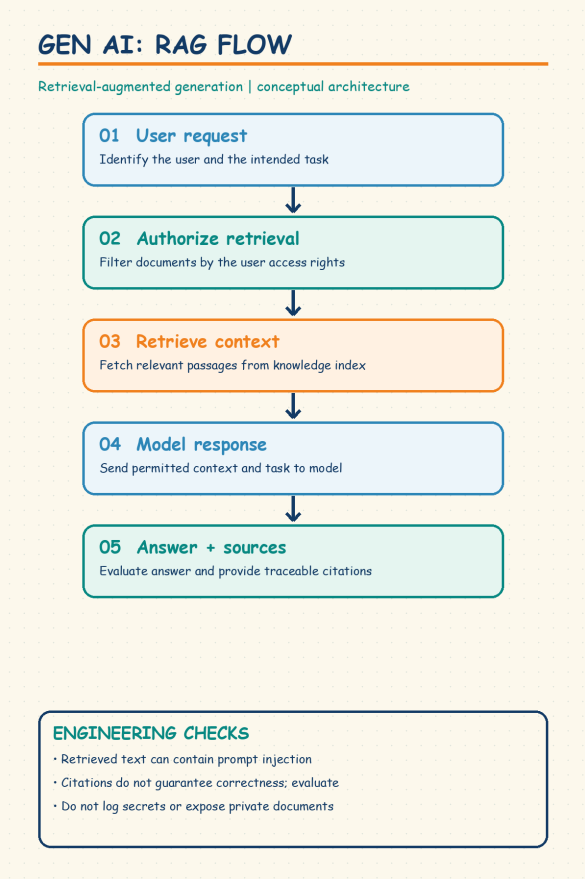
IoT self-check

• Secure connectivity: IoT Core; fleet onboarding/updates: Device Management; security audit signals: Device Defender.

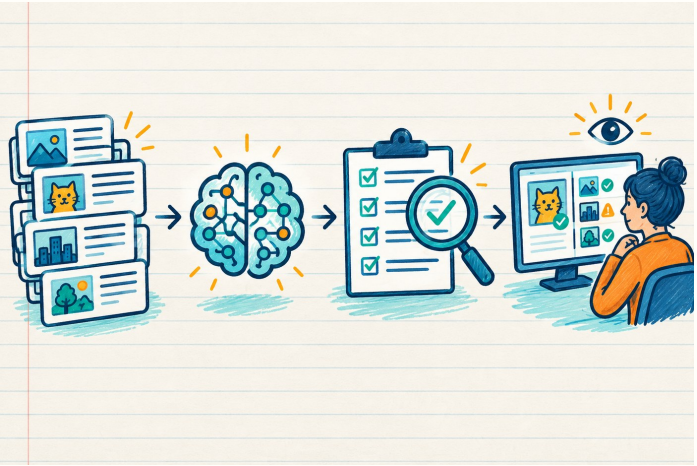
• Local action when network unavailable: Greengrass-type edge design; offline data buffering aur fail-safe policy add karo.

• Industrial asset dashboards: SiteWise; 3D/operational digital representation: TwinMaker.

• Device certificate compromised? Revoke/isolate, investigate, rotate credentials aur fleet process update karo.



Machine learning aur AI: task samjho, phir tool chuno



TASVEER SE SAMJHO

Machine learning: data, model, test, monitor

Pehle examples aur data taiyar karo, model ka result evaluate karo, phir application mein use karke monitor karo. Model ka output hamesha sahi ho, aisa assume mat karo.

Data > Model > Evaluate > Deploy

Story: shop ka smart assistant

Riya ke paas product images, customer reviews, invoices aur support calls hain. Wo poochti hai: 'Kya ek AI service sab solve karegi?' Zaroori nahi. Image analysis, OCR, speech transcription, recommendation aur generative response alag tasks hain. Pehle desired output, acceptable error, privacy aur human review define karo.

ML lifecycle: data collect → label/clean → train/choose model → evaluate → deploy → monitor → improve. Generative AI: foundation model prompt/context ke basis par text/image/etc generate karta hai; plausible output factual correctness guarantee nahi. Responsible use: access control, consent, representative tests, bias checks, monitoring aur sensitive decisions par human oversight.

Augmented AI (A2I): human review loop

A2I supported ML outputs par human-review workflows build karne mein help karta hai. Example: invoice extraction confidence low ho to reviewer amount aur vendor verify kare; high confidence outputs bhi sample audit se check karo. Reviewer workforce, escalation, privacy, quality control aur audit log define karo. Human review workflow supported integrations ke saath AWS ya external model-output use cases mein relevant ho sakta hai.

Bedrock: foundation models ke APIs

Bedrock supported foundation models ko API ke through use aur generative AI applications banane ke managed capabilities deta hai. Model catalog, access eligibility, Region aur features change hote hain; supported providers aur Amazon Nova jaise models current docs se check karo. Model choice mein quality, latency, context size, cost, safety aur data handling compare karo.

Prompt, system instructions, retrieval, optional customisation aur application logic milkar solution bante hain. Example: authorised product documentation retrieve karo → model se answer generate karo → source citations aur validation dikhayo. Retrieval permissions enforce karo; malicious document prompt injection attempt kar sakta hai. 'Private customisation' ka meaning actual service terms/settings se verify karo; secrets casually prompts mein mat bhejo.

CodeGuru: code review aur profiling

CodeGuru Reviewer supported automated reasoning/ML insights se possible bugs, security issues aur code-quality recommendations identify karne ka workflow provide karta tha. CodeGuru Profiler runtime performance mein costly code paths dhoondhne mein help karta hai. Recommendations test aur developer review ke baad apply karo. Product availability, supported languages, agent overhead aur pricing verify karo.

Comprehend: text se structured signals

Comprehend NLP capabilities supported text mein language, entities/people/places/brands/events, key phrases, sentiment, tokenisation/parts of speech aur topic grouping jaise tasks kar sakti hain. Custom entity recognition/classification workflows bhi relevant ho sakte hain; current features check karo.

Example: 1,000 product reviews mein common complaints identify karo. Sarcasm, mixed language aur context se sentiment wrong ho sakta hai. Inputs privacy classify karo, model outputs ko sample review aur business validation se check karo.

DevOps Guru: operational anomalies

Operational signals jaise latency, error rates aur resource constraints mein abnormal patterns surface karne ka ML-based service example. Supported configuration accounts, CloudFormation stacks ya tags ke resources se data collect karke related anomalies, approximate context aur possible recommendations show kar sakti thi. Alert triage, data scope aur current product lifecycle verify karo; generated root-cause suggestion hypothesis hai, proof nahi.

Forecast: historical series se future estimate

Demand forecast mein past sales ke saath price, discounts, season, store location aur product features relevant ho sakte hain. Historical Forecast service time-series aur associated variables se prediction workflows support karta tha. Example: raincoat demand ke liye season aur location include karo. Historic accuracy/performance percentages ko guarantee na samjho; holdout evaluation, uncertainty aur changing conditions check karo. New usage ke liye service lifecycle/alternatives verify karo.

Fraud Detector: suspicious activity ka risk

Fraud Detector historically customer data se fraud-detection models train/deploy karne ka service option tha. Use case: transaction ko risk score dena aur high-risk case human review mein bhejna. Fraud labels delayed/biased ho sakte hain; false positives legitimate customer ko block karte hain. Threshold, appeal, privacy, fairness aur current service availability validate karo; fraud prediction final legal judgement nahi.

Comprehend Medical: clinical text extraction

Clinical notes se supported medical entities — condition, medication, dosage, strength, frequency ya procedure — aur relationships extract karne ka NLP use case. Medical terminology mapping workflows ICD-10-CM, RxNorm ya SNOMED CT jaise ontologies se relate ho sakte hain; supported feature/region verify karo.

Medical data highly sensitive hai. HIPAA eligibility ka matlab automatic compliance nahi: applicable agreement, configuration aur legal obligations assess karo. Clinician/human verification necessary hai; extraction error care ya claims ko affect kar sakti hai.

Kendra: enterprise search

Kendra supported enterprise data sources mein search aur relevance features provide karta hai. Documents multiple repositories mein hon to search index/connector se discoverability improve ho sakti hai. Connector supported ACLs, index freshness, query relevance aur document permissions test karo. Employee ko wahi documents return hon jo woh dekhne authorised hai.

Lex: text aur voice conversations

Lex conversational interfaces/chatbots build karne ka tool hai. Speech input mein ASR speech ko text mein convert karta hai; NLU text se intent/slots identify karne mein help karta hai. Dialogue flow design, test aur supported web/mobile/chat/voice integrations mein publish kiya ja sakta hai.

Example: 'mera order kahan hai?' → order-status intent → order ID collect → authenticated backend query → response. Unknown intent fallback aur human handoff add karo. Request pricing, language support aur free-tier eligibility current docs se verify karo.

Lookout aur Monitron: anomalies aur industrial inspection

Lookout for Equipment: historical ML workflow pressure, compressor flow ya fan RPM jaise sensor data se equipment-specific abnormal pattern identify karta tha. Sensor quality, thresholds, lifecycle aur maintenance response plan test karo.

Lookout for Metrics: sales revenue/customer acquisition metrics mein abnormal changes, related-anomaly grouping aur severity/alert use case. Its documented discontinuation was October 10, 2025; new designs ke liye current transition guidance/alternatives verify karo. S3, Redshift, RDS ya SaaS connectors ka historical feature example availability ka proof nahi.

Lookout for Vision: manufacturing images mein missing part, damaged surface, wafer defect ya other visual anomaly detect karne ka historical use case. Camera angle, training set, false negatives aur current lifecycle check karo; automated inspection manual quality assurance ko universally replace nahi karti.

Monitron: vibration/temperature sensors, gateway, cloud analysis aur mobile app ke through predictive-maintenance workflow ka example. Sensor placement, alert response, device support aur current lifecycle verify karo. Anomaly alert failure ki certainty nahi.

PartyRock: hands-on generative AI experimentation

PartyRock code-free app builder ke through prompts aur foundation model responses experiment karne ka experience hai. Use case: fictional product description generator bana kar prompt variations compare karna. Service terms, model choice, current availability aur data sensitivity policies check karo; production data ko demo mein mat daalo.

Personalize: recommendations

Personalize user interactions jaise page views, signups, purchases aur item catalog jaise products, articles, video ya music use karke supported recommendation workflows provide karta hai. Optional user attributes, segmentation aur retail/media recommenders feature/terms ke hisaab se evaluate karo.

Example: 'similar products' suggest karne se pehle consent, cold-start users, diversity, bias aur click-through evaluation define karo. Model suggestions ko filter/authorisation ke saath serve karo; privacy and retention settings current terms se verify karo.

Polly: text se speech

Polly text-to-speech APIs se supported languages/voices mein audio generate kar sakta hai. Output ko playback ya supported format (e.g., MP3) mein store karne ka option ho sakta hai; standard/neural voices, narration styles aur Brand Voice-type custom engagement availability check karo. Character-based pricing ke current terms, voice rights aur audio retention assess karo.

Example: educational app paragraph read kare. Pronunciation, language, accessibility aur user consent test karo; audio reuse ke legal terms verify karo.

Amazon Q: enterprise aur developer assistant

Q Business: connected enterprise information par supported questions, summaries, content aur task workflows ke liye. Connector permissions aur user-level authorisation essential hain; answer verify karo.

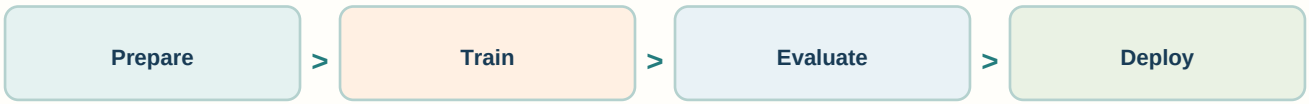
Q Developer: coding, tests, software upgrade, troubleshooting, scanning aur AWS assistance ke use cases. Suggested patches aur multi-step actions review/test karke approve karo. Dono assistants generated responses mein errors kar sakte hain.

Rekognition: image aur video analysis

Supported image/video APIs objects, scenes, text, activities, content moderation aur selected facial-analysis/search use cases address kar sakti hain. Custom Labels se task-specific categories, jaise machine parts ya unhealthy plants, ke liye labelled images aur model training workflow use ho sakta hai.

Face processing ke legal rights, consent, bias, false-match/error rate aur human oversight specially important hain. Sensitive surveillance ya high-impact decisions bina appropriate governance aur validation ke mat automate karo.

SageMaker AI: model banane se production tak



SageMaker AI tooling ML lifecycle cover karta hai: data preparation, feature management, training/tuning, model deployment aur MLOps workflows. Select supported components based on data, team skills, infrastructure, access aur cost. 'Managed' model quality ka guarantee nahi.

Autopilot aur Canvas

Autopilot: tabular data aur target column se supported automated candidate model building/training/tuning. Number predict karna regression; category (spam/not spam) classification. Candidate metrics aur feature leakage review karo; deployment se pehle holdout testing aur fairness assessment karo.

Canvas: business analysts ke liye visual point-and-click ML workflows ka option. No-code interface data quality, experiment design aur human review ki zaroorat eliminate nahi karta. Current supported data sources/features check karo.

Clarify, data labeling, Data Wrangler

Clarify: selected attributes ke across potential bias aur model explainability/feature importance reports evaluate karne mein help karta hai. Bias metrics limitations aur protected-group legal context carefully review karo; bias detection fairness proof nahi.

Data labeling: images, text ya video ko labels dekar training datasets build karte hain. Instructions, reviewer quality, annotation disagreement aur privacy govern karo.

Data Wrangler: visual data selection, cleaning, exploration, feature engineering aur visualisation workflows provide kar sakta hai. Missing values, data leakage aur train/test split explicitly handle karo.

Edge aur Feature Store

SageMaker Edge: historical model optimisation, secure deployment/monitoring aur device-fleet workflows ka option. Compiler model ko edge hardware ke liye prepare kar sakta tha; OTA deployment aur agent multiple models/selected prediction samples manage karne mein help kar sakte the. Current lifecycle and supported alternatives verify karo; sample upload privacy and device connectivity settings review karo.

Feature Store: named/versioned features ko teams ke across reuse karne aur supported training/inference workflows mein access karne ke liye repository. Online/real-time aur offline/batch access patterns, metadata, freshness aur consistency plan karo. Supported Athena integration/offline queries ka configuration verify karo; training-serving skew ab bhi measure karo.

Geospatial, HyperPod aur JumpStart

Geospatial: satellite/location-type data sources, preprocessing, pretrained models aur interactive map visualisation se agricultural yield, insurance risk, urban planning ya retail-site analysis use cases address kiye ja sakte hain. Data license, cloud cover, bias, geographic scale aur feature availability check karo.

HyperPod: distributed ML training infrastructure ke liye managed capabilities; supported Trainium/NVIDIA accelerator clusters aur distributed libraries use ho sakti hain. Checkpointing aur supported failure detection/recovery long jobs mein help karte hain, but uninterrupted training guaranteed nahi. Quotas, placement, orchestration, experiment cost aur restore test carefully plan karo.

JumpStart: supported pretrained/open models aur solution templates se prototype, deploy ya fine-tune ka starting point. Model license, input/output, vulnerabilities, cost aur dataset handling review karo. Historical catalogue counts constantly change hote hain.

Model building, training, deployment aur pipelines

Model building mein candidate algorithms aur evaluation metrics compare karo. Supported Studio notebooks, JupyterLab, RStudio ya code editor environments use kiye ja sakte hain. Training mein CPU/GPU/accelerator capacity, hyperparameter tuning, distributed frameworks (e.g. DeepSpeed, Horovod, Megatron where supported), checkpoint and reproducibility plan karo.

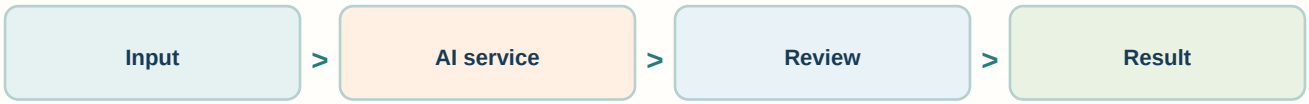
Inference deployment options workload ke real-time/batch requirements, throughput aur cost se choose karo. Model version, latency, access, drift aur rollback monitor karo. Pipelines data preparation → training → evaluation → approval → deployment ko automate kar sakti hain; thresholds aur approval gates define karo. Studio Lab jaise learning environments ke limits, eligibility aur no-cost terms current docs se check karo.

Frameworks aur learning environments

Apache MXNet/Gluon, PyTorch aur TensorFlow deep-learning framework examples hain; ecosystem lifecycle, supported versions aur dependencies current docs se verify karo. Hugging Face pretrained models/fine-tuning NLP aur other tasks ke liye relevant ho sakte hain; model license aur provenance inspect karo. TorchServe, distributed training libraries aur framework-specific tools ka support separately check karo.

Deep Learning AMIs preconfigured EC2 environments; Deep Learning Containers prebuilt container images; SageMaker managed workflow another option. Compatible containers ECS/EKS/EC2/SageMaker par run ho sakte hain; images, GPU drivers, platform versions aur security patches check karo. AMI/container artifact ke price ke alawa compute, storage, network aur other charges ho sakte hain.

Specialist AI: documents, voice, language aur health



Textract: document se text, forms aur tables

Textract scanned documents/images/PDFs se text aur supported handwriting, tables aur form fields extract kar sakta hai. Queries se natural-language question jaise 'customer name kya hai?' ke liye supported fields dhoondhne ka workflow hai. Use cases invoices, receipts, paystubs, loan/claims documents; formatting aur scan quality impact accuracy.

Flow: invoice upload → extract → confidence/validation rules → low-confidence values par A2I human review → approved data store. Human reviewer permissions, data retention aur sensitive information handling ensure karo. Every document perfectly processed hoga, assume mat karo.

Transcribe: speech ko text

Transcribe audio files (supported WAV/MP3 etc.) ya supported live streams ko text mein convert karta hai. Word-level timestamps original audio location tak navigate karne mein help karte hain. Background noise, overlapping speakers, accents, volume, pitch, speed aur language switching accuracy affect kar sakte hain.

Customer-service transcripts, subtitles aur content analysis use cases hain. Transcribe Medical healthcare speech/clinical documentation workflows ke liye; Call Analytics supported call-specific transcripts/insights ke liye. Consent, sensitive data, output errors aur clinician review required; supported languages/features check karo. Console, CLI/SDK se compatible jobs initiate kiye ja sakte hain.

Translate: languages ke beech text

Neural machine translation supported language pairs ke text translate karti hai. Website/app localisation, large text analysis aur cross-language communication use cases hain. Domain terminology, context, cultural nuance aur human review important hain; sensitive documents ke data handling and pricing terms verify karo.

DeepComposer aur DeepRacer: learning by doing

DeepComposer historical keyboard-plus-learning service example tha jahan tutorials, sample code aur data ke saath generative music models explore kiye ja sakte the. Hardware/service status and current alternatives verify karo.

DeepRacer reinforcement-learning education ka model-car/simulator example hai: agent actions leta hai, reward signal se long-term goal optimise karne ki training hoti hai. Virtual track aur physical model deployment ka learning pattern tha. Current program, model-car support, safety aur compute charges check karo. RL ko labelled training examples ki exact definition ke bina oversimplify mat karo.

HealthLake: health data organise karo

Health data clinical notes, lab results, claims, medical images, conversations ya ECG/EEG jaise time series mein scattered aur inconsistent ho sakta hai. HealthLake supported healthcare-data storage, transformation, query aur analysis workflows aur medical NLP integrations provide kar sakta hai. Applicable data model, eligibility, Region, integrations aur current retention settings verify karo.

HIPAA-eligible label ke baad bhi appropriate contractual agreements, access control, audit, encryption, data minimisation aur clinical/legal review organisation ko karna hota hai. Data extraction/normalisation errors ko verify karo.

HealthScribe: clinical documentation assistance

HealthScribe supported patient-clinician conversation processing mein speech recognition aur generative AI se preliminary notes, speaker roles aur medical terms generate karne ka option hai. Consent, clinician review, privacy settings aur actual current service retention terms verify karo. Draft clinical note authoritative diagnosis nahi hai; accountable clinician ko correctness check karni chahiye.

Panorama: edge computer vision

Panorama devices/SDK ka historical on-premises IP-camera computer-vision use case: multiple feeds se local inference, manufacturing defects, asset tracking ya traffic/lane monitoring. Limited bandwidth ya local data-processing requirements mein edge architecture relevant ho sakti hai. Current appliance, partner-device and service lifecycle verify karo. Low latency, correct detection aur physical safety outcome test-dependent hain; camera privacy aur independent safety controls necessary hain.

Kiro: AI-assisted engineering system

Kiro AI-assisted development workflows ke liye IDE/CLI aur supported agentic capabilities ka example hai. Requirements/specification, code changes aur validation mein assistance mil sakti hai; actual integrations/availability check karo. Agent ko repository access dene se pehle scope, approvals aur secret handling define karo. Generated code, dependency changes aur test evidence human review ke bina trust mat karo.

Summary quiz

• Scanned invoice mein tables? Textract; speech recording ko text? Transcribe; translated text? Translate.

• Prompt-based foundation model app? Bedrock; custom ML training lifecycle? SageMaker AI.

• Clinical AI output final diagnosis? Nahi: qualified human review and privacy controls.

• ML model accurate dikh raha hai; kya deployment complete? Nahi: bias, drift, access, costs aur monitoring evaluate karo.

Management aur governance: cloud ko organised aur observable rakho

Set rules > Observe > Investigate > Improve

Story: 30 teams, 300 accounts, ek control room

Riya ki company ab multiple teams aur AWS accounts use karti hai. Agar har team manual settings kare, permissions aur configurations inconsistent ho sakte hain. Governance ka goal innovation rokna nahi: clear account ownership, safe guardrails, standard deployment, audit, monitoring aur remediation provide karna hai.

Flow: accounts organise karo → approved infrastructure deploy karo → signals aur changes monitor karo → finding par act karo → review/improve. Ek dashboard se sab issues automatically solve nahi hote.

AWS Auto Scaling: related resources ka scaling plan

AWS Auto Scaling selected applications/resources ke scaling plans configure karne aur capacity adjust karne mein help karta hai. Historical supported examples EC2 instances/Spot fleets, ECS tasks aur DynamoDB tables/indexes jaise resources hain; exact current support check karo. Target utilisation, min/max limits aur application dependencies carefully configure karo. EC2 Auto Scaling instance fleet management se iska scope compare karo.

CloudFormation: infrastructure ko code banao

CloudFormation template AWS resources aur relationships describe karti hai; deployed resources ka collection stack kehlata hai. Example: app server, security group aur load balancer ko versioned template mein define karo. Resource dependencies ko supported mechanism se model karo aur updates via stack perform karo. Infrastructure Composer visual editing ka option ho sakta hai.

Change set review karo: kuch property updates resource replace ya interrupt kar sakti hain. Stateful database/storage par deletion/replacement behavior, backup, IAM, quotas aur rollback plan separately assess karo. Template version control helpful hai; secure defaults aur review automatically guaranteed nahi.

CloudTrail: AWS API activity ka audit trail

CloudTrail supported API activity record kar sakta hai — console, CLI, SDK aur service-initiated calls bhi scope mein aa sakte hain. Event fields mein caller identity, time, source IP, action, request/response details available ho sakte hain, depending on event and configuration. Management events aur data events ki coverage/cost alag hain; organisation trail, log storage, encryption aur retention plan karo.

Question: bucket policy kis identity ne change ki? Relevant CloudTrail event ka time, principal, action aur source investigate karo. CloudTrail application logs ya full network traffic ka replacement nahi.

CloudWatch: metrics, logs, alarms

CloudWatch operational monitoring ke liye metrics, logs, alarms, dashboards aur supported event integrations provide karta hai. AWS/on-premises application signals combine karne ka use case ho sakta hai. Metric dimensions, period, statistic, missing-data handling aur alarm action ko precisely configure karo.

Example: checkout error rate + latency + dependency saturation dashboard; actionable alarm owner/runbook ke saath. Logs mein request correlation ID useful hai, but tokens/passwords/PII avoid karo. Retention, ingestion aur query costs monitor karo.

Compute Optimizer: right-sizing recommendation

Utilisation history dekhkar supported EC2, EBS aur Lambda configurations ki recommendations mil sakti hain; supported resource list/charge model current docs se check karo. Over-provisioned resource cost badhata hai; under-provisioned resource slow ho sakta hai. CPU patterns, volume IOPS aur function memory jaise dimensions evaluate ho sakte hain.

Recommendation ko production change mat samjho: peak workload, memory/latency, architecture compatibility, burst behavior aur business constraints load test karo. Infrequent seasonal traffic historical average mein hide ho sakta hai.

Console Mobile Application: on-the-go visibility

Mobile console supported resources, alarms, metrics, billing/health information aur selected actions dekhne ka interface hai. Account/Region, mobile device security aur MFA enforce karo. Supported features aur current UI version verify karo; mobile alert critical incident communication plan ka ek part hai, pura plan nahi.

Control Tower: multi-account landing zone

Control Tower supported landing zone aur governance controls establish karne mein help karta hai. Separate workloads/accounts, central logging/security aur policy-driven guardrails ka foundation ban sakta hai; Organisations, Config aur Service Catalog jaise components relevant ho sakte hain.

Example: development team ko approved baseline ke saath new account mile, central team governance visibility rakhe. Account owner, emergency access, control coverage, exceptions aur custom requirements review karo. Landing zone automatic compliance certification nahi.

AWS Config: resource configuration history

Config supported resource inventory, configuration history aur selected rules evaluations capture kar sakta hai. Example: storage resource ki policy/setting expected guardrail se match karti hai ya nahi? Recording scope, supported resource types, evaluation timing aur rule permissions check karo.

Contrast: CloudTrail batata hai kis API call se change hua; Config resource ki recorded configuration aur rule evaluation view karne mein help karta hai. Dono ki scope configurable hai.

AWS Health: provider events aur personal impact

AWS Health supported service events, account/resource-relevant issues aur scheduled changes ki alerts/guidance dikhata hai. General service status aur account-specific view different ho sakte hain. Incident mein application metrics aur dependency impact correlate karo; provider event dikhe bina apni configuration ko ignore mat karo.

Launch Wizard: guided enterprise deployment

Launch Wizard supported third-party workloads (e.g. SQL Server Always On ya SAP/HANA scenarios, where available) ke sizing/configuration/deployment mein guide karta hai. Performance, nodes aur connectivity requirements input karne par proposed resources aur estimated cost compare karo; supported CloudFormation artifacts future deployments mein baseline ho sakte hain. Production readiness, license, HA and security approval independently validate karo.

License Manager: software licensing controls

License Manager AWS aur supported on-premises servers ke license inventory/rules workflows mein help karta hai. Vendor agreements — Microsoft, SAP, Oracle, IBM etc. — alag hote hain; rules license terms ko model kar sakti hain, with selected preventive or notification behavior. Dashboard, Service Catalog, Systems Manager, Organizations aur BYOL marketplace integrations support/version par depend karte hain. Actual contract aur compliance interpretation legal/licensing team se confirm karo.

Managed Grafana aur Managed Prometheus

Managed Grafana: multiple metrics/logs/traces sources ko query, correlate aur visualise karne ka managed Grafana workspace. Supported data sources CloudWatch, OpenSearch, X-Ray, SiteWise, Timestream aur managed Prometheus jaise ho sakte hain. SSO, dashboard permissions aur data source access configure karo.

Managed Service for Prometheus: Prometheus-compatible metric ingestion/storage/query aur query language ke use cases, especially containers. Workspace reliability, Region/AZ architecture, metric cardinality, retention, collection and query cost current docs se check karo. Grafana dashboard aur Prometheus metrics store alag roles hain.

Organizations: accounts aur guardrails

Organizations multiple AWS accounts create/group/govern karne mein help karta hai; organisational units policy grouping aur consolidated billing enable kar sakte hain. Central audit, resource-sharing aur security integrations configure karo. SCP guardrails permissions ko constrain kar sakte hain; SCP khud IAM allow grant nahi karta. Account owner, root security, emergency access aur organisational policy changes test karo.

OpsWorks aur Proton: platform operations ke historical examples

OpsWorks: Chef/Puppet-based server configuration management aur stacks ka historical service. Availability/end-of-support aur replacement before new design check karo.

Proton: platform teams ke liye container/serverless infrastructure templates, deployment and environment standards coordinate karne ka historical offering. Current lifecycle aur alternatives

verify karo. Regardless of tool, versioned templates, ownership, security review and upgrade path essential hain.

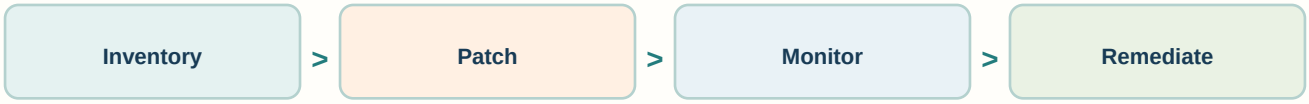
Q Developer in chat applications: ChatOps

Supported chat integrations operational notifications aur authorised diagnostic commands, Lambda invocation ya support case workflows enable kar sakti hain. Slack/Microsoft Teams/other supported channels ki exact integration current docs se verify karo. Chat channel membership aur AWS execution role permissions scope karo; sensitive findings public channel mein mat bhejo. An alert link aur runbook se team response faster ho sakta hai, but chat itself audit/approval replacement nahi.

Service Catalog: approved products

Organisation approved VM images, servers, databases ya multi-tier architectures ko catalogue ke roop mein publish kar sakti hai. Users permitted product provision karte hain; central team template/version/access governance own karti hai. Approved item ka configuration, costs aur compliance evidence independently validate karo.

Systems Manager: infrastructure operations toolbox



Systems Manager supported managed nodes/resources ke operational data aur automation ko organise karta hai. Relevant EC2/on-prem nodes ke agent, IAM aur network prerequisites verify karo. Resource groups application layer ya environment ke hisaab se logical grouping karte hain; operational view supported CloudTrail, Config, inventory, patch, CloudWatch, Trusted Advisor aur Health information link kar sakta hai.

• Run Command: authorised shell/PowerShell commands ya administrative actions managed nodes par. Command scope aur output logs review karo.

• State Manager: desired configuration associations define/maintain karne ke workflows, jaise approved firewall/agent configuration. Changes ko safe rollout aur drift monitoring se manage karo.

• Inventory: installed software, OS/agent aur selected configuration data collect/query karo. Collection timing aur scope verify karo.

• Maintenance Window: planned recurring time window for supported operational tasks; change impact aur timezone specify karo.

• Patch Manager: supported OS/software patch selection, scanning/deployment and baselines. Maintenance window aur reboot behavior application impact ke saath test karo.

• Automation: repeatable runbooks for supported maintenance/deployment, AMI creation/updates, patch/agent workflows. Execution IAM aur approval steps define karo.

• Parameter Store: configuration values aur supported secure strings store karta hai; SecureString ke encryption ke liye KMS configure ho sakta hai. Every parameter automatically encrypted nahi hota. Sensitive values logs mein expose mat karo.

• Distributor: package creation, versioning aur distribution; Run Command/State Manager ke through supported installation. Package authorisation aur signed provenance assess karo.

• Session Manager: authorised interactive sessions ke liye browser/CLI access; supported setup mein inbound SSH port ya bastion ke bina access possible. Managed-node agent, IAM aur network endpoints prerequisites hain. Session logging ko explicitly configure karo; CloudTrail API event aur actual session command/output logs alag cheezein hain.

Trusted Advisor aur notifications

Trusted Advisor supported cost, security, performance aur operational checks/recommendations show karta hai; coverage/support-plan scope vary kar sakta hai. Each finding ka owner aur remediation plan rakho. AWS User Notifications supported Health events, CloudWatch alarms aur instance state changes ko console notification centre, email, supported chat/mobile push ya API paths par distribute kar sakta hai. Alert fatigue aur routing/test failure plan karo.

Well-Architected Tool

Tool workload review organise karta hai: workload describe karo, questions answer karo, risks aur improvement actions record karo. Six pillars: operational excellence, security, reliability, performance efficiency, cost optimisation aur sustainability. Tool/fee/Region support current documentation se confirm karo. Review ek structured discussion hai, security guarantee ya automatic compliance certificate nahi.

Incident exercise

Website 5xx badh raha hai: (1) CloudWatch metrics aur ALB/app logs se impact check; (2) Health se provider event check; (3) CloudTrail mein recent deployments/API changes; (4) Config history se relevant resource settings; (5) Systems Manager se authorised remediation; (6) incident notes aur follow-up create. Root cause prove hone tak guesses ko facts mat banao.

Media: camera se viewer ki screen tak

TASVEER SE SAMJHO
Media: camera se viewer tak

Video capture ke baad suitable formats aur resolutions taiyar ho sakte hain, phir viewers ko stream deliver hoti hai. Latency, playback compatibility aur content rights bhi plan karo.

Input > Process > Deliver > Monitor

Video workflow ko kitchen ki tarah samjho

Raw camera feed ingredient hai, encoding usse suitable formats mein prepare karti hai, packaging devices ke format/rights ke liye output banati hai, aur delivery viewer tak pahunchati hai. Recorded file aur live stream ke workflows different hain. Content rights, consent, network, codec, availability aur cost har stage par matter karte hain.

Elastic Transcoder: file conversion ka historical option

Elastic Transcoder cloud media files ko target playback formats mein convert karne ka historical service example tha: ek original video se phone/tablet/PC compatible versions. Service lifecycle, current replacement, supported codecs aur pricing verify karo. Quality, aspect ratio aur audio sync sample output par test karo.

Interactive Video Service (IVS): interactive live stream

IVS supported live streaming workflows ke liye managed option hai. Streaming software live video feed bhejta hai; player SDK aur timed metadata APIs se application mein interactive features add kiye ja sakte hain. Example: live class with chat/polls; video delay aur moderation requirements pehle decide karo. Viewer latency network aur location ke hisaab se vary karti hai; global zero-latency guarantee nahi.

Nimble Studio: creative studio ka historical workflow

Animation/VFX team ko virtual workstations, scalable rendering aur shared high-speed storage ke saath distributed collaboration ki need ho sakti hai. Nimble Studio historical managed studio example tha. Current service lifecycle, asset security, licensing, artist access aur migration alternatives verify karo.

Elemental Appliances and Software: on-prem video processing

Broadcast team ko existing camera/router interfaces, managed network ya bandwidth limitations ke chalte on-premises encoding/package/delivery chahiye ho sakti hai. Elemental Live, Server aur Conductor ke appliance/software deployment forms historical examples hain; Link compact device se live video cloud tak bhejne ka use-case hai. Current device models/support, connectivity, hardware operations aur cost verify karo.

MediaConnect: live video transport

MediaConnect selected live video contribution/distribution transport workflows mein help karta hai. Example: stadium se live feed ingest karo, partner distributor ko share karo, ya processing ke liye stream replicate karo. Source redundancy, stream permissions, network encryption, transport monitoring aur failover plan karo. IP workflow choose karne se satellite/fibre jaise reliability automatically guaranteed nahi hoti.

MediaConvert: recorded file se viewing formats

File-based transcoding ke liye MediaConvert evaluate karo. Example: recorded course video → multiple resolutions/codecs → on-demand playback assets. Job status, role access, input validation, output storage aur encoding cost monitor karo. Audio/video quality aur device compatibility test karo.

MediaLive: live encoding

MediaLive supported live video encoding/processing ke liye hai. Large live source ko multiple delivery bitrates/resolutions mein encode karke live event ya continuous channel ke workflows build kiye ja sakte hain. Input redundancy, health alerts, output destination aur per-hour/resource costs assess karo.

MediaPackage: format aur protection

MediaPackage supported video packaging/origin workflows se multiple device playback formats aur selected DVR-style start-over/pause/rewind capabilities provide kar sakta hai. DRM integration aur content entitlement configure karne ki need ho sakti hai. Device compatibility, rights, failover, latency aur origin access test karo.

MediaStore aur MediaTailor

MediaStore: media-optimised origin-storage workflow ka historical service example. Current lifecycle aur recommended origin alternatives check karo.

MediaTailor: supported streams mein ad insertion aur channel assembly workflows. Ad-decision integration, content/ad measurement aur client/server-side metrics support use-case ke hisaab se verify karo. Privacy/consent, ad policy, content rights, playback and reporting quality separately test karo.

Related: Kinesis Video Streams connected-device video ingestion/playback scenario ke liye; broadcast media workflow se requirements compare karo.

Migration aur transfer: ghar shift karne jaisi planning

TASVEER SE SAMJHO
Migration: ghar shift karne jaisi taiyari

Pehle current system aur dependencies samjho, phir move ka plan banao aur destination par test karo. Moving truck ek analogy hai; data transfer ka actual method alag ho sakta hai.

Discover > Plan > Move > Validate

Chhe steps

• Discover: servers, apps, owners, network dependencies, data aur licenses identify karo.

• Assess: business criticality, target compatibility, data sensitivity, costs aur RPO/RTO define karo.

• Plan: rehost/replatform/refactor/retain/retire decision workload-specific lo; cutover aur rollback owner assign karo.

• Move: application aur data replicate/transfer karo, capacity aur bandwidth monitor karo.

• Validate: application behavior, row counts, checksums, security, performance aur user acceptance test karo.

• Cutover: write freeze/replication lag, communication, rollback window aur post-cutover monitoring plan karo.

Application Discovery Service

Existing data-center servers ka configuration, utilisation aur dependencies collect karke migration inventory banane mein help karta hai. Collected information se application dependency map aur TCO estimate improve ho sakta hai; supported export CSV aur Migration Hub integrations evaluate karo. Data collection scope, encrypted storage, permissions aur privacy verify karo. Automated discovery incomplete ho sakti hai; application owners se validate karo.

AWS Transform MGN: server migration

Supported migration workflow source physical/virtual/cloud servers ko AWS target environment mein replicate aur launch karne mein help karta hai. Non-disruptive test launch ka use-case cutover se pehle application behavior check karna hai. Server replication se app compatibility, licenses, data consistency ya minimal downtime automatically guaranteed nahi. Current naming, OS support, target setup, security and rollback verify karo.

Database Migration Service (DMS) aur DMS Serverless

DMS supported source-target databases ke full load aur selected ongoing change-data-capture migrations mein help karta hai. Homogeneous migration same-family database ke beech ho sakti hai; heterogeneous migration mein schema/type/behavior compatibility ka extra work hota hai. Source database migration ke dauran operation continue kar sakta hai depending on setup, but cutover downtime aur replication lag separately plan karo.

Serverless DMS replication capacity model simplify kar sakta hai; source/target support, networking, permissions, schema conversion, monitoring aur cost check karo. Data consistency ke liye counts aur critical records reconcile karo; migration complete message business validation ka replacement nahi.

Mainframe Modernization

Mainframe application migration/modernisation workflows mein assessment, code/tooling, managed runtime aur phased application transition ka use-case hai. Dependencies, transaction semantics, licensing, staff expertise aur regression tests critical hain. Rehost vs refactor choice business risk aur target compatibility par depend karti hai; regulated workloads ke liye sign-off lo.

Migration Hub

Migration Hub supported applications/migration activities ke tracking aur consolidated progress view mein help karta hai. Source inventory, migration tools aur progress indicators scope/version par depend karte hain. Dashboard par status green hona security, performance ya user acceptance ka final proof nahi.

Snow Family: physical data transfer aur edge

Large datasets ya weak connectivity mein physical devices use-case ho sakte hain; Snowball/Snowball Edge ke historical variants data transfer, storage aur selected edge compute/GPU workflows address karte the. Device availability, capacity, Region, shipping, chain of custody, encryption, operational limits aur lifecycle current offering se verify karo. Old 80/210 TB capacity figures ko current inventory na samjho.

Example: remote site ka large archive device par securely load → verified shipping → cloud import → checksum validation. Physical transfer mein custody aur timelines plan karna zaroori hai.

DataSync: network ke through data movement

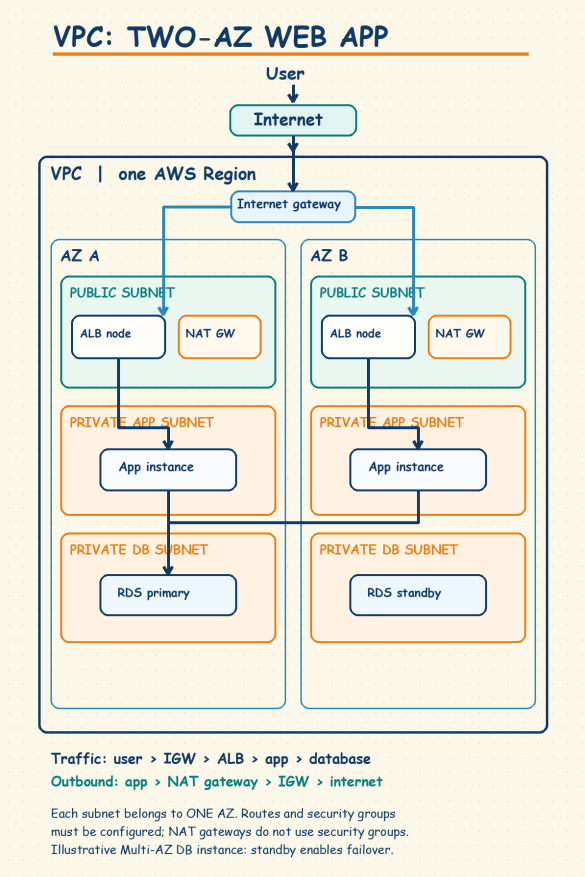
DataSync supported on-premises/cloud storage systems ke beech data copy automate karne ka option hai. Internet ya Direct Connect connectivity, agent requirements, permissions, metadata, integrity verification aur bandwidth scheduling check karo. Data volume aur changed data rate se transfer duration estimate karo; copying, storage, requests aur data transfer pricing separately review karo.

Estimate: 2 TB data aur sustained 100 MB/s theoretical throughput par roughly 20,000 seconds, ya 5.6 hours, assuming decimal units and no overhead. Actual encryption, network congestion, file count, retries aur validation se time badal sakta hai.

Transfer Family: managed file-transfer endpoints

Transfer Family supported SFTP, FTPS, FTP aur other currently supported protocols ke endpoints offer karta hai, selected S3/EFS-backed workflows ke saath. Partner authentication, user isolation, IAM/storage access, encryption, logging, network exposure aur protocol security configure karo. FTP without transport encryption sensitive data ke liye inappropriate ho sakta hai; supported endpoint options verify karo.

Migration quiz: DataSync = storage-to-storage transfer; Transfer Family = users/partners ke file-transfer endpoints; DMS = database migration/replication; MGN = supported server migration. Inhe interchangeable mat samjho.



Networking aur content delivery: request ki poori journey

User > Route > Private network > Service

Story: customer browser mein shop kholta hai

Customer shop.example.com type karta hai. DNS name ko address/target se resolve karta hai; connection network path se edge/load balancer tak jata hai; security rules permitted traffic allow karte hain; application request process karke response bhejti hai. Kisi bhi layer par mistake ho to browser ko similar timeout ya error dikh sakta hai. Isliye packet path ko diagram aur logs ke saath trace karo.

Flow: user → DNS → edge / API / load balancer → VPC network → app → database. Har connection ke liye identity, route, port, encryption, availability aur return path check karo.

API Gateway: managed API entry

API Gateway supported REST, HTTP aur WebSocket APIs create/publish/manage karne ka option hai. Example: POST /orders → request validation/authorisation → Lambda/backend integration. Rate limiting/throttling, auth, CORS, logging aur deployment stage explicit design karo. CORS browser behavior control hai, API authorisation ka substitute nahi. API type ke features, latency, quotas aur cost compare karo.

App Mesh: service-mesh historical example

App Mesh microservices ke beech traffic/visibility configuration ka service-mesh example tha, including compatible proxy-based workflows. Service-to-service routing, retries, observability aur security useful concerns hain. Product lifecycle aur alternatives current docs se check karo; mesh enable karne se application authorization aur failure handling automatically perfect nahi hoti.

CloudFront: users ke paas cached content

CloudFront CDN supported edge locations se content deliver/cache kar sakta hai. Origin S3 ya application backend ho sakta hai. Example: product images edge cache se; sensitive account response ko authorised cache behavior ke saath handle karo. Cache key, TTL, invalidation, origin access, TLS aur signed access carefully configure karo. Stale content aur cache poisoning risks test karo.

Cloud Map: service discovery

Cloud Map supported resources ke names/registrations se applications ko service endpoints discover karne mein help karta hai. Example: worker ko orders.internal service address chahiye, hard-coded IP nahi. Registration health, DNS/API discovery, TTL aur service availability design karo; discovery alone network routes ya permissions create nahi karta.

Direct Connect: dedicated network connection

On-premises network se AWS tak dedicated connectivity option hai. Bandwidth, provider circuit, virtual interfaces, routing/BGP, physical path diversity, failover aur charges assess karo. Dedicated link automatically encrypted VPN ya TLS ka equivalent nahi; confidentiality requirements ke liye additional encryption plan karo. DNS, CIDR overlap aur return routing bhi verify karo.

Elastic Load Balancing: traffic distribute karo

ELB incoming traffic ko registered targets aur health checks ke hisaab se distribute karta hai. Type requirement se choose karo:

• ALB: HTTP/HTTPS layer-7 routing, host/path-based rules aur supported app features.

• NLB: transport-level TCP/UDP/TLS use cases, supported configuration aur workload behavior ke hisaab se.

• Gateway Load Balancer: supported network appliance fleet insertion/scaling patterns.

• Classic Load Balancer: older/basic offering; supported use aur current lifecycle check karo.

Health-check path/port, listener, target groups, security groups, subnet placement aur graceful draining test karo. ALB healthy hone se database/backend dependency healthy hai, ye guarantee nahi.

Global Accelerator: global entry aur traffic routing

Global Accelerator supported application endpoints ke liye static entry IPs aur AWS global network routing options provide karta hai. Endpoint health, traffic distribution, regional failure behavior aur actual client latency evaluate karo. CloudFront caching CDN hai; Accelerator ka networking/routing role different hai. Static IPs ko authorization ke replacement mat samjho.

Integrated Private Wireless aur AWS Private 5G

Private wireless enterprise sites par dedicated cellular connectivity ka design ho sakta hai; AWS/partner-integrated hardware, spectrum, coverage, devices aur operations ko provider-specific evaluate karo. Historical Private 5G offering ka scope/lifecycle current docs se check karo. Device SIM/identity, radio planning, segmentation aur safety requirements production se pehle verify karo.

PrivateLink: private service connectivity

PrivateLink supported endpoint/service connections ko private networking path se access karne mein help karta hai, without assuming public internet route. Interface endpoint network interfaces, DNS, endpoint/service permissions aur security groups ko correct configure karo. Private path ka matlab IAM, endpoint policy, encryption ya service-side access checks unnecessary nahi. Endpoint hourly/data processing charges assess karo.

Route 53: DNS aur routing policies

Route 53 managed DNS, domain registration aur supported routing/health-check features provide karta hai. Browser DNS name query karta hai; answer selected record/routing policy ke according milta hai. Latency, weighted, failover aur other policies use-case dependent hain. DNS TTL/cache aur health check behavior test karo; failover instantaneous guarantee nahi.

Example: primary endpoint unhealthy → configured DNS failover secondary answer, but client cache aur secondary app/database readiness matter karti hai.

Transit Gateway: networks ka hub

Transit Gateway multiple VPCs aur supported on-prem connections ko hub-and-spoke model mein connect kar sakta hai. Attachments aur transit gateway route tables se segmentation aur routing define hoti hai. Example: development VPC production database tak access nahi kare; explicit routes aur security controls design karo. Return route, overlapping CIDR, DNS aur per-attachment/data charges check karo.

Verified Access: application access policy

Verified Access supported corporate app access ke liye identity/device context-based policies enforce karne ka option hai, without requiring traditional broad network VPN access for that pattern. Trust provider configuration, app permissions, device posture aur audit logs validate karo. Application-level authorisation still needed.

VPC: apna logical network

VPC Region-scoped logically isolated network hai jisme IP ranges, subnets, routes aur security controls define karte ho. Ek subnet ek AZ mein hota hai. Public subnet ke route table mein internet gateway route hota hai; internet access ke liye applicable public IP, rules aur return routing bhi chahiye. Private subnet ke outbound internet ke liye NAT or other design evaluate ho sakta hai; public IP automatically nahi milta.

Security groups stateful allow rules hain; network ACLs stateless subnet-level rules support karte hain. App tier SG ko ALB SG se required port allow karo; database tier SG ko app tier se DB port allow karo. NACL return ports, DNS aur route tables ka role bhi test karo.

Exercise: customer → public ALB → private app → private database across two AZs. AZ-A aur AZ-B ke subnets separate draw karo; backup/restore aur data-layer failure separately handle karo.

VPC Lattice: service-to-service connections

VPC Lattice supported services across instances, containers aur serverless workloads ko service-network connectivity, routing/access policies aur monitoring model ke saath connect karne mein help karta hai. Service auth, resource sharing, DNS, network paths aur cost current features ke hisaab se evaluate karo. Policy layer app authorisation ka complete replacement nahi.

Site-to-Site VPN aur Client VPN

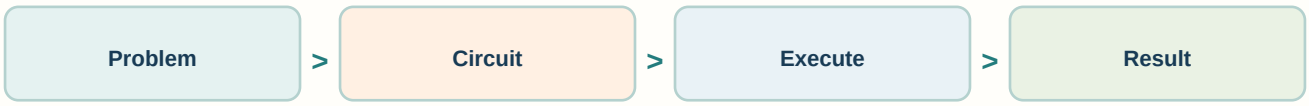
Site-to-Site VPN: on-prem network/customer gateway aur VPC ya Transit Gateway ke beech encrypted tunnel connectivity. Client VPN: individual remote users ke liye client-based VPN access to supported AWS/on-prem resources. Authentication, route propagation, network ACL/SG, tunnel health, redundancy aur return path test karo. Dedicated Direct Connect aur VPN combine karna design-specific decision hai.

Quantum: naye computation model ka exploration

TASVEER SE SAMJHO

Quantum: alag computation approach

Switch ordinary binary choice ki analogy hai; beech ka sphere quantum state ki visual idea deta hai. Ye visual full physics ya quantum advantage ka proof nahi; next notes mein model aur limitations padho.



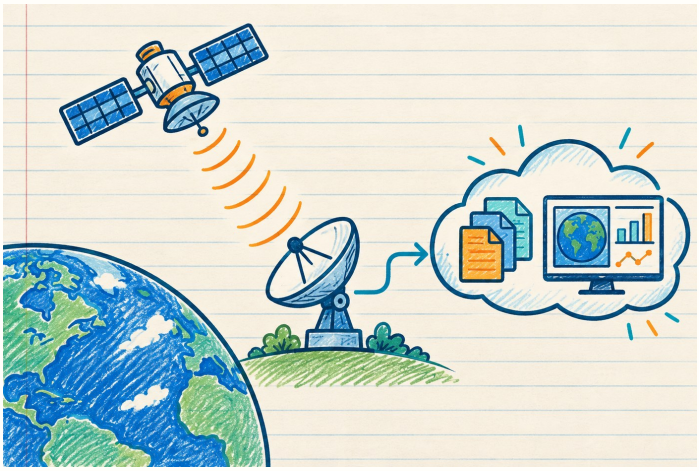
Amazon Braket

Quantum computing quantum-mechanical effects ko use karke algorithms explore karta hai. Chemical/material simulation, drug discovery, optimisation aur ML research potential areas hain, but every problem mein quantum speedup proven ya practical nahi. Braket supported development environment se circuits/algorithms build, simulators par test aur selected quantum hardware par jobs run karne ka access provide karta hai.

Example: small circuit simulator se validate karo → supported gate-based device par experiment → results classical baseline se compare karo. Provider devices, queue time, noise/error, quotas aur

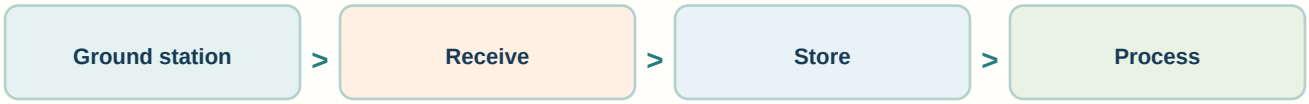
per-task charges current docs se verify karo. Historical partner names ko current catalog mat samjho.

Satellite: orbit se data ground tak



TASVEER SE SAMJHO
Satellite data ka safar

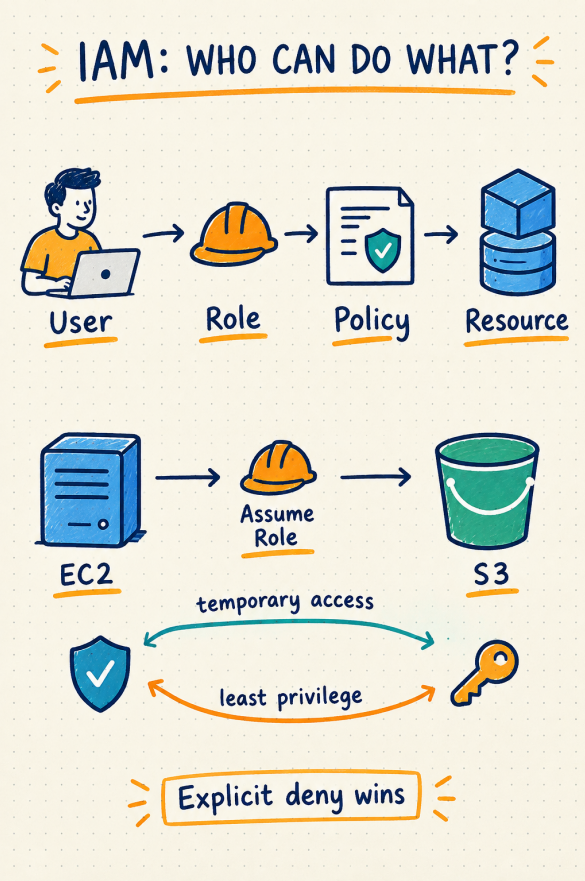
Satellite se signal ground station ko milta hai; data supported workflows mein cloud tak jaakar store aur analyse kiya ja sakta hai. Antenna time, licensing aur connectivity plan karni hoti hai.



AWS Ground Station

Satellite weather forecasting, Earth imaging, communications ya broadcast ke liye use ho sakta hai. Ground station antennas satellite se data receive (downlink) aur authorised command/control signal send karte hain. Khud ground station build/lease karne ke bajay managed Ground Station service supported contacts schedule, communicate aur data-processing integrations mein help kar sakti hai.

Example: imaging satellite ka pass schedule → data receive → permitted storage/processing pipeline → image analysis. Orbit/contact window, spectrum/legal approvals, data encryption, mission availability, location aur cost mission-specific hain. Managed scheduling se missed pass, satellite failure ya regulatory duties automatically disappear nahi hote.



Security, identity aur compliance: layers mein socho

Identity > Policy > Action > Audit

Ek building ke security layers

Online shop ko building samjho: ID card identity hai, room permissions authorisation, locked door network controls, camera logs detection aur fire drill recovery. Ek control fail hone par doosra control damage limit kare — isi ko defence in depth kehte hain. AWS infrastructure ki security aur customer configuration/data ki security ka shared responsibility split service par depend karta hai.

Cycle: identify assets → restrict access → protect data/network → detect abnormal activity → investigate/respond → recover → learn. Least privilege, temporary credentials, MFA, encryption, audit aur tested backups core themes hain.

Amazon Cognito: app users ka sign-in

Cognito web/mobile apps ke supported sign-up, sign-in aur user identity workflows provide karta hai. User pools authentication aur supported identity flows se relate karte hain; identity pools ka role supported AWS credential federation ke liye different ho sakta hai. Social/federated identity provider, token validation, account recovery aur MFA settings current requirements ke mutabik choose karo.

Example: customer sign in kare, API valid token check kare aur sirf us customer ke orders return kare. Authentication successful hone se har order access allowed nahi: object-level application authorisation zaroori hai.

Detective: investigation context

Detective security findings aur related activity ko investigate karne mein help karta hai. GuardDuty/Macie/Security Hub jaise supported inputs ke context se entities aur events ke relationships explore kiye ja sakte hain. Example: suspicious role activity ke time/source/resources correlate karo. A finding ya graph root cause ka automatic proof nahi; logs aur evidence validate karo.

GuardDuty: threat-detection signals

GuardDuty selected AWS data sources mein suspicious activity detect karne ka managed threat-detection service hai. Supported signals aur protection plans configuration/Region par depend karte hain. Finding par severity, affected account/resource, evidence, containment aur owner triage karo. Threat detection ko complete preventive security system ya 100% malicious activity coverage mat samjho.

Inspector: vulnerability assessment

Inspector supported EC2, container images aur other eligible workloads mein software vulnerabilities ya applicable exposure evaluate kar sakta hai. Findings mein risk context aur prioritisation signals mil sakte hain; supported scanning agents/integrations, package coverage aur current features check karo. Patch recommendation ko business criticality, exploitability aur regression testing se prioritise karo.

Macie: S3 mein sensitive data discover

Macie supported S3 inventory/analysis se sensitive data discovery aur classification workflows mein help karta hai. Example: customer identifiers galat bucket prefix mein aa gaye hon to selected discovery job signal de sakta hai. Detection sample/coverage aur classification accuracy perfect nahi; IAM/bucket policy, encryption, lifecycle aur incident response independent controls hain.

Security Lake: central security data

Security Lake supported AWS, SaaS, on-premises aur other data ko central security-data lake mein collect/normalise karne ke use cases address karta hai, with supported OCSF format and query/consumer integrations. Sources, Region/account scope, retention, access and ingestion costs configure karo. Central lake bana dena detections aur response ownership ka replacement nahi.

Verified Permissions: application ke fine-grained rules

Verified Permissions supported policy-based authorisation decisions ke liye use ho sakta hai. Example: 'support agent can view this order only when tenant matches and role permits'. Authentication se principal pata chalta hai; authorisation decide karta hai action allowed hai ya nahi. Policy testing, tenant isolation aur decision context validate karo; policy syntax correct hona business rule correctness ka proof nahi.

Artifact aur Audit Manager: evidence workflows

Artifact: AWS reports, selected agreements aur compliance evidence access karne ka portal. Scope, service, Region, reporting period aur agreement terms check karo. Provider report aapke app ka compliance certificate nahi.

Audit Manager: supported AWS usage/evidence collection aur assessment workflow organise karta hai. Controls, evidence completeness, human review aur auditors ke requirements explicitly map karo. Automated evidence sab organisational controls cover nahi karta.

Certificate Manager: TLS certificates

ACM supported TLS certificate provisioning, deployment aur renewal workflows simplify karta hai. ELB, CloudFront aur API Gateway jaise integrations ka exact scope/domain validation/Region requirements check karo. Certificate expiry, DNS validation, export permissions aur custom domain settings monitor karo. TLS encryption user authentication/authorization ka replacement nahi.

CloudHSM aur Directory Service

CloudHSM: dedicated hardware security module environment ki need ke liye option. Key custody, hardware isolation, user/operator responsibilities, backup aur recovery plan samjho; KMS se operational overhead/capability compare karo. Exportability/security standard details specific configuration aur current docs se verify karo.

Directory Service: managed directory options, including supported Microsoft AD integration. Domain/trust relationships, DNS, network paths, user lifecycle aur directory administrator access design karo. Directory availability/failover test karo.

Firewall Manager: multi-account security policies

Firewall Manager supported accounts/resources par firewall-related policies centrally manage karne mein help karta hai. WAF, network firewall ya other supported integrations ka actual scope verify karo. Policy rollout nonproduction mein test karo; false positives aur account/Region exceptions monitor karo. Central policy configured hone ka matlab all traffic automatically protected nahi.

IAM: AWS access ka foundation

IAM principals/users, roles aur policies ke through AWS actions/resources par access control karta hai. Workforce federation/Identity Center se temporary access prefer karo; workloads ko suitable role do. Role trust policy batati hai kaun assume kar sakta hai; permissions policy batati hai allowed actions/resources. Explicit deny applicable allow ko override karta hai; SCP, boundaries, session aur resource policies ka exact interaction request par depend karta hai.

Example: app role ko sirf reports/ objects read karne ka access. Bucket listing aur object reads alag actions/resources ho sakte hain. KMS encrypted data ke liye key permissions bhi lag sakti hain. AccessDenied mein caller, account, action, ARN, Region, conditions aur applicable policies inspect karo; administrator access blindly mat do.

KMS: cryptographic keys

KMS managed cryptographic keys aur supported AWS encryption integrations provide karta hai. IAM permissions aur key policy determine karti hain ki kaun key use/manage kar sakta hai. Key selection, rotation, deletion protection, cross-account access aur restore/recovery access review karo. Key disable/delete hone se encrypted data inaccessible ho sakta hai.

KMS key application password nahi hai. Secrets Manager secret value manage karta hai; KMS encryption key use kar sakta hai. Dono ke permissions distinct hain.

Network Firewall: network traffic controls

Network Firewall supported VPC traffic filtering/inspection capabilities deta hai. Architecture mein route tables ke through intended traffic ko firewall endpoint tak le jana padta hai; rule order, stateful/stateless behavior, logs, AZ placement aur fail-open/fail-closed behavior test karo. Firewall bhi application authentication aur secure code ka substitute nahi.

Resource Access Manager (RAM): resources share

RAM supported resource types ko accounts ya organisation ke saath share karne mein help karta hai. Resource share, owner account, invited principals aur applicable IAM/service permissions separately evaluate karo. Example: approved shared subnet ya transit gateway connectivity scenario. Shared hone se sab principals ko unrestricted access automatically nahi milna chahiye.

Secrets Manager: credentials ko code se bahar rakho

Secrets Manager passwords/API credentials jaise secrets ko store/retrieve aur supported rotation workflows se manage karta hai. Example: application role ko specific DB credential read permission; secrets ko environment logs, source repository ya screenshots mein expose mat karo. Rotation implementation, cached secret refresh, KMS access, network, audit aur cost test karo.

Security Hub CSPM: posture aur findings

Security Hub CSPM supported security checks aur findings aggregate/prioritise karta hai. Multiple account/Region coverage aur standards settings explicit configure karo. Findings ka triage owner, exception process aur remediation timeline define karo. Green dashboard comprehensive security/compliance guarantee nahi.

Shield: DDoS defence

Shield Standard aur Advanced DDoS protection offerings ke features, coverage, support aur pricing different hain. CloudFront/Route 53 aur other supported resources ke integrations aur Advanced protections current docs/terms se verify karo. Layered architecture, app scalability, incident response aur origin protection ab bhi important hain; every attack ko block karne ka promise mat samjho.

IAM Identity Center: workforce access

Identity Center supported workforce identity federation aur multiple AWS accounts/applications ke permission sets manage karne mein help karta hai. Identity provider, MFA, group membership, assignment, session duration aur break-glass access plan karo. Workforce users ko long-lived IAM credentials distribute karne ke bajay temporary access patterns evaluate karo.

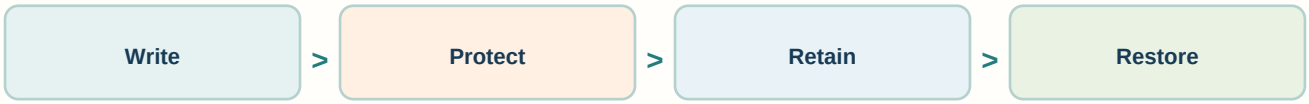
WAF aur CAPTCHA: web request filtering

WAF supported web applications/APIs par rules ke through HTTP request filtering karta hai. Common attack patterns, rate-based conditions aur managed/custom rules use ho sakte hain. CAPTCHA/challenge selected suspected bot traffic ke liye apply kar sakte ho; accessibility, user impact aur false positives test karo. WAF SQL injection ya authentication weaknesses ko fix karne ka substitute nahi.

Incident practice

GuardDuty alert mila: (1) affected account, role, resource aur timestamps verify; (2) relevant CloudTrail/other logs preserve; (3) authorised containment implement; (4) secret/key exposure assess; (5) recovery and credential rotation; (6) root cause aur follow-up document. Investigating while preserving evidence aur avoiding customer-data leakage essential hai.

Storage aur recovery: data ko sahi shape mein rakho



Teen storage mental models

Block: EBS jaise volume server ko disk blocks deta hai; filesystem/OS us par data organise karta hai. File: EFS/FSx jaise shared file systems directories/files interface dete hain. Object: S3 bucket mein key/object/API model provide karta hai. App ka interface, sharing, recovery, access frequency aur performance requirement pehle decide karo.

AWS Backup: policy se backup, test se confidence

AWS Backup supported services ke backups ko policies/plans se centrally organise karne mein help karta hai; AWS Organizations ke saath multi-account governance design ki ja sakti hai. Supported resources may include EC2/EBS, RDS/Aurora, DynamoDB, EFS, selected FSx aur Storage Gateway; actual feature/Region support verify karo.

Backup frequency, retention, backup vault, encryption key, cross-account/cross-Region copies aur restore permissions define karo. RPO acceptable data-loss window; RTO acceptable recovery time. Backup successful dikhe tab bhi isolated restore aur application-level integrity test karo. Immutable/isolated backup controls aur access separation ransomware scenario mein important hain.

EBS: EC2 ke liye block volumes

EBS persistent block volumes supported EC2 workloads ke liye deta hai. Volume data AZ ke andar component failure ke against replicated design hoti hai; single-AZ volume ko cross-AZ backup assume mat karo. Instance/AZ placement, volume type, size, IOPS, throughput, encryption aur delete-on-termination settings inspect karo.

Snapshots point-in-time recovery ke liye relevant hain, lekin app consistency aur restore test separate hain. Disk full ho to filesystem/inodes/logs inspect karo. Volume size badhane ke baad OS partition/filesystem expansion bhi lag sakti hai. Provisioned storage aur snapshot charges continue ho sakte hain after instance stop.

Elastic Disaster Recovery: replication se recovery

Elastic Disaster Recovery supported on-premises/cloud source servers ka data selected AWS Region/account ke staging area mein replicate karne aur recovery instances launch karne mein help karta hai. Replication settings, launch configuration, network/IAM aur source compatibility plan karo. Routine non-disruptive recovery drills aur supported failback testing se actual RPO/RTO measure karo.

Historical China/Outposts alternatives ka product support time ke saath change ho sakta hai; current Region, product lifecycle aur migration guidance verify karo. Replication healthy dikhne ka matlab application-level recovery automatically successful nahi.

EFS: shared Linux file system

EFS supported clients ke liye NFS file system interface provide karta hai; multiple EC2 instances shared files access kar sakti hain. Regional and One Zone storage/availability options configuration par depend karte hain; mounted clients ke liye mount targets, security groups, DNS, network aur POSIX permissions configure karo. On-prem access supported connectivity such as VPN/Direct Connect ke through design ki ja sakti hai.

Web content, development, analytics, media, containers aur compatible application storage use cases ho sakte hain. Throughput/performance/storage modes aur workload compatibility test karo. EFS Standard, infrequent access aur Archive jaisi supported classes ke access/retrieval/transition economics verify karo; exact feature and Region support check karo.

Amazon File Cache: dispersed data ke liye fast temporary view

File Cache supported on-prem/cloud NFS aur S3 datasets ko file-based compute workload ke liye high-speed cache view mein access karne ka option hai. Example: analytics job multiple data sources ke shared cached view se data read kare. Cache coherency, source permissions, data residency, bandwidth, eviction, consistency aur service availability evaluate karo. Performance claims workload-specific hain; benchmark karo.

FSx family: right file system choose karo

FSx for Lustre: high-performance parallel filesystem for HPC, ML aur media workflows. Supported S3 data-repository integration se dataset load/process/write-back patterns possible hain; deployment type, data durability, import/export semantics, compute path and cost verify karo. On-prem connectivity via VPN/Direct Connect scenario assess karo.

FSx for NetApp ONTAP: ONTAP-based managed file storage option. Supported NFS, SMB aur iSCSI access, snapshots, compression/deduplication aur enterprise integration evaluate karo. Protocol/version and multi-protocol permissions test karo.

FSx for OpenZFS: OpenZFS-based managed file storage option for compatible Linux/NFS-style workflows. Snapshot, throughput, capacity, network and migration compatibility current docs se verify karo.

FSx for Windows File Server: Windows shared-file workloads ke liye SMB/NTFS, supported Active Directory aur DFS integration options. Home directories, .NET/ERP or CRM workloads evaluate karo. Domain/DNS dependency, security ACLs, backup, availability aur SSD/storage pricing plan karo.

S3: object storage ki detailed journey

S3 bucket objects ko keys ke through store karti hai. Websites, application assets, backups, analytics, IoT data aur archives ke use cases mein access policy, Block Public Access, encryption, retention, versioning aur lifecycle requirements define karo. A prefix folder jaisa dikhta hai, lekin object API/filesystem semantics alag hain. Published design durability figure availability, protection from accidental delete ya successful restore ki guarantee nahi hai.

Storage classes: access pattern aur recovery se choose

• S3 Standard: frequently accessed data ka general-purpose option.

• S3 Intelligent-Tiering: unknown/changing access pattern ke liye supported automatic tiering; monitoring and other charges evaluate karo.

• S3 Express One Zone: supported high-performance one-AZ workloads; AZ failure/residency trade-offs samjho.

• S3 Standard-IA / One Zone-IA: less-frequent access; minimum storage duration, retrieval and one-AZ resilience differences verify karo.

• S3 Glacier Instant Retrieval: archive data jisko supported immediate retrieval chahiye.

• S3 Glacier Flexible Retrieval: infrequent archival access with supported retrieval tiers/time/cost.

• S3 Glacier Deep Archive: long-term archive, restore time typically hours depending on tier; fees and minimum duration check karo.

• S3 on Outposts: supported on-prem storage use-case; supported configuration and residency requirements confirm karo.

Lifecycle policy objects ko supported storage classes mein transition ya expiry manage kar sakti hai. Versioning, noncurrent versions, minimum-duration charges, retrieval request/restore costs aur legal retention carefully configure karo. Transitions ko application/operation needs ke against test karo.

Object Lock aur retention

S3 Object Lock supported versioned objects par write-once-read-many retention/legal-hold controls apply kar sakta hai. Governance aur compliance modes ke permission/retention effects alag hain; changing/deleting protected versions difficult ya impossible ho sakta hai during retention. Production use se pehle legal review aur isolated test karo. Object Lock encryption/key recovery aur backup strategy ka substitute nahi.

S3 incident drill

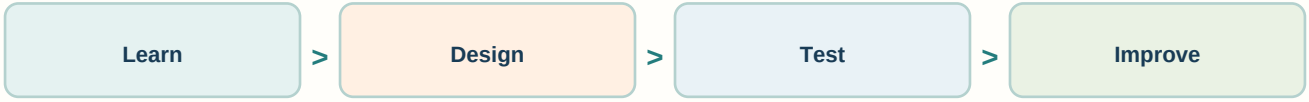
AccessDenied? Principal/account, bucket vs object action/ARN, bucket policy, endpoint policy, explicit deny aur KMS permissions investigate karo. Bucket delete fail? Object versions/delete markers aur retention settings check karo. Public access suspected? Approved incident procedure se contain, investigate aur evidence retain karo.

Storage Gateway: on-prem apps ko cloud storage se connect

Storage Gateway supported VM/hardware appliance ke through hybrid file, volume ya virtual-tape workflows provide karta hai. NFS, SMB, iSCSI jaise supported protocols se applications connect kar sakti hain; gateway types ke backend/integration options S3, archival storage, EBS ya FSx use-cases mein differ karte hain. Local cache active data access improve kar sakta hai; link failure, cache state, bandwidth management aur recovery behavior test karo.

Use cases backup/archive, disaster recovery, tiering aur migration. Connectivity, local appliance power, permissions, encryption, monitoring aur actual stored/cached data costs plan karo.

Learning aur architecture: next steps



Safe hands-on shuruaat

AWS account/Free Tier offers aur eligibility time ke saath change hoti hain; valid payment method, charges aur applicable terms signup se pehle verify karo. Free Tier unlimited free usage ya automatic hard spending cap nahi. Authorised sandbox, MFA, least privilege, budget alerts, resource inventory aur cleanup checklist se shuru karo. AWS Skill Builder, supported self-paced labs, official online talks aur service guides further learning ke starting points hain.

Well-Architected ke six pillars

• Operational excellence: deployment, monitoring, incident response aur continual improvement ko repeatable banao.

• Security: identities, data, network, detection aur incident response protect karo.

• Reliability: failures handle karne, recover karne aur demand meet karne ki design/test capability.

• Performance efficiency: right resource type/size aur evolving workload ke hisaab se performance measure karo.

• Cost optimisation: cost drivers, ownership aur business value ke against waste manage karo.

• Sustainability: resource use aur associated environmental impacts ko workload context mein reduce karne ke opportunities evaluate karo.

Pillars ke beech trade-off hai: more replicas reliability improve kar sakti hain, par cost/resource use badh sakta hai. Workload ki business needs, risk tolerance aur measurements ke basis par decide karo.

Lenses: alag workload, alag questions

Specialised lenses application-specific questions add karte hain. Examples: serverless; container build/images; machine learning; data analytics; hybrid networking; IoT; SAP; games; streaming media; healthcare; financial services; HPC; SaaS; government; connected mobility; migration. Available lenses aur current guidance version check karo. Workload owner, risk, action, priority aur review date record karo.

Final project: online shop ka complete sketch

• DNS and edge → load balancer/API → app across AZs → appropriately protected data store.

• Workforce/application roles → least privilege → KMS/secrets → logging and threat detection.

• Metrics, alarms, incident owner and runbook → backup policies → isolated restore drill.

• Template-based deployment and tested rollback → budgets, tags and cost review.

• Define RPO/RTO, user-facing availability goal, data residency and architectural trade-offs.

This diagram is a learning starting point, not a production-ready reference. Threat model, technical validation, service lifecycle, Region, pricing, quotas aur compliance review ke bina production deploy mat karo.

Beginner glossary: terms ko apni bhasha mein samjho

• API: software ko kisi service se request aur response exchange karne ka defined interface.

• Availability: service kitne samay ya kitni requests ke liye kaam kar rahi hai; durability se alag measure.

• Backup: data ki recovery copy. Restore test ke bina backup ki practical usefulness prove nahi hoti.

• CIDR: IP address range likhne ka notation, jaise 10.0.0.0/16.

• Client aur server: client request bhejta hai; server request handle karke response deta hai. Ek system dono roles le sakta hai.

• DNS: domain name ko network records se map karta hai; DNS lookup aur application availability alag cheezein hain.

• Encryption: readable data ko key ke through protected form mein badalna; permissions aur key management bhi zaroori hain.

• Fault tolerance: component failure ke bawajood system ko specified level par operate karne ki design capability.

• IAM principal: user, role session ya doosri eligible identity jo request karti hai. Policy authorisation decide karne mein help karti hai.

• Idempotency: ek operation ko repeat karne se intended business effect duplicate na ho; retries mein bahut useful.

• Latency aur throughput: latency ek operation ka delay; throughput kisi time interval mein processed work ki quantity.

• Managed service: provider kuch operational tasks handle karta hai; exact shared responsibility har service ke liye alag check karo.

• Multi-AZ: workloads ya services ko multiple Availability Zones mein design karne ka pattern; implementation service-specific hota hai.

• Observability: metrics, logs aur traces se system ke internal behavior ko samajhna; useful telemetry ko configure aur interpret karna hota hai.

• Provisioning: resource ko select aur configure karke use ke liye available banana; delete karna aur cleanup bhi lifecycle ka hissa hai.

• Region / AZ: Region geographical location hai; Availability Zones us Region ke isolated infrastructure locations hain.

• RPO / RTO: acceptable data-loss window / acceptable recovery time; dono ko business se agree karke test karo.

• Scalability / elasticity: capacity adjust karne ki capability / changing demand ke saath capacity adjust karne ka pattern.

• SDK: programming language mein service APIs use karne ki library aur tools.

• Subnet / VPC: VPC isolated virtual network boundary; subnet VPC ka IP range jo ek AZ mein hota hai.

• TLS: network connections ke liye encryption aur peer authentication provide karne wala protocol; certificate validation skip mat karo.

• Vulnerability / threat / risk: exploitable weakness / potential harmful actor ya event / impact aur likelihood ka context-based assessment.

Official learning resources

• AWS Documentation — service setup, limits aur feature details.

• AWS Architecture Center — architecture examples aur design guidance.

• AWS Decision Guides — use case ke hisaab se services compare karne ki shuruaat.

• AWS Well-Architected Framework — architecture review ke pillars aur guidance.

• AWS Glossary — AWS terminology ka official reference.

• AWS Blog aur AWS Skill Builder — further learning aur updates.

Engineering aur practice toolkit

Aage architecture deep dives, selected services ke troubleshooting drills, aur hands-on exercises hain. Har exercise authorised environment mein karo; permissions, charges aur cleanup pehle plan karo.

A1 IAM: authorization reasoning

Trust, permissions, organization controls aur debugging

Request ka decision model

Har API call ke context mein principal, action, resource aur conditions hote hain. Identity policy aur resource policy se permission mil sakti hai; permissions boundaries, session policies aur organization SCPs applicable permissions ko constrain kar sakte hain. Exact combinations principal aur resource policy ke type par depend karti hain; simple union/intersection rule ko har case par apply na karein. Koi applicable explicit Deny normally allow ko override karta hai.

Role trust policy batati hai kaun role assume kare; role permissions policy batati hai role session kya kar sakta hai. External account access mein trusting account ke role trust relationship ke saath caller-side authorization bhi relevant hai. External ID appropriate third-party use cases mein confused deputy risk reduce karta hai; use password samajh kar treat na karein.

Worked failure: EC2 -> encrypted S3

Symptom: app ko GetObject par AccessDenied milta hai. Pehle instance ka intended role aur actual caller identity verify karein. Phir object key/ARN, role permission, bucket policy aur applicable explicit denies inspect karein. Agar object SSE-KMS encrypted hai, KMS key authorization bhi required ho sakti hai; cross-account access mein key policy aur caller permissions dono examine karein. CloudTrail evidence sensitive ho sakta hai: output sanitize karein.

Security compromise na karein: s3:* on * ya public bucket enable karna diagnosis nahi hai. Ek controlled test object, one change at a time, timestamped observations aur rollback use karein.

PRACTICE

Ek team ko sirf reports/ prefix ke objects read karne hain. Alag se ListBucket ki zarurat, prefix condition, KMS key access aur explicit-deny guardrails identify karein. Simulator result ko real request context ke saath compare karein.

Official reference: https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_evaluation-logic.html

A2 VPC: packet-level reasoning

Routes, security groups, return path aur DNS

Route trace

Assume VPC 10.0.0.0/16: public-A 10.0.1.0/24; public-B 10.0.2.0/24; app-A 10.0.11.0/24; app-B 10.0.12.0/24. Subnet ek hi AZ mein hota hai; CIDRs overlap nahi karne chahiye. Subnet route table mein VPC local route internal connectivity support karta hai. Public subnet default IPv4 route 0.0.0.0/0 -> IGW; private app subnet default route -> appropriate NAT gateway, jab internet egress zaruri ho.

NAT gateway public subnet mein hona aur public route via IGW hona chahiye. Per-AZ NAT design zonal failure isolation improve kar sakta hai, par cost badhta hai. S3 gateway endpoint supported Region route tables mein S3 destination ke liye private path de sakta hai; endpoint policy, bucket policy aur IAM still matter. IPv6 routing alag plan karein.

Troubleshooting timeout

Timeout se pehle classify karein: name resolution, TCP connection, TLS handshake ya application response? VPC DNS settings aur resolver check karein; phir source subnet route, destination address, SGs, NACL inbound and outbound/ephemeral ports, listener, target health aur OS service check karein. SG stateful hai; NACL stateless. VPC Flow Logs accepted/rejected traffic evidence de sakte hain, lekin app health ya sab packet-level causes prove nahi karte.

WORKED TRACE

Client 443 -> internet-facing ALB SG -> app SG port 8080 -> database SG port 5432. App SG source ALB SG; DB SG source app SG. Health-check path aur app port mismatch hone par targets unhealthy ho sakte hain. Rules ko exact deployment ke hisaab se verify karein.

Official reference: https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html

A3 DNS, TLS aur content delivery

Name resolution se cache behavior tak

Request journey

Browser cached DNS answer use karega ya resolver se domain ka answer lega. Route 53 hosted zone records DNS answers manage karte hain; record type, TTL aur routing policy behavior ko influence karte hain. DNS update instantly sab clients par apply nahi hota: caches apne TTL aur implementation ke mutabik refresh karte hain. Route 53 health checks aur failover policies design karte samay actual health endpoint ko dependent components se align karein.

HTTPS mein certificate hostname, validity aur trust chain verify hoti hai. TLS termination CloudFront, ALB ya application par ho sakti hai; har segment ke encryption requirements aur certificate management independently plan karein. A valid certificate application ko automatically secure nahi karta.

CloudFront caching

CDN static aur suitable dynamic content ko edge par serve kar sakta hai. Cache key headers, cookies aur query strings ke selection par depend karta hai. Agar personalized response wrong cache key ke saath cache ho jaye to user data leak ho sakta hai. Private content ke liye appropriate origin access controls, authorization, cache policy aur test strategy use karein. Invalidation aur versioned filenames ke cost/operational trade-offs compare karein.

EXERCISE

A product image update ke baad kuch users purani image dekhte hain. DNS TTL aur CDN cache TTL ko alag identify karein; response headers, object versioning, cache key aur invalidation/versioned filename check karein.

Official reference: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Introduction.html

A4 Compute, scaling aur capacity

Demand ka model aur controlled deployment

Capacity sizing example

Suppose measured representative load test mein ek app instance 40 requests/second ko latency target ke andar handle karta hai. Expected peak 200 requests/second aur 25% headroom ke liye initial planning: 200 × 1.25 / 40 = 6.25, yani at least 7 instances. Yeh sirf first approximation hai; CPU, memory, concurrent requests, AZ loss, downstream database aur burst behavior measure karna zaruri hai.

Agar do AZs mein roughly equal load share ho aur one-AZ failure survive karna hai, surviving capacity ko peak traffic support karna chahiye; sirf total seven instances spread kar dena sufficient nahi. Availability targets aur acceptable degraded behavior ke mutabik capacity aur cost calculate karein.

ASG and ALB

Launch template immutable configuration ka baseline hai; AMI updates, IAM instance profile, user data aur SG review karein. ASG min, desired aur max define karein. Target tracking useful hai, lekin warmup, cooldown, noisy metrics aur maximum spending constrain karna hoga. ALB health check liveness signal hai; readiness, application dependencies aur graceful shutdown ka separate design karein.

Spot interruption expect karein; checkpointing, retries aur multi-capacity strategy ke bina critical stateful single-instance workload ko Spot par na depend karein. Compare On-Demand, Savings Plans and Reserved Instances based on the actual commitment terms and utilization; specific discounts assume na karein.

DRILL

Load test ke baad one target stop karke traffic, target health aur replacement observe karein. Lab ka ALB, instances, volumes, public IPs aur monitoring resources delete karke billing dobara check karein.

Official reference: https://docs.aws.amazon.com/autoscaling/ec2/userguide/what-is-amazon-ec2-auto-scaling.html

A5 Storage: data lifecycle aur recovery

Consistency, protection, access pattern and cost

Object design

S3 key namespace applications ke read/write pattern aur security boundaries ke saath plan karein. Versioning accidental overwrite/delete recovery mein madad kar sakti hai; versioned bucket ki cleanup mein versions aur delete markers dono dekhne padte hain. Object Lock jaisi retention choices compliance requirements ke liye useful ho sakti hain, lekin irreversible retention ke consequences samajh kar hi configure karein.

Server-side encryption and KMS key policies protect data at rest, but access authorization, network transport, audit, key availability and backup policies equally important hain. S3 lifecycle transition/expiration rules ko minimum storage duration, request and retrieval charges ke saath evaluate karein; class names se alone cost decide na karein.

Block and shared files

EBS block device hai; filesystem consistency application/OS behavior se affect hoti hai. Snapshot se volume restore ke baad application-consistent recovery test karein, especially databases par. EFS managed shared filesystem hai; Linux clients, mount targets, NFS permissions, throughput mode aur access points design ka part hain. S3, EBS aur EFS interchangeable interfaces nahi hain.

DECISION EXAMPLE

Web assets -> S3; single instance boot disk -> EBS; multiple app instances ke POSIX-style shared files -> EFS. Har option ke liye read/write pattern, recovery objective, encryption, permissions aur current pricing compare karein.

Official reference: https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html

A6 Databases: availability aur data modeling

Relational transactions aur DynamoDB access patterns

RDS design

Database engine choose karte samay transactions, supported features, maintenance, query workload aur version lifecycle consider karein. Multi-AZ DB instance deployment standby maintain kar sakti hai for failover; standby read scaling ke liye general-purpose reader nahi hai. Read replicas ki replication and failure behavior engine-specific hai. Multi-AZ backup ka substitute nahi; deletion, corruption ya malicious write replicas tak bhi propagate ho sakte hain.

Database app subnets se separate private subnets mein rakhein, SGs sirf required app sources se database port allow karein, credentials securely store/rotate karein aur connection pooling monitor karein. Backup retention, point-in-time recovery, restore test, encryption key access aur actual RTO document karein.

DynamoDB access patterns

Pehle questions likhein: get order by ID, list orders for customer by date, find orders by status. Partition key / sort key aur relevant indexes in queries ko efficiently support karne ke liye design karein. A scan large dataset mein expensive ho sakta hai. Hot key, item size, request capacity mode, conditional writes aur consistency requirements ka load test karein.

Example: PK CUSTOMER#123 aur SK ORDER#2026-11-07#456 customer ke orders ko sort-key prefix/time ordering se query karne ka ek possible model hai. Alternate query by order ID ya status ke liye additional index ya different access design chahiye. Transaction, retention aur privacy requirements ke saath design validate karein.

FAILURE ANALYSIS

Orders slow hain: app latency, DB connections, CPU, memory, storage, query plans, locks aur recent deployment ko correlate karein. Randomly instance resize karne se inefficient query ya lock problem solve hona guaranteed nahi.

Official reference: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-modeling-nosql.html

A7 Containers: build se orchestration tak

Docker images, ECS, Fargate aur EKS

Image aur runtime

Image application code, dependencies aur runtime layers package karti hai; container us image ka running process environment hai. Image ko immutable digest se reference karna mutable latest tag se zyada reproducible hota hai. Minimal trusted base image, dependency scanning, non-root user, no baked-in secrets aur regular rebuilds supply-chain hygiene ka hissa hain.

Container stateless rakhne ki koshish karein: durable data managed data store mein, logs stdout/stderr ya configured collector mein, secrets approved secret store se. CPU/memory limits, readiness, graceful termination aur resource cleanup configure karein. Container isolation ko complete security boundary na samjhein.

ECS versus EKS

ECS AWS-native container orchestration hai; task definition container configuration batati hai, service desired task count maintain kar sakti hai. Fargate server management abstract karta hai; EC2 launch type mein host capacity aap manage karte hain. EKS managed Kubernetes control plane provide karta hai; Kubernetes API/ecosystem ka benefit milta hai par cluster configuration, worker choices, workload RBAC, add-ons aur operations ki responsibility rehti hai.

Choose based on team skills, portability constraints, operational budget, workload needs and actual supported features. Small team ke simple workload ke liye managed ECS option operationally simpler ho sakta hai; Kubernetes requirement hone par EKS fit ho sakta hai. Costs: compute, load balancers, networking, logs, registry and EKS control plane where applicable.

PRACTICE

Local container build and run karein; endpoint test karein; image ko non-root configuration ke saath scan karein. Cloud deployment se pehle identity, registry, image provenance, cost aur cleanup plan likhein.

Official reference: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html

A8 Event-driven systems and correctness

Retries, ordering, idempotency aur backpressure

At-least-once implications

Many event-driven integrations duplicates deliver kar sakti hain. Producer/consumer workflow mein stable event ID aur durable processing record use karke idempotency design karein. Example: payment order event pe repeat charge na lagne ke liye payment provider ka supported idempotency key, persisted order state aur atomic conditional update use karein. Exactly-once business outcome individual component ki delivery claim se automatically prove nahi hota.

SQS message consumer ko receive ke baad visibility timeout ke duration ke liye temporarily hide karta hai. Processing complete hone par delete zaruri hai; timeout processing time se chhota hua to concurrent duplicate processing ho sakti hai. Timeout, retries, max receive count, redrive policy aur DLQ monitoring workload ke hisaab se configure karein. Standard/FIFO ki ordering and throughput properties alag hain.

Failure design

Poison message permanently fail hota ho to infinite retry downstream ko harm kar sakti hai. DLQ isolate karne mein useful hai but replay se pehle schema, idempotency aur original root cause review karein. Backoff with jitter retry storms reduce karta hai. Lambda retry behavior invocation mode aur event source mapping ke according alag hota hai; poster ka DLQ automatic nahi hai.

S3 -> Lambda pipeline mein input aur output prefixes separate rakhein, warna output upload se recursive invocation ho sakti hai. Metrics: queue age, depth, failure rate, throttling aur DLQ messages. Event payload sensitive ho sakta hai; logs aur retention mein minimize karein.

TABLETOP

Duplicate event 3 baar aaya; consumer pehle attempt ke baad crash hua. Har step par persisted state kya hai, aur retry ko kaise decide hoga ki business side effect pehle ho chuka hai?

Official reference: https://docs.aws.amazon.com/lambda/latest/dg/invocation-retries.html

A9 IaC aur CI/CD: safe change

From plan to deployment and rollback

Reproducibility

IaC repository mein resources, variables aur environment boundaries define karein. Pull request mein validation, static analysis, policy checks, test evidence aur expected resource changes include karein. CloudFormation change set resource create, modify ya replacement impact dekhne mein help karta hai; replacement se stateful resource data loss ya downtime risk ho sakta hai. Drift detection useful hai, lekin supported properties aur operational limitations verify karein.

Terraform use karte hain to state sensitive ho sakta hai; secure backend, locking where supported, least-privilege access aur recovery plan chahiye. Tool choice se zyada important reproducible changes, version control, review aur state protection hain.

Pipeline gates

Typical sequence: source commit -> unit tests -> build immutable artifact -> dependency/secret scan -> infrastructure plan -> review -> deploy test environment -> integration tests -> approved production deployment -> health checks -> monitor. Build once/promote same artifact prevents environment-specific rebuild drift. Pipeline identity ko scoped role dein; long-lived credentials repository mein na rakhein.

Rollback always simple reverse deployment nahi hai: database schema change irreversible ho sakti hai. Expand-contract migration use karein: backward-compatible field add, dual-compatible application deploy, migrate data and verify, then remove old field in later release. Rollback trigger and database compatibility before rollout define karein.

RELEASE DRILL

Canary deploy 5% traffic se start karein only if infrastructure supports it. Compare error rate, latency and business outcome with baseline; increase or rollback based on pre-agreed thresholds. 5% is illustrative, not a universal best practice.

Official reference:
https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/using-cfn-updating-stacks-changesets.html

A10 Reliability: SLO, RPO aur RTO

Quantify uncertainty and recovery objectives

Availability math

Monthly availability target 99.9% ka error budget = 0.1% of time. Illustrative 30-day month: 30 × 24 × 60 × 0.001 = 43.2 minutes. 99.99% ke liye 4.32 minutes. Yeh downtime budget sirf time-based availability ke simple model mein hai; request-based SLI aur excluded events ki definition alag ho sakti hai.

Agar strictly independent serial components ki availability A1 and A2 ho, combined availability approximately A1 × A2 hoti hai. Real failures correlated ho sakte hain, isliye multiplication alone production assurance nahi hai. Shared database, DNS, deployment aur operator actions common failure domains ban sakte hain.

Recovery planning

RPO maximum acceptable data loss measured in time hai; RTO target restoration time. Example RPO 15 minutes and RTO 2 hours: backup/replay strategy ko data loss target meet karna hoga, restore procedure ko tested recovery duration meet karna hoga. Measured restore includes discovery, access, provisioning, data restore, validation and traffic recovery, not just snapshot completion.

Strategies from backup-and-restore to warm standby or multi-site differ in cost and achievable recovery. Cross-account and cross-Region copies, encryption key recovery and least-privileged recovery roles evaluate karein. Regular restore drills and evidence are stronger than a green backup-job status.

EXERCISE

Checkout SLI define karein: successful completed orders / valid checkout attempts, measurement window aur exclusions. 99.9% target par 1 million eligible attempts mein indicative error budget 1,000 failed attempts hoga. Define how to handle dependency outages and double counting.

Official reference: https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/welcome.html

A11 Security operations and governance

Detect, contain, recover, and learn

Multiple accounts aur guardrails

Organization-level account separation production, non-production, security and logging boundaries improve kar sakti hai. SCPs maximum available permissions par guardrails laga sakte hain; SCP khud IAM permission grant nahi karta. Member accounts, root access management, break-glass procedure, log protection and delegation ka operational model define karein.

CloudTrail management events AWS API activity provide karte hain; specific data events aur configurations additional selection/cost require kar sakti hain. GuardDuty threat detection, Security Hub findings aggregation aur Config resource configuration monitoring ke use cases alag hain. In services ko enabled dikhna sufficient nahi: findings triage, owner aur response SLA bhi chahiye.

Compromised credentials runbook

1. Incident process activate; preserve evidence and timeline. 2. Identify principal, scope, active sessions, impacted accounts and API activity. 3. Contain using approved identity/session restrictions and network controls, considering business impact; rotate or revoke exposed credentials as appropriate. 4. Investigate unauthorized resources/data access; remediate root cause. 5. Recover, verify and document follow-up. Exact steps incident type and organizational process se adapt karein.

Secret ko chat, log ya ticket mein paste na karein. Evidence store ko restrict karein. Encrypted resources par key disable/delete karne se outage ya permanent data loss ho sakta hai; emergency response decisions authorized owners ke saath coordinate karein.

THREAT MODEL

Asset: order database. Threat: stolen application role credentials. Controls: least privilege, short-lived credentials, network restrictions, audit, anomaly detection, incident runbook and tested restore. Har control ke failure mode aur owner identify karein.

Official reference: https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html

A12 FinOps: measurable cloud cost

Unit economics, budgets aur hidden charges

Cost model

Illustrative monthly estimate = compute + storage + database + load balancing + NAT processing/hour + requests + data transfer + logs/monitoring + backup + support/other applicable charges. Har item ke quantities, unit price, Region aur assumptions likhein. Yeh equation tax, discount aur every service-specific charge automatically include nahi karti. Calculator aur pricing pages se current rates verify karein.

Worked fictitious example: compute 120 + DB 100 + storage 25 + network 35 + observability 20 = 300 units/month. Agar 100,000 successful orders hon, indicative infrastructure cost per order = 300/100,000 = 0.003 units; fixed cost allocation, shared services, currency, tax aur failed orders define kiye bina yeh business profit metric nahi hai.

Optimization loop

Tagging and cost allocation se spend owner/workload ke against map karein. First establish baseline, then rightsizing, idle cleanup, storage lifecycle, data transfer path and commitments evaluate karein. Performance/reliability ko cost ke liye sacrifice karne se business impact zyada ho sakta hai. Budget alert warning hai, spending hard limit nahi; alarms late aa sakte hain, and separate governance is needed.

LAB CLEANUP CHECKLIST

Terminate instances; inspect EBS volumes/snapshots, load balancers, NAT gateways, public IPv4 addresses, database instances, buckets including all versions, endpoints and log groups. Delete only lab-owned resources. Billing later revisit karein.

Official reference: https://calculator.aws/

A13 Capstone engineering blueprint

One application; security, delivery, operations and recovery together

Scenario and assumptions

Scenario: online shop with catalog, checkout and order history. Example nonbinding targets: availability 99.9%, RPO 15 minutes, RTO 2 hours. Record workload profile, concurrency, average/peak request rate, protected data, jurisdictions and budget with product owners. Validate targets against actual cost and business risk.

Architecture: Route 53 and optional CloudFront -> internet-facing ALB across two public subnets -> stateless application capacity in two private app subnets -> private relational database Multi-AZ deployment; product images in private S3. IAM roles give app narrowly scoped access; secrets in managed secret store. Diagram is conceptual: routes, endpoints, TLS, encryption, database placement and logging need detailed implementation.

Acceptance and failure tests

Functional: browse and checkout; double-submit order does not create duplicate charge. Security: unauthorized principal denied; no public database; secrets absent from repo. Reliability: one app instance failure does not break target SLO under tested load; database failover and backup restore tested in controlled environment. Delivery: new version can be rolled back without breaking schema. Operations: dashboards, alert owner and runbook exist; cost estimates reviewed.

Execution phases

Phase 1: build threat model and diagrams, define measurable targets and cost estimate. Phase 2: deploy isolated dev account foundations through IaC. Phase 3: deploy app, data and delivery pipeline. Phase 4: test functionality, least privilege, failover, restores and alerts; capture evidence. Phase 5: clean up disposable resources; record actual cost and retrospective. This is a blueprint, not a claimed deployed system.

PORTFOLIO DELIVERABLES

Architecture diagram, assumptions, IaC repository with no secrets, test cases and sanitized results, cost estimate versus actual, risk register, runbook and lessons learned.

Official reference: https://aws.amazon.com/well-architected/

A14 Self-assessment with reasoning

Practice questions before interviews or project reviews

Why can a private instance not reach S3?

Check DNS and routing first; gateway endpoint association, endpoint policy, bucket policy, IAM permissions and KMS where applicable. NAT is not the only possible path.

Why are two app instances not always highly available?

Both may share one AZ, one database, a single deployment pipeline or another common dependency. Model and test failure domains.

Why did an SQS event process twice?

Visibility timeout expiry, consumer crash before deletion or service delivery semantics can produce duplicates. Design idempotency and appropriate timeout.

Is 99.99% availability guaranteed by a Multi-AZ database?

No. Entire user journey, common-mode failures, change process and measurement define application availability.

Does encryption prevent an authorized identity from reading data?

No. Encryption protects data under its key and access model; a permitted principal can decrypt. IAM, key policies, monitoring and least privilege remain essential.

Can a cost alert stop all charges?

No. Budgets alert; separate action, guardrails and resource cleanup are needed. Billing delays and non-compute charges still matter.

What proves a backup works?

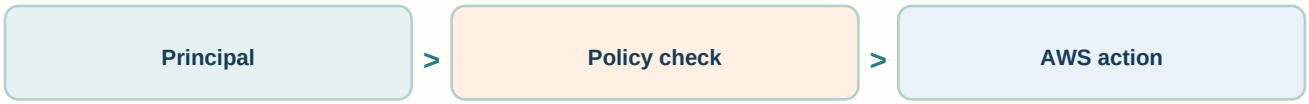
A successful test restore plus application-level data validation and recorded recovery time, not backup status alone.

NEXT STEP

Pick one weak answer; draw the request or failure path, reproduce safely in a sandbox where possible, then compare observations with official documentation.

C01 IAM

Identity, policy evaluation aur temporary credentials



1. Mental model — kya aur kyun?

Principal identity request bhejti hai: user, assumed-role session ya federated identity. Action, resource aur context keys request define karte hain. Identity-based permissions role ya user se attach ho sakti hain; supported services resource policies bhi provide karte hain. Role trust policy alag hai: woh assume karne wale principals define karti hai, actual work permissions define nahi karti.

2. Configuration and design

Workforce ke liye federation/Identity Center aur short-lived credentials prefer karein. Application ke liye narrowly scoped role; Effect, Action, Resource aur conditions review karein. Applicable explicit deny overrides allow. SCPs, session policies, boundaries and resource policies ka interaction principal type aur request ke according evaluate karein; simple universal formula assume na karein.

3. Failures and troubleshooting

AccessDenied mein actual caller identity, target account, ARN, action, region, resource policy, conditions, KMS access aur organization controls inspect karein. Permission broaden karne ke bajay one controlled hypothesis test karein. Role assume fail ho to trust relationship aur caller-side permission separately examine karein.

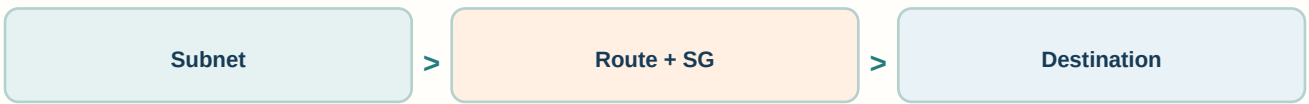
4. PRACTICE / DESIGN DRILL

Paper drill: ek app ko reports/ prefix ke objects read karne hain. GetObject aur ListBucket ke resource types identify karein, trust policy alag likhein, aur explicit deny scenario reason karein. Simulator ke baad controlled real test only if authorized; test role/policy remove karein.

Official service guide: AWS Identity and Access Management

C02 VPC

Subnets, routes, endpoints aur packet journey



1. Mental model — kya aur kyun?

VPC Region-scoped private network hai. Subnet ek AZ mein hota hai aur route table destination-to-target mapping define karti hai. Public subnet mein IGW route hota hai; direct IPv4 internet connectivity ke liye appropriate public IP aur rules bhi chahiye. Private subnet default route NAT ki taraf ho sakta hai for outbound connections.

2. Configuration and design

Security groups stateful allow rules hain; NACL subnet-level stateless allow/deny controls hain. S3 aur DynamoDB gateway endpoints relevant route tables use karte hain; interface endpoints private network interfaces aur security groups use kar sakte hain. Endpoint policy, service resource policy aur IAM all still matter. NAT gateway security group attach nahi karta.

3. Failures and troubleshooting

Timeout debug: DNS resolution, selected route, SG, NACL return traffic/ephemeral ports, destination listener, OS firewall aur app logs sequentially examine karein. DNS and routing ke change se pehle expected packet path aur rollback record karein. VPC Flow Logs network evidence dete hain, full application diagnosis nahi.

4. PRACTICE / DESIGN DRILL

Draw 10.0.0.0/16 with two AZs, one public and two private subnets per AZ; label IGW, per-AZ NAT if required, ALB, app and DB. Routes aur security-group source/destination likhein. NAT and endpoints ke charges calculate karein; diagram-only practice zero AWS resources create karti hai.

Official service guide: Amazon Virtual Private Cloud

C03 EC2

Instance lifecycle, image, access aur operations

1. Mental model — kya aur kyun?

EC2 instance AMI se boot hota hai; instance type compute/memory/network characteristics define karta hai. Root disk commonly EBS-backed hoti hai. Stop, start aur terminate alag actions hain: retained volumes, snapshots, addresses aur associated services continue charge kar sakte hain. Termination settings data deletion ko affect kar sakti hain.

2. Configuration and design

Choose workload-compatible architecture, AMI provenance, subnet, SG, IAM instance profile, storage and monitoring. Instance metadata access ko current security guidance ke mutabik configure karein. Systems Manager Session Manager ke liye managed-node, IAM, agent and network prerequisites check karein; blindly public SSH enable na karein.

3. Failures and troubleshooting

Instance unreachable: account/Region and state confirm; status checks, route, SG, access method, agent, boot logs and storage examine karein. High CPU: app process, load profile and recent change correlate karein. Spot can be interrupted; checkpointing and replacement strategy rakhein.

4. PRACTICE / DESIGN DRILL

Authorized sandbox mein temporary low-cost instance choose karein; before launch price and access method verify karein. uname, df -h and process inspection karein; terminate karein, associated EBS/public IP/snapshots review karein aur later billing inspect karein. AWS Free Tier eligibility assume na karein.

Official service guide: Amazon Elastic Compute Cloud

C04 EBS

Block storage, lifecycle aur point-in-time snapshots

1. Mental model — kya aur kyun?

EBS EC2 workloads ke liye block storage deta hai. Volume filesystem ke andar mounted ho sakta hai; volume size aur filesystem size alag concepts hain. Volume and EC2 placement AZ rules apply karte hain. Different volume types performance and cost characteristics ke saath aate hain; actual workload IOPS, throughput and latency measure karein.

2. Configuration and design

Encryption at rest, key access, delete-on-termination, snapshots and restore procedure launch se pehle decide karein. Snapshot point-in-time recovery ka component hai, lekin application consistency ke liye database/OS coordination aur restore tests important hain. Snapshot policy and retention protected data ke mutabik configure karein.

3. Failures and troubleshooting

Disk full hone par filesystem usage, inode usage, logs and deleted-but-open files ko investigate karein. Volume expansion ke baad OS partition/filesystem expansion separately required ho sakti hai; commands OS and filesystem-specific hain. Recovery ke time key permissions aur new volume placement validate karein.

4. PRACTICE / DESIGN DRILL

Lab on disposable nonproduction data: record volume size, snapshot, restore to new volume in correct AZ, mount using OS-specific procedure, verify file checksum. Cleanup: restored volume, snapshot and instance resources remove; billing revisit karein.

Official service guide: Amazon Elastic Block Store

C05 S3

Objects, authorization, versioning aur lifecycle

1. Mental model — kya aur kyun?

S3 mein bucket objects store karti hai; object key aur optional version ke through address hota hai. Prefixes familiar folder-like organization dete hain, but applications ko object API semantics samajhni chahiye. Public-access settings, IAM policy and bucket policy jointly effective access ko influence karte hain.

2. Configuration and design

New objects encrypted at rest by default, but encryption type, KMS key access and organizational requirements ko explicitly verify karein. Versioning overwrite/delete recovery mein help karti hai. Lifecycle policies current and noncurrent versions ke transitions/expiration configure kar sakti hain; retention, minimum duration and request/retrieval charges assess karein.

3. Failures and troubleshooting

AccessDenied troubleshoot: principal, bucket/key, ListBucket vs GetObject resource type, bucket policy, endpoint policy, explicit deny, encryption/key permissions and organization constraints check karein. Bucket deletion fail ho to all versions and delete markers identify karein. Presigned URL ko bearer credential jaise handle karein.

4. PRACTICE / DESIGN DRILL

Dummy file upload karein, access verify karein, versioning ke saath second version upload aur restore observation karein. Test complete par all object versions and delete markers delete, then bucket delete. Check storage/request charges; retention controls ka unintended use na karein.

Official service guide: Amazon Simple Storage Service

C06 EFS

Shared file system, mounts aur permissions

1. Mental model — kya aur kyun?

EFS managed file storage hai jo supported clients ke saath NFS access patterns support karta hai. Multiple instances common filesystem share kar sakti hain; application locking, permissions aur concurrent writes ke consequences phir bhi application-specific hain. S3 object API aur EFS POSIX-style filesystem same interface nahi hain.

2. Configuration and design

Mount targets, VPC DNS and security group rules required network path create karte hain. Availability and performance mode, throughput mode, access points, encryption and backup plan actual workload ke liye evaluate karein. Access points identity/path management mein help kar sakte hain; POSIX permissions aur IAM authorization ki configuration verify karein.

3. Failures and troubleshooting

Mount fail: mount target health, AZ/VPC connectivity, DNS resolution, NFS port 2049 security groups and network ACLs check karein. Writes fail par mount mode, filesystem POSIX permissions, access point configuration and client user identity inspect karein. Broad NFS access se pehle source scope confirm karein.

4. PRACTICE / DESIGN DRILL

Design drill: two application servers across AZs read a shared uploads directory. Draw mount targets, client SG, filesystem SG and backup boundary. Deployment before cost review; cleanup file system and mount targets when authorized.

Official service guide: Amazon Elastic File System

C07 RDS

Managed relational databases and recovery

1. Mental model — kya aur kyun?

RDS relational database engines ke liye managed provisioning and operations support karta hai; supported engines and versions differ. Aap schema, queries, application credentials, authorization and data classification ke liye responsible rehte hain. Multi-AZ DB instance standby failover improve karta hai; yeh backup ya ordinary read scaling ka direct substitute nahi.

2. Configuration and design

DB subnet group across suitable AZs, private networking, SG source, encryption, secrets, backup retention and maintenance windows specify karein. Automated backup retention defaults creation path ke hisaab se differ kar sakte hain; explicit value set karke verify karein. Read replicas ka consistency and failover behavior engine-specific hai.

3. Failures and troubleshooting

Connection failure: endpoint, port, DNS, SG, database user, secret and connection limits check karein. Slow workload: CPU, memory, I/O, locks, connection count and query plans correlate karein. Credentials logs mein expose na karein. Instance resize symptom ko temporarily hide kar sakta hai without fixing query.

4. PRACTICE / DESIGN DRILL

Recovery drill: specify RPO/RTO, restore backup to isolated instance, connect with authorized identity, validate application-level data and record elapsed time. Delete restored instance and snapshots only when appropriate; charges and retention review karein.

Official service guide: Amazon Relational Database Service

C08 DynamoDB

Key-based access, consistency and workload design

1. Mental model — kya aur kyun?

DynamoDB data items key-based access patterns ke around design hote hain. Partition key and optional sort key query patterns define karte hain. Secondary indexes alternate queries ke liye use ho sakte hain; design se pehle application ke exact access patterns and expected load likhein.

2. Configuration and design

Example PK CUSTOMER#123, SK ORDER#timestamp#id se customer order history query model ho sakta hai. Order lookup by a different key ke liye separate index/model chahiye. Capacity mode, item size, partition distribution, conditional writes, TTL behavior and data retention evaluate karein. TTL deletion exact immediate timer nahi samjhein.

3. Failures and troubleshooting

Throttling par workload distribution, hot keys, consumed capacity, retries and relevant limits check karein. Duplicate event processing ke liye conditional write use ho sakti hai, lekin end-to-end side effect idempotency separately design karein. Read consistency options and index behavior latest service docs se verify karein.

4. PRACTICE / DESIGN DRILL

Paper drill: list five queries, specify table/index key for each, and identify one query requiring index. Estimate request patterns and sensitive data fields. If live test, table, backups and streams cleanup verify; current pricing check karein.

Official service guide: Amazon DynamoDB

C09 Lambda

Invocation models, execution role and failure handling

1. Mental model — kya aur kyun?

Lambda event-driven code execution platform hai; function code ke saath runtime, memory, timeout, configuration and execution role define hote hain. Invocation modes and event sources ke retry, batching and failure behaviors alag hote hain. Stateless/idempotent handler design repeated invocations se unintended side effects reduce karta hai.

2. Configuration and design

Execution role sirf necessary downstream permissions de; secrets secure store se retrieve karein and sensitive values log na karein. Timeout, memory, concurrency, dependency packaging and VPC networking workload ke according test karein. Network-attached function ko public subnet mein rakhne se automatic public internet access assume na karein.

3. Failures and troubleshooting

API timeout ya error investigate: CloudWatch logs, duration, errors, throttling, downstream dependency, input event and permission context. SQS event source mapping batches poll karti hai; visibility timeout, batch failures, retry and DLQ configuration source-specific hain. DLQ automatic enabled nahi hoti.

4. PRACTICE / DESIGN DRILL

Example image workflow: distinct input/output prefixes use karein taaki generated output se recursive trigger na ho. Test same event twice with a stable idempotency key. Delete function, event mappings, queues, buckets, versions and log groups if created for lab.

Official service guide: AWS Lambda

C10 SQS

Queue, visibility timeout and dead-letter handling

1. Mental model — kya aur kyun?

SQS producer aur consumer ke beech buffer provide karta hai. Receive karne par message visibility timeout ke duration ke liye temporarily hidden hota hai; successful processing ke baad message delete karna hota hai. Standard queue duplicate delivery aur out-of-order behavior ke liye application design karein; FIFO queue ki ordering and deduplication semantics alag hain.

2. Configuration and design

Visibility timeout processing duration aur retries se align karein. Dead-letter queue and redrive policy poison messages isolate karne mein help karte hain; max receive count, retention and alarms configure karein. Encryption, queue policy, IAM access, message size and data sensitivity review karein.

3. Failures and troubleshooting

Queue backlog: oldest-message age, visible/in-flight counts, consumer throughput, errors and downstream limits evaluate karein. Replaying DLQ se duplicate side effects ho sakte hain; root cause fix and idempotency before redrive. Lambda integration par batch and partial-failure configuration specific documentation se validate karein.

4. PRACTICE / DESIGN DRILL

Scenario: worker database commit ke baad delete se pehle crash hota hai. Message fir aa sakta hai; stable message business identifier aur conditional state ensure karein ki duplicate order na bane. Is scenario ko table-top mein trace karein.

Official service guide: Amazon Simple Queue Service

C11 SNS and EventBridge

Fan-out, routing and event contracts

1. Mental model — kya aur kyun?

SNS publish/subscribe notification pattern provide karta hai; subscribers supported endpoints par messages receive kar sakte hain. EventBridge event buses and rules events ko matching targets tak route kar sakte hain. Dono ke delivery models, filtering, retry, targets and pricing interchangeable assume na karein.

2. Configuration and design

Event contract mein event ID, schema version, source, event type and occurrence time specify karein. Consumers ko unknown optional fields tolerate karne aur schema evolution handle karne ka plan dein. Sensitive payload minimize karein; target IAM permissions and encryption settings review karein.

3. Failures and troubleshooting

No delivery par producer success, rule pattern/filter, target configuration, permissions, retries and destination error metrics dekhein. Event rules directly business transaction completion prove nahi karte. Multiple subscribers se duplicate/reordered events aur independent failures possible hain.

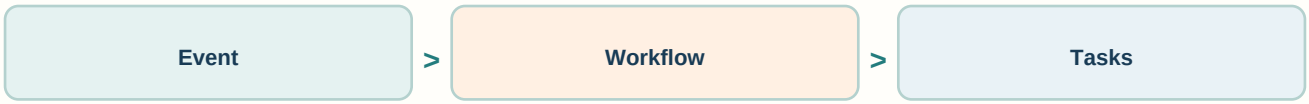
4. PRACTICE / DESIGN DRILL

Drill: OrderPlaced event se email notification aur analytics process trigger design karein. Email failure se order transaction rollback hona chahiye ya nahi? Retries, deduplication, DLQ/destination and data retention requirements document karein.

Official service guide: Amazon EventBridge

C12 Step Functions

Workflow states, retries and compensation



1. Mental model — kya aur kyun?

Step Functions distributed workflows ko state machine model ke through coordinate karta hai. Workflow tasks, choices, waits, retries and error handling define karte hain. Standard and Express workflow execution characteristics differ; actual duration, throughput, history, delivery behavior and pricing current docs se select karein.

2. Configuration and design

Every task ke timeouts, error taxonomy, retries with backoff and catch/compensation logic define karein. Execution role least privilege ho. Workflow state may contain sensitive data; payload size limits, logging and retention controls verify karein. State machine successful ho tab bhi downstream business outcome validation important ho sakta hai.

3. Failures and troubleshooting

Long-running checkout: reserve inventory -> charge payment -> request shipment. Payment failure par release inventory compensation; shipment failure par refund decision. Refund itself fail ho sakta hai, isliye incident queue/manual review and idempotent actions required hain. Saga distributed ACID transaction nahi hota.

4. PRACTICE / DESIGN DRILL

Draw success path, timeout path and compensation failure path. Record state after each side effect and idempotency key. Deploy only after pricing/review; delete state machine and downstream test resources after approved lab.

Official service guide: AWS Step Functions

C13 API Gateway

API entry, authentication and integration

1. Mental model — kya aur kyun?

API Gateway APIs ko publish and manage karne ke options deta hai. HTTP APIs, REST APIs and WebSocket APIs ki feature sets and pricing differ; requirements se type choose karein. Integration Lambda, HTTP backend ya other supported targets se ho sakti hai.

2. Configuration and design

Authentication and authorization model, request validation, throttling, CORS, TLS, response/error format and observability design karein. CORS browser behavior hai, API authorization ka replacement nahi. API key ko secure user identity ka universal replacement na samjhein. Resource policies and authorizers supported configuration ke hisaab se evaluate karein.

3. Failures and troubleshooting

403: caller identity, authorizer, policy, stage and route settings investigate. 5xx/timeouts: integration status, backend logs, permissions, payload and timeout limits assess. Throttling par traffic spikes and quota settings inspect karein; indiscriminate retries outage badha sakte hain.

4. PRACTICE / DESIGN DRILL

Contract exercise: POST /orders idempotency key, request validation, status code, error body and authorization define karein. Test unauthorized, invalid payload, duplicate submission and backend timeout; no real customer/payment data use karein.

Official service guide: Amazon API Gateway

C14 ALB and Auto Scaling

Request routing, health checks and capacity

1. Mental model — kya aur kyun?

Application Load Balancer HTTP/HTTPS listener rules se requests target groups ko route karta hai. Targets ka health check path, port and expected response correct hona chahiye. Auto Scaling group desired capacity maintain aur scaling policies ke basis par capacity adjust kar sakta hai.

2. Configuration and design

Min, desired and max capacity, launch template, AZ distribution, instance warmup and health-check configuration define karein. ALB SG incoming listener port allow kare; app SG ALB SG se app port allow kare. Graceful termination and connection draining consider karein. Availability assumes app and database dependencies bhi appropriately resilient hon.

3. Failures and troubleshooting

Unhealthy targets: correct registered targets, subnet routes, target SG, process listening port, health path and response codes inspect karein. Scaling fails: launch template/AMI/role, subnet IP availability, quota, health checks and scaling activity investigate karein. Data-layer bottleneck ho to extra app instances alone solve nahi karenge.

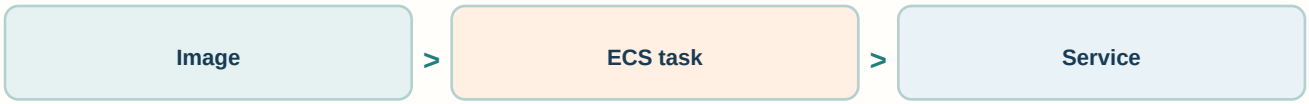
4. PRACTICE / DESIGN DRILL

Capacity example: measured 40 requests/s per instance, peak 200 requests/s and 25% headroom gives ceil(200*1.25/40)=7 instances as initial approximation. AZ loss, uneven distribution, warmup and load testing ke liye extra evaluation zaruri hai.

Official service guide: Amazon EC2 Auto Scaling

C15 ECS and Fargate

Tasks, services and container lifecycle



1. Mental model — kya aur kyun?

ECS task definition application container configuration describe karti hai; service desired tasks maintain kar sakti hai. Fargate serverless compute option hai for supported container workloads; EC2 capacity option mein underlying hosts manage karne padte hain. Cluster, task, service and task definition alag concepts hain.

2. Configuration and design

Image provenance, pinned digest, task CPU/memory, IAM task role, execution role, network mode, logs and secrets define karein. Task role app ke AWS API permissions ke liye; execution role selected task-launch operations ke liye relevant hota hai. Image pull networking and registry access prerequisites verify karein.

3. Failures and troubleshooting

Task fail to start: stopped reason, image URI/permissions, network routes, CPU/memory compatibility, secrets access and logs check karein. Target unhealthy: container port, application listener, load balancer target group and health checks inspect karein. Scaling application dependency limits ko ignore na kare.

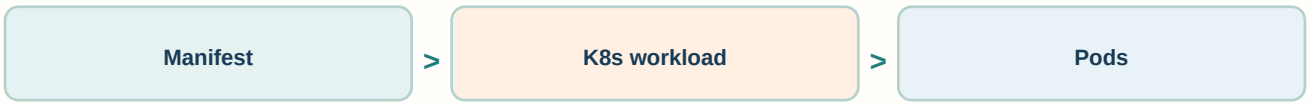
4. PRACTICE / DESIGN DRILL

Create deployment plan with two tasks across failure domains, immutable image digest, health checks, rollback and budget. Live lab requires costs and permissions review; stop service/delete cluster resources, ALB and logs after test.

Official service guide: Amazon Elastic Container Service

C16 EKS

Kubernetes control plane and shared responsibilities



1. Mental model — kya aur kyun?

EKS managed Kubernetes service hai. Kubernetes workload primitives include pods, deployments, services and namespaces; control plane managed hone se cluster workload, permissions, add-ons, networking and operations ki saari responsibility automatically disappear nahi hoti. Worker capacity options alag operational trade-offs rakhte hain.

2. Configuration and design

Cluster IAM integration, Kubernetes RBAC, least-privilege service accounts, network policies where supported, secrets handling, image supply chain and version upgrades plan karein. Pod requests/limits and disruption budgets capacity and rollout behavior impact karte hain. Exact add-on compatibility and upgrade paths docs se verify karein.

3. Failures and troubleshooting

Pods pending: scheduling events, CPU/memory, node capacity, taints/tolerations and network IP availability. CrashLoop: container logs, configuration, probes and dependency failures. Cluster available hone ka matlab application healthy hona nahi hai. Logging, control plane and worker cost explicitly estimate karein.

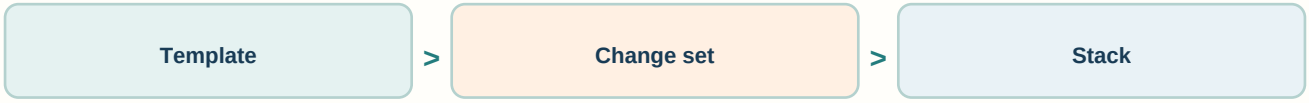
4. PRACTICE / DESIGN DRILL

Exercise: two-replica deployment with readiness/liveness probes, restricted permissions and rolling update. Note failure domains and how application handles a pod restart. Production-grade cluster deployment needs separate security review and cost approval.

Official service guide: Amazon Elastic Kubernetes Service

C17 CloudFormation

Templates, change sets and safe updates



1. Mental model — kya aur kyun?

CloudFormation templates AWS resources ko declaratively describe karti hain; deployed collection stack kehlata hai. Parameters and outputs reusable configuration support karte hain. Resource update behavior can be in-place, interruption-causing or replacement; exact resource property documentation review karein.

2. Configuration and design

Changes Git review aur template validation se pass karein. Change set proposed changes inspect karne mein help karta hai; changeset review destructive replacement ke risk ko eliminate nahi karta. State-bearing resources ke deletion policies, backups, dependencies and rollback documented hone chahiye.

3. Failures and troubleshooting

Stack failure: events ka earliest meaningful error, IAM permissions, quotas, invalid properties, dependency and service status check karein. Drift detection supported resource properties ke scope ke andar out-of-band changes identify karne mein help kar sakta hai; complete configuration audit assume na karein.

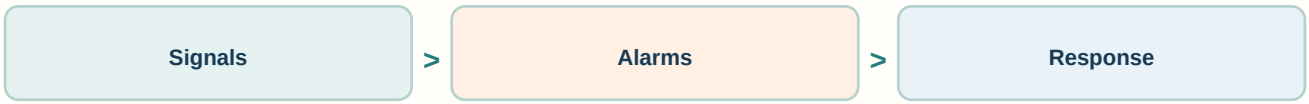
4. PRACTICE / DESIGN DRILL

Use the existing private bucket template in this handbook. In a permitted test account, review template and change set, deploy only after cost estimate; remove object versions/delete markers and verify stack deletion. Do not use real secrets in templates or outputs.

Official service guide: AWS CloudFormation

C18 CloudWatch

Metrics, logs, alarms and dashboards



1. Mental model — kya aur kyun?

CloudWatch metrics numeric measurements hain; logs event records provide karte hain; alarms specified metric conditions evaluate karke configured actions kar sakte hain. Metric namespace, dimensions, aggregation, period and missing-data behavior alert meaning ko influence karte hain.

2. Configuration and design

User-facing availability, error rate, latency and saturation se meaningful SLIs choose karein. Application logs mein correlation ID add karein, but tokens, passwords and unnecessary personal data remove karein. Log retention, encryption, access controls, ingestion/query cost and dashboard ownership define karein.

3. Failures and troubleshooting

Alert fires: confirm actual impact, compare baseline, inspect associated logs and recent deployments. Missing metrics ko zero assume na karein; alarm missing-data configuration inspect karein. CloudTrail management events AWS API activity record karte hain; CloudWatch application monitoring ka replacement nahi.

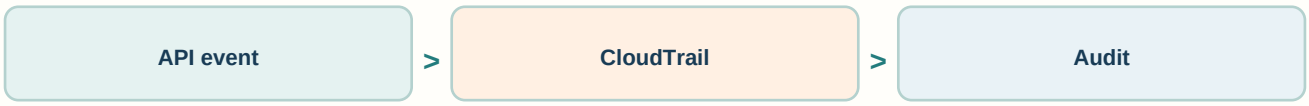
4. PRACTICE / DESIGN DRILL

Exercise: /health check and checkout success rate define karein; one actionable alarm with threshold, owner, severity and runbook likhein. On a test account, delete created alarms, dashboards and log groups after lab and review charges.

Official service guide: Amazon CloudWatch

C19 CloudTrail and Config

Audit trail versus configuration evaluation



1. Mental model — kya aur kyun?

CloudTrail AWS API/account activity ke audit evidence ke liye useful hai. Management events and selected data events ki scope, collection configuration and cost different ho sakti hai. AWS Config supported resources ki configuration history and compliance rules evaluate karta hai. Dono ke data types and purpose alag hain.

2. Configuration and design

Organization-wide collection, centralized log storage, log integrity validation, encryption, access segregation and retention policy plan karein. Data event logging selectively configure karte samay sensitive resource coverage and costs balance karein. Neither service alone provides a complete security program.

3. Failures and troubleshooting

Who deleted resource? CloudTrail event time, caller identity, source, API action, account, Region and request parameters inspect karein. Why is resource noncompliant? Config rule evaluation, resource configuration, recording and rule scope inspect karein. Logs mein private account identifiers ko redact karein.

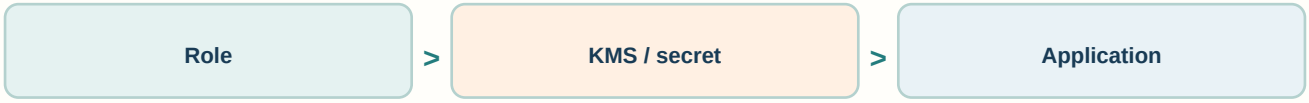
4. PRACTICE / DESIGN DRILL

Tabletop: S3 public-access configuration unexpectedly changed. Trace actor and time in CloudTrail, review resource configuration and approved policy, contain change, then document corrective action. Actual tests only in authorized sandbox.

Official service guide: AWS CloudTrail

C20 KMS and Secrets Manager

Encryption keys, secrets and access boundaries



1. Mental model — kya aur kyun?

KMS cryptographic keys create/control karne ke managed capabilities provide karta hai; services KMS-integrated encryption use kar sakte hain. Secrets Manager credentials and other secrets store/retrieve/rotate workflows support karta hai. Encryption key and application password same concept nahi hain.

2. Configuration and design

Key policy, IAM permissions, grants and service integration ke effect ko request context ke saath evaluate karein. Data encryption at rest alone authorized misuse prevent nahi karti. Rotation, backup/restore permissions, key availability, secret caching and credentials rotation failure handling plan karein.

3. Failures and troubleshooting

SSE-KMS object read fails: S3 authorization ke saath relevant KMS key access, key policy, conditions and cross-account behavior inspect karein. KMS key disable/delete ya secret rotation without dependency analysis outage kar sakti hai. Secrets logs, prompt, ticket ya repository mein paste na karein.

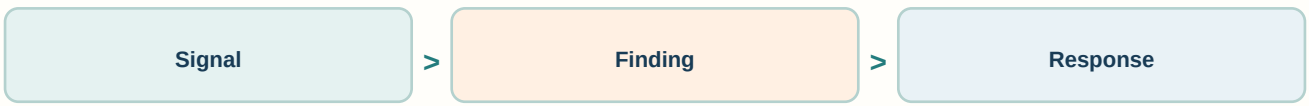
4. PRACTICE / DESIGN DRILL

Design exercise: app role reads one secret and decrypts data only for one workload. Map role policy, secret policy if applicable, key policy, audit trail and break-glass recovery. Do not perform destructive key operations as a beginner lab.

Official service guide: AWS Key Management Service

C21 Security services

Detect, prioritize, investigate and respond



1. Mental model — kya aur kyun?

GuardDuty managed threat detection, Inspector vulnerability assessments, Macie sensitive-data discovery, Security Hub findings aggregation aur AWS WAF web request filtering ke distinct use cases hain. Service enabled hone se automatically secure environment ya complete compliance guarantee nahi hoti.

2. Configuration and design

Enablement scope, supported Regions, data sources, feature configuration, costs, delegated administration and retention evaluate karein. Findings ka severity, asset owner, triage workflow and response SLA document karein. WAF rules false positives create kar sakte hain; staged testing and rollback plan rakhein.

3. Failures and troubleshooting

Finding aane par evidence preserve karein, principal/resource/account identify karein, exposure assess karein, approved containment implement karein and eradication/recovery verify karein. Broad automatic remediation bina testing important workloads disrupt kar sakti hai. CloudTrail and application logs se context build karein.

4. PRACTICE / DESIGN DRILL

Tabletop: leaked application credentials se unexpected API call. Define incident owner, affected identity, investigation sources, credential containment, evidence storage, clean recovery and post-incident actions. No real compromise simulation without authorization.

Official service guide: Amazon GuardDuty

C22 AWS Backup

Policy, recovery points and restore validation

1. Mental model — kya aur kyun?

AWS Backup selected AWS resource types ke backup management ko centralize karta hai; actual support, feature availability and backup behavior resource-specific hain. Backup plans, vaults, recovery points, copy policies and retention together design karein.

2. Configuration and design

RPO and RTO business requirements se establish karein. Cross-account/Region strategy, encryption keys, role permissions, immutable retention where appropriate and recovery account access check karein. Deletion protection or retention policy production obligations se align karein; unintended retention costs aur removal constraints ho sakte hain.

3. Failures and troubleshooting

Backup job success restore proof nahi. Regular restore drills mein access, target environment creation, key permission, data integrity and application-level verification include karein. Measure total recovery time including detection, decision, restore and cutover; compare actual result with target.

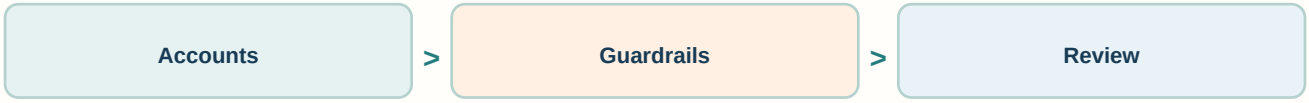
4. PRACTICE / DESIGN DRILL

Exercise: order DB ka isolated restore, validate sample record count and checkout query, record elapsed time and missing permissions. Clean up only disposable restored resources; retain actual production recovery points according to policy.

Official service guide: AWS Backup

C23 Organizations and Control Tower

Multi-account guardrails and governance



1. Mental model — kya aur kyun?

Organizations accounts and organizational units manage karne aur policies apply karne ka framework deta hai. SCPs applicable principals ke maximum permission boundary par guardrail laga sakte hain; SCP khud permission grant nahi karta. Control Tower governed multi-account landing zone workflows aur controls provide karta hai.

2. Configuration and design

Identity federation, account ownership, central audit, baseline security, budgets, network connectivity, backup and emergency access for each account design karein. Dedicated production and nonproduction accounts impact separation mein help kar sakte hain; shared administrator, network or automation common failure domains remain.

3. Failures and troubleshooting

Sudden AccessDenied after policy change: principal and account OU, SCP inheritance, explicit deny, identity policy, resource policy and session context check karein. Guardrail ko test environment mein validate karein; recovery roles ko unintentionally lock out karne se avoid karein.

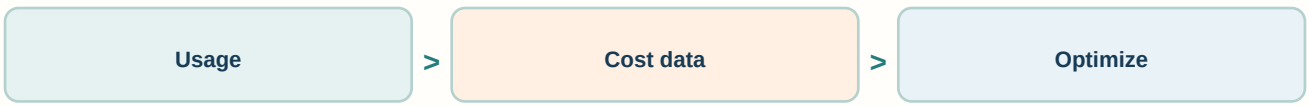
4. PRACTICE / DESIGN DRILL

Paper design: management, security/log archive, shared services, dev and production accounts; draw trust, logging and billing relationships. Define account vending checklist and break-glass process. Never modify organization policies without authorized review.

Official service guide: AWS Organizations

C24 Cost Management

Cost allocation, forecasting and cleanup



1. Mental model — kya aur kyun?

AWS cost multiple dimensions se banta hai: compute time, capacity commitments, storage, requests, transfer, managed service fees, monitoring and other service-specific charges. Cost Explorer and cost allocation tags workload-level analysis mein help kar sakte hain; tag activation and allocation method explicitly configure karein.

2. Configuration and design

Budget thresholds notification/automation options ke saath set ho sakte hain, but budget alert hard spending cap nahi hai. NAT gateway, load balancer, public IPv4, snapshots, logs and retained resources lab cost mein important ho sakte hain. Actual account pricing, currency, region, tax, credits and support plans vary.

3. Failures and troubleshooting

Cost spike: time window, service, linked account, Region and usage type narrow karein. Resource inventory, recent deployments, transfer patterns and retention changes correlate karein. Rightsizing se pehle performance baseline and rollback maintain karein; blind deletion se outage/data loss ho sakta hai.

4. PRACTICE / DESIGN DRILL

Exercise: list each capstone component and estimated monthly usage. Calculate fixed + usage-based components from current AWS pricing calculator; compare estimate versus measured actual after lab. Cleanup by owner and resource inventory; revisit billing for delayed charges.

Official service guide: AWS Billing and Cost Management

C25 Route 53

DNS records, routing and failover

1. Mental model — kya aur kyun?

DNS hosted zone domain ke records manage karti hai. Record type and routing policy decide karte hain resolver ko kaunsa answer milega. TTL caches ko influence karta hai, isliye record update sab clients tak instantly pahunchne ki guarantee nahi.

2. Configuration and design

Public and private hosted zones alag resolution contexts serve karte hain. Routing choice, health checks, domain delegation and certificate hostname requirements document karein. Failover health check ko real application dependencies ke saath align karein.

3. Failures and troubleshooting

Wrong answer: authoritative zone, NS delegation, record type, TTL and resolver cache inspect karein. DNS resolves but site fails to TCP/TLS or application layer troubleshoot karein; DNS success alone checkout success prove nahi karta.

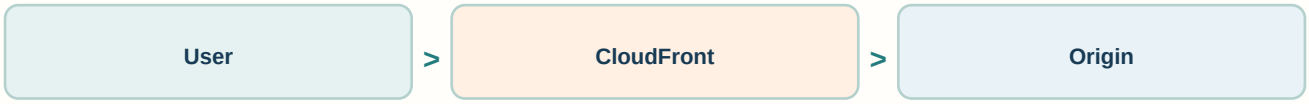
4. PRACTICE / DESIGN DRILL

Paper drill: primary and secondary endpoints, health check, failover policy and expected cached DNS behavior draw karein. Record business impact while cached answers expire.

Official service guide: Amazon Route 53

C26 CloudFront

Edge caching and origin protection



1. Mental model — kya aur kyun?

CloudFront content delivery network hai jo configured origins se content lekar edge locations se serve kar sakta hai. Cache behavior and cache key decide karte hain kaunse requests cached response share karenge.

2. Configuration and design

Private origin access, TLS, allowed methods, cache policy, headers, cookies, query strings and compression evaluate karein. Personalized responses ke liye incorrect cache key data leak cause kar sakti hai. Origin authorization ko independently verify karein.

3. Failures and troubleshooting

Stale content: object version, cache TTL, cache key and origin response headers inspect karein. 403/5xx: origin permissions, DNS, TLS and application response review karein. Invalidation or versioned paths ka cost and operational impact compare karein.

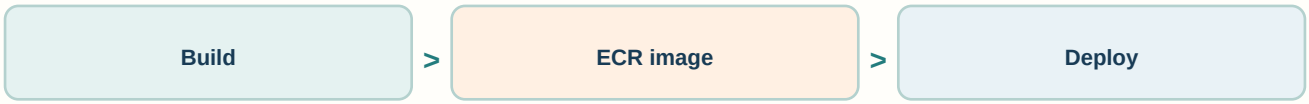
4. PRACTICE / DESIGN DRILL

Example: product images use versioned filenames; private account pages cache na karein unless explicitly safe and tested. Demonstrate two users cannot receive one another’s data.

Official service guide: Amazon CloudFront

C27 ECR

Container image storage and lifecycle



1. Mental model — kya aur kyun?

ECR container image registry provide karta hai. Image ko tag ya immutable digest se reference kiya ja sakta hai; mutable tag latest se exact artifact identify karna difficult ho sakta hai.

2. Configuration and design

Repository access policy, execution role, image scanning capabilities, encryption, lifecycle policy and cross-account or cross-Region needs plan karein. CI pipeline mein trusted base image, dependency scan and deployment digest record karein.

3. Failures and troubleshooting

Image pull failure: registry URI, image existence, IAM authorization, network/DNS path and execution-role permissions inspect karein. Tag overwrite ya image lifecycle deletion deployment rollback ko affect kar sakti hai.

4. PRACTICE / DESIGN DRILL

Build dummy container locally, assign version, record digest and deployment configuration. Any cloud push requires approved repository and cost review; remove only test images and repository after verification.

Official service guide: Amazon Elastic Container Registry

C28 ElastiCache

Cache architecture and stale data

1. Mental model — kya aur kyun?

ElastiCache managed in-memory data stores for caching and other supported use cases offer karta hai. Cache-aside pattern mein application miss par database read karke cache populate karti hai. Cache ko authoritative durable database assume na karein.

2. Configuration and design

TTL, eviction, memory sizing, encryption, authentication, subnet/SG configuration and failover behavior choose karein. Cache key namespacing and data classification important hain. Selected engine and deployment type ki features independently verify karein.

3. Failures and troubleshooting

Cache hit rate drop: key pattern, expiration, evictions, memory pressure and traffic change investigate karein. Cache failure mein fallback database overload ho sakta hai; rate limits and circuit breakers consider karein.

4. PRACTICE / DESIGN DRILL

Product price cache: update price in database, invalidate or update relevant cache entry, then test old vs new value across concurrent users. Inventory reservation should use authoritative conditional write.

Official service guide: Amazon ElastiCache

C29 Athena

SQL analysis on data in S3

1. Mental model — kya aur kyun?

Athena S3 data par SQL querying support karta hai. Data format, partition design, catalog metadata and access permissions query correctness and performance affect karte hain. Different data engines and connectors ke capabilities verify karein.

2. Configuration and design

Dataset permissions, query-result location, encryption, catalog access, workgroup controls and cost limits plan karein. Columnar formats and partition pruning relevant workloads mein scanned data reduce kar sakte hain, but benchmark actual queries.

3. Failures and troubleshooting

Query fails: S3 read permissions, result bucket write access, KMS authorization, table location/schema and partition metadata inspect karein. Unexpected bill: data scanned, format, partition predicates and workgroup settings review karein.

4. PRACTICE / DESIGN DRILL

Dummy orders dataset par count by date query design karein; inspect scanned data and permissions. Cleanup query outputs, sample objects and temporary resources; pricing verify karein.

Official service guide: Amazon Athena

C30 Glue

Data catalog and ETL workflows

1. Mental model — kya aur kyun?

Glue data integration workflows and catalog capabilities provide karta hai. Data source, metadata, transform job and destination alag operational components hain. Schema evolution, record quality and governance analytics pipeline ka core part hain.

2. Configuration and design

Job IAM role, source/destination access, networking, encryption, job bookmarks where applicable, retries and capacity define karein. Glue Data Catalog metadata store karta hai; actual underlying data access bhi separately authorize karna hota hai.

3. Failures and troubleshooting

ETL job failed: run logs, role permissions, schema mismatch, data type conversion and connectivity inspect karein. Data correctness validate karein: success status se complete or accurate transformation guarantee nahi milti.

4. PRACTICE / DESIGN DRILL

Example: raw orders -> validate required ID and amount -> curated output -> quarantine bad records. Record counts and duplicate IDs reconcile karein; delete test jobs and datasets after approved sandbox lab.

Official service guide: AWS Glue

C31 Redshift

Data warehouse design and governance

1. Mental model — kya aur kyun?

Redshift analytics-oriented data warehouse service hai. OLTP transactional database aur analytical warehouse different access patterns ke liye optimize hote hain. Batch ingestion, SQL reporting and concurrency requirements ke context mein choose karein.

2. Configuration and design

Cluster or serverless option, data ingestion, workload management, encryption, IAM/database access and cost controls evaluate karein. Table layout and data access design query profile par depend karte hain; exact recommendations engine features and deployment se verify karein.

3. Failures and troubleshooting

Slow report: query plan, data volume, concurrency, filtering, compute usage and ingestion freshness inspect karein. User cannot query: database grants, IAM integration and network permissions separately evaluate karein.

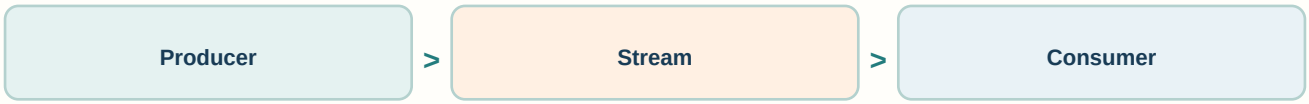
4. PRACTICE / DESIGN DRILL

Design drill: daily order dashboard, raw-to-curated transformation, access boundaries and data freshness SLA. Estimate cost before creating any warehouse; clean up test resources.

Official service guide: Amazon Redshift

C32 Kinesis

Streaming ingestion and consumer design



1. Mental model — kya aur kyun?

Kinesis family streaming data use cases support karti hai; individual services ki retention, processing and delivery features alag hain. Producer sends records; consumers process them and checkpoint progress according to selected integration.

2. Configuration and design

Partition-key distribution, throughput, retention, replay, ordering requirements, monitoring and encryption plan karein. Consumer lag and upstream retry handling matter karte hain. Duplicate delivery and processing may require idempotent consumers.

3. Failures and troubleshooting

Growing lag: producer rate, consumer processing capacity, downstream errors, partition skew and throttling check karein. Replays se duplicates ya out-of-order business effects ho sakte hain, so deduplication and state management design karein.

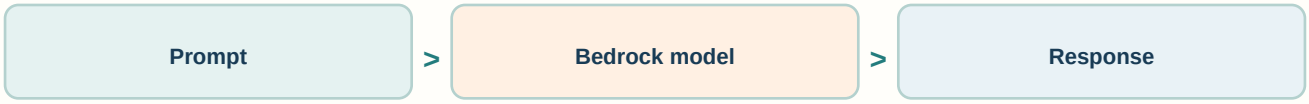
4. PRACTICE / DESIGN DRILL

Example telemetry: device ID partitioning, event timestamp, event ID and late-arrival policy define karein. Simulate a consumer restart and verify that repeated processing is safe.

Official service guide: Amazon Kinesis

C33 Bedrock

Foundation models and responsible generation



1. Mental model — kya aur kyun?

Bedrock managed generative AI capabilities and supported foundation models offer karta hai. Model availability, input/output formats, Region, quotas and pricing vary. Model inference ko deterministic database query jaisa assume na karein.

2. Configuration and design

Choose model against task evaluation, latency, data policy and cost. For retrieval-augmented generation, enforce document access at retrieval, filter sensitive content, manage prompt injection risk and evaluate grounded answers. Application-level authorization remains required.

3. Failures and troubleshooting

Poor answer: inspect retrieved document quality, access filtering, prompt, model settings and evaluation examples. Hallucination and harmful content need layered controls and human review for high-impact decisions. Never log secrets or personal data unnecessarily.

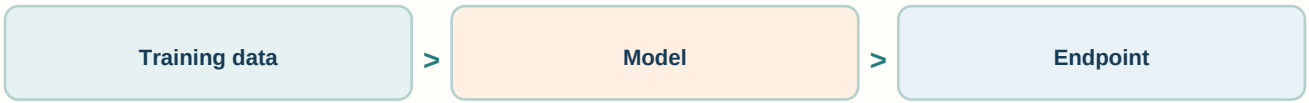
4. PRACTICE / DESIGN DRILL

Build evaluation set with authorized/unauthorized document tests and known answers. Compare correctness, citation quality, latency and cost; do not use confidential data in an unapproved prototype.

Official service guide: Amazon Bedrock

C34 SageMaker AI

Machine-learning lifecycle and operations



1. Mental model — kya aur kyun?

SageMaker AI managed machine-learning lifecycle workflows support karta hai, such as training and deployment in supported configurations. Data preparation, model choice and evaluation still require engineering and domain expertise.

2. Configuration and design

Create representative training, validation and test splits; prevent data leakage and track data/model versions. Compute selection, network isolation, role permissions, endpoint scaling, inference latency and model monitoring plan karein.

3. Failures and troubleshooting

Accuracy drops: compare production input distribution with training data, data pipeline changes, labeling and model version. High inference latency: model size, batching, endpoint capacity and downstream time examine karein.

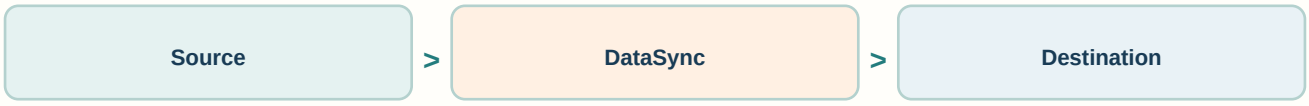
4. PRACTICE / DESIGN DRILL

Design experiment comparing a baseline and candidate model, include offline metrics, slices, privacy checks and rollback threshold. Clean endpoints, training jobs, artifacts and storage after an approved lab.

Official service guide: Amazon SageMaker AI

C35 DataSync

Online data transfer planning



1. Mental model — kya aur kyun?

DataSync online data transfer workflows ke liye available option hai. Source and destination, transfer path, data volume, change rate and verification requirements identify karein. Transfer completion alone application cutover proof nahi hai.

2. Configuration and design

Network connectivity, service access roles, encryption, scheduling, metadata requirements, bandwidth and task verification configure karein. Relevant source/destination support, quotas and current pricing verify karein.

3. Failures and troubleshooting

Failure or mismatch: task logs, source access, destination policy, file permissions, connectivity and verification results inspect karein. Reconciliation aur subsequent deltas cutover plan ka part hone chahiye.

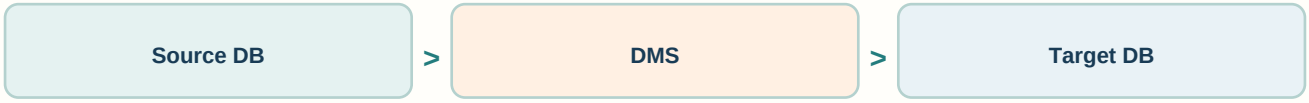
4. PRACTICE / DESIGN DRILL

Estimate baseline transfer time using measured throughput and include overhead, concurrent changes and validation. Test subset first; do not transfer confidential production data without approved controls.

Official service guide: AWS DataSync

C36 Database Migration Service

Full load, change capture and cutover



1. Mental model — kya aur kyun?

DMS database migration workflows mein supported source and target combinations ke liye data movement support karta hai. Schema conversion, application compatibility and cutover validation separate workstreams hain; migration service se all SQL or application code automatically compatible nahi hota.

2. Configuration and design

Source/target versions and network connectivity, schema preparation, full load, change-data capture, data type mappings, encryption and least-privilege credentials evaluate karein. Define downtime, rollback and reconciliation approach before replication begins.

3. Failures and troubleshooting

Lag persists: change rate, target apply capacity, source logs, task errors and network throughput inspect karein. Validate counts, key records and business transactions; matching record counts alone all data correctness prove nahi karte.

4. PRACTICE / DESIGN DRILL

Tabletop cutover: pilot load, measure replication lag, freeze writes where required, reconcile data, switch app and retain rollback path. Dispose of migration infrastructure only after authorized verification.

Official service guide: AWS Database Migration Service

C37 Direct Connect

Dedicated connectivity and resilience

1. Mental model — kya aur kyun?

Direct Connect dedicated network connection to AWS provide karta hai. Capacity, provider, location and virtual interface design workload connectivity requirements se align karein. Dedicated connectivity alone all application paths encrypted hone ki guarantee nahi hai.

2. Configuration and design

Address planning, BGP routing, on-premises devices, multiple connectivity paths, failover and DNS resolution test karein. Application TLS and any network-layer encryption requirements separately define karein. Costs include connectivity and related service charges.

3. Failures and troubleshooting

Connectivity fails: physical/provider status, BGP session, prefixes, virtual interface and route propagation, security controls and return path examine karein. Two connections shared facility/device use karte hon to common failure domain ho sakta hai.

4. PRACTICE / DESIGN DRILL

Draw two paths with separate failure domains and a VPN backup if appropriate. Run failover test plan and define acceptable traffic interruption; coordinate with network owner.

Official service guide: AWS Direct Connect

C38 Cognito

Application identity and user authentication

1. Mental model — kya aur kyun?

Amazon Cognito application user authentication and identity-related capabilities provide karta hai. User pools and identity pools distinct use cases address karte hain; design start karne se pehle users, tokens, identity provider and AWS resource access requirements define karein.

2. Configuration and design

Configure authentication flows, MFA as appropriate, token handling, app client settings, callback URLs and least-privilege role mappings. Application authorization and tenant isolation must still be implemented; valid authentication alone resource access permission nahi hai.

3. Failures and troubleshooting

Login failure: app client, redirect URI, identity provider configuration, token validation and clock-related expiry inspect karein. Access issue: authenticated principal ke authorization claims and backend policy verify karein; tokens ko logs mein reveal na karein.

4. PRACTICE / DESIGN DRILL

Test sign-in, expired token, unauthorized tenant access and account recovery in sandbox with dummy users. Delete test user pool and associated resources according to policy.

Official service guide: Amazon Cognito

C39 Certificate Manager

Certificate lifecycle and TLS termination

1. Mental model — kya aur kyun?

ACM certificates ko provision/manage karne ke supported workflows provide karta hai. Public certificates, private certificates and imported certificates ke lifecycle and renewal responsibilities differ. Certificate ka domain coverage and validation mode application access ko affect karta hai.

2. Configuration and design

Domain validation, Region and supported integration, certificate ownership, renewal monitoring and deployment target plan karein. TLS termination ke baad downstream connection encryption requirement separately define karein. Certificate issuance does not secure vulnerable application code.

3. Failures and troubleshooting

TLS error: hostname match, expiration, trust chain, DNS validation, ALB/CDN attachment and selected Region inspect karein. Imported certificate renewal may require manual operational process; confirm actual product behavior.

4. PRACTICE / DESIGN DRILL

Draw browser -> TLS termination -> backend encryption path. Document renewal owner and alerting. Never expose private keys in notes, tickets or repositories.

Official service guide: AWS Certificate Manager

C40 WAF

Application-layer request filtering

1. Mental model — kya aur kyun?

AWS WAF web requests ke filtering controls provide karta hai for supported integrations. Rules and rule groups request attributes inspect karke allow, block, count or other configured actions use kar sakte hain. WAF is not a substitute for secure application code, identity controls or network segmentation.

2. Configuration and design

Web ACL, rule priority, managed/custom rules, rate controls, logging and privacy requirements define karein. Start with observe/count where suitable, test false positives and document rollback. Managed rule behavior and costs current documentation se verify karein.

3. Failures and troubleshooting

Legitimate users blocked: match logs, rule priority, client characteristics and recent change inspect karein; narrowly tune exception rather than disabling all protections. Attack traffic increase: correlate WAF metrics with origin and application signals.

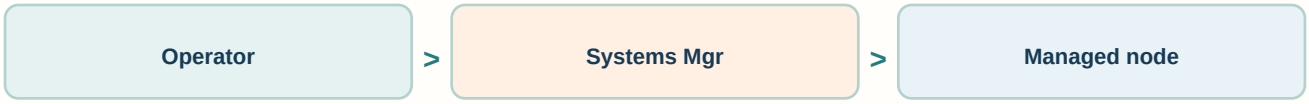
4. PRACTICE / DESIGN DRILL

Exercise: create staging rule with test traffic, monitor count results, define false-positive threshold and rollback. Live deployment only in authorized environment and after cost/security review.

Official service guide: AWS WAF

C41 Systems Manager

Managed-node operations and automation



1. Mental model — kya aur kyun?

Systems Manager managed nodes par operational actions, inventory and supported automation workflows support karta hai. Session Manager can provide controlled shell access without requiring inbound SSH when prerequisites are met, but IAM, agent, node registration and network connectivity still matter.

2. Configuration and design

Managed-node role/profile, agent installation/version, service endpoints or egress path, logging, access permissions and session controls plan karein. Patch management requires maintenance schedule, testing, rollback and evidence; automation is not a substitute for change approval.

3. Failures and troubleshooting

Node not managed: instance role, agent status, VPC endpoint/DNS or internet route and Region/account verify karein. Session denied: caller IAM permission and resource conditions check karein. Commands logged by the system must not contain secrets.

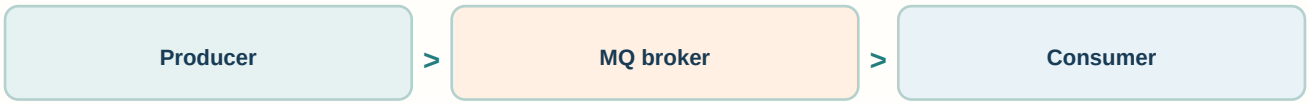
4. PRACTICE / DESIGN DRILL

On an approved disposable node, verify managed-node status, open a controlled session and inspect OS. Terminate test instance, remove lab-specific resources and review logs and costs.

Official service guide: AWS Systems Manager

C42 Amazon MQ

Managed message brokers and migration



1. Mental model — kya aur kyun?

Amazon MQ managed message broker service hai for supported engines/protocols. Existing application compatibility aur broker-specific semantics workload design mein relevant hain; SQS, SNS and EventBridge se operational and delivery models alag hain.

2. Configuration and design

Engine/version, network exposure, credentials, encryption, deployment topology, broker capacity, monitoring and backup/recovery plan review karein. Broker supported protocol ka matlab application-specific features ke liye complete compatibility assume nahi karni chahiye.

3. Failures and troubleshooting

Publish/consume failure: broker status, security group, DNS, credentials, protocol settings, connection limits and consumer lag inspect karein. Retry storm or poison message handling business logic and broker configuration dono par depend karta hai.

4. PRACTICE / DESIGN DRILL

Migration exercise: inventory producers and consumers, message ordering and acknowledgement semantics, durability requirements and cutover rollback. Test duplicates and broker failover with synthetic messages before production migration.

Official service guide: Amazon MQ

14 Hands-on lab: IAM reasoning

Policy ko read karke permissions predict karein.

Goal aur safety

Account mein credentials share kiye bina ek narrow read-only permission ko samajhna. IAM Policy Simulator ya equivalent approved test identity use karein. Production permissions change na karein.

Steps

- 1. Topic 030 ka policy snippet padhein. Principal, effect, action aur resource identify karein.
- 2. Substitute example bucket name with a test bucket that you control.
- 3. Predict: reports/a.txt par GetObject allowed? images/a.txt par? Bucket ListBucket?
- 4. Simulator mein applicable context ke saath test karein. Agar actual request test kar rahe hain, bucket policy, KMS aur boundaries bhi account karein.
- 5. Results aur reason likhein; lab policy aur test objects remove karein.

EXPECTED REASONING

Identity policy example reports/* par GetObject allow karti hai. Images prefix aur ListBucket ko yeh statement allow nahi karti. Actual outcome other applicable policies, conditions, encryption and explicit denies par depend karega.

15 Hands-on lab: private S3

Ek small object ke through storage aur cleanup seekhein.

Before: current Region, access permissions, account plan aur S3 pricing check karein. Budget alert lagayein. Dummy text file use karein, real confidential data nahi.

• 1. S3 console mein unique test bucket create karein aur selected Region confirm karein.

• 2. Block Public Access enabled verify karein. Bucket policy mein public access add na karein.

• 3. test.txt upload karein. Authorized identity se object download karke content compare karein.

• 4. Versioning on karne par file ka second version upload karke version list inspect karein.

• 5. Cleanup: all object versions AND delete markers remove karein; bucket empty hone ke baad bucket delete karein.

• 6. Resource inventory aur billing dashboard baad mein verify karein.

WHAT CAN FAIL?

AccessDenied: identity permissions ya bucket policy check karein; KMS-encrypted objects par key permissions bhi relevant ho sakti hain. Delete bucket failure: old versions aur delete markers check karein.

Official reference: https://docs.aws.amazon.com/AmazonS3/latest/userguide/Versioning.html

16 Hands-on lab: EC2 checklist

A temporary instance ka lifecycle safely samjhein.

Before: approved account, budget alert, Region, IAM permissions, AMI aur instance pricing confirm karein. Systems Manager use karna ho to agent, role aur endpoint/network prerequisites verify karein. Open SSH from everywhere use na karein.

• 1. Suitable supported Linux AMI aur workload-compatible instance type select karein.

• 2. Network, subnet aur SG ka reason likhein; IAM role ko sirf required permissions dein.

• 3. Instance launch karein aur approved access method se connect karein.

• 4. OS, disk aur service status non-destructive commands se inspect karein.

• 5. Stop aur terminate ka difference apne words mein likhein; final lab mein instance terminate karein.

• 6. EBS volumes, snapshots, IP addresses aur other chargeable resources independently check/delete karein; billing revisit karein.

uname -a df -h free -h ps aux | head

TROUBLESHOOT

Access fail ho to correct account/Region, instance state, role, approved access prerequisites, network routes aur SG rules evidence se verify karein. Blindly 0.0.0.0/0 open na karein.

17 Capstone: online shop architecture

Ek realistic design banayein, phir controlled phases mein implement karein.

Requirements first

Workload: customers product pages browse karein aur orders place karein. Requirements define karein: expected users, availability target, sensitive data, maximum budget, RPO aur RTO. Numbers aapke scenario ke honge; unhein guess karke production design approve na karein.

Proposed conceptual architecture

• Entry: DNS and optional CDN; TLS certificate aur access controls.

• App: ALB aur app instances across multiple AZs; health checks aur scaling policy.

• Data: private relational database for order transactions; S3 for product images; encryption and role-based access.

• Operations: application logs, metrics, alerts, CloudTrail and documented incident process.

• Recovery: backups, restore drills, deployment rollback and defined RPO/RTO.

Acceptance tests

• Authorized user product page aur checkout use kar sakta hai; unauthorized access denied.

• One app target unhealthy hone par routing behavior expected hai.

• Backup se restore drill documented hai; achieved recovery time targets se compare hua.

• Cost estimate includes compute, database, NAT, load balancer, requests, storage, logs and transfer.

• IaC changes reviewed hain; lab resources delete hone ke baad billing review scheduled hai.

REALITY CHECK

Yeh design blueprint hai, fully deployed implementation nahi. Production use ke liye threat modeling, load tests, actual runbooks, compliance review aur independent technical review karein.

18 Revision questions + answers

Khud answer dein, phir neeche explanation se compare karein.

Public subnet ka matlab public instance?

Nahi. Public route table mein IGW route hota hai; instance ke liye public IP aur appropriate rules bhi important hain.

SG aur NACL mein core difference?

SG stateful interface-level control; NACL subnet-level stateless control hai.

Role aur policy same?

Nahi. Role assumable identity hai; trust policy who-can-assume aur permissions policies allowed actions define karti hain.

Multi-AZ backup ka replacement?

Nahi. Availability aur backup different failure scenarios address karte hain; restore testing bhi zaruri hai.

Budget alert hard cap hai?

Nahi. Alert notification hai; charges automatically stop hone ki guarantee nahi.

S3 versioning delete cleanup?

Current object delete karna enough nahi; all versions aur delete markers remove karein.

Aage practice kaise karein

1. Draw: bina notes dekhe ek two-AZ web application ka sketch banao. Internet-facing aur private components alag dikhao; har arrow par port aur direction likho.

2. Explain: IAM role ka trust aur permissions alag samjhao. S3 versioning, backup aur Multi-AZ availability ka farq ek-ek sentence mein batao.

3. Test safely: approved sandbox mein budget alert, least-privilege permissions aur cleanup checklist ke saath practice karo. Restore aur incident response drills likhkar review karo.

4. Update: Service choose karne se pehle AWS documentation, Region support, quotas aur actual pricing check karo. Apne workload ke liye reliability, performance aur cost ke trade-offs note karo.

Final challenge: online shop ko samjho

Situation: Sale shuru hote hi visitors badh gaye, ek application server fail ho gaya aur product image galti se overwrite ho gayi. Team ko customer impact aur cost dono dekhne hain.

Socho 1: Load balancer healthy targets ko traffic bhej sakta hai aur Auto Scaling configured capacity maintain kar sakta hai. Lekin database aur baaki dependencies bhi load handle karti hain ya nahi, test karna hoga.

Socho 2: Agar S3 versioning pehle se enabled thi aur purani version available hai, authorised user usse recover karne ka option dekh sakta hai. Versioning ko independent backup ka complete replacement mat samjho.

Socho 3: CloudWatch signals aur application logs se failure investigate karo. Billing aur resource inventory mein idle capacity check karo; investigation ke naam par broad permissions ya public access mat kholo.